Enhancing Automated Unit Test Generation with Large Language Models: A Systematic Literature Review

JUNWEI ZHANG, The State Key Laboratory of Blockchain and Data Security, Zhejiang University, China and Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security, China
XING HU*, School of Software Technology, Zhejiang University, China
CUIYUN GAO, Harbin Institute of Technology, China
XIN XIA, College of Computer Science and Technology, Zhejiang University, China and Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security, China
SHANPING LI, College of Computer Science and Technology, Zhejiang University, China

Automated unit test generation is a fundamental yet challenging task in software engineering, playing a critical role in ensuring software correctness, reliability, and maintainability. While traditional approaches such as search-based software testing and symbolic execution have achieved notable success, they often suffer from limited semantic understanding, high configuration costs, and scalability constraints. Recent advances in Large Language Models (LLMs) have fundamentally reshaped the landscape of automated unit testing by enabling models to reason over source code semantics and generate executable, context-aware test cases. Despite the rapid growth of this research area, a comprehensive and task-oriented synthesis of existing work remains lacking. This paper presents a systematic literature review of LLM-based unit test generation. This review draws on research from leading SE and AI conferences and journals, including 69 papers published across 25 distinct venues, along with 47 high-quality preprint papers, bringing the total to 116. Our review aims to answer three key research questions: (1) which unit testing tasks have been addressed using LLMs, (2) how LLMs are adapted and integrated into the unit test generation pipeline, and (3) what datasets, benchmarks, and evaluation practices are employed in existing studies. To this end, we organize the literature from a task-centric perspective, covering test generation, test input generation, test oracle generation, and test evolution, and from a methodological perspective, categorizing LLM adaptation strategies into fine-tuning, prompt engineering, and agent-based approaches.

Our analysis reveals that current research predominantly focuses on function- and class-level test generation, with comparatively limited attention given to test input generation, oracle construction, and long-term test evolution. Decoder-only LLMs, particularly GPT-family and LLaMA-based models, dominate the field, while encoder-only and encoder–decoder models remain underexplored. We further observe substantial disparities in dataset characteristics, programming language coverage, and evaluation metrics, which hinder fair comparison and reproducibility across studies. Based on empirical evidence extracted from the surveyed literature, we identify key challenges facing LLM-based unit test generation. Building on these findings, we outline several promising research directions, such as dataset optimization, structure-aware context modeling, agent

*The corresponding author.

Authors' Contact Information: Junwei Zhang, The State Key Laboratory of Blockchain and Data Security, Zhejiang University, China and Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security, China, jw.zhang@zju.edu.cn; Xing Hu, School of Software Technology, Zhejiang University, China, xinghu@zju.edu.cn; Cuiyun Gao, Harbin Institute of Technology, Shenzhen, China, gaocuiyun@hit.edu.cn; Xin Xia, College of Computer Science and Technology, Zhejiang University, China and Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security, China, xin.xia@acm.org; Shanping Li, College of Computer Science and Technology, Zhejiang University, China, shan@zju.edu.cn.

ACM 1557-7392/2026/3-ART
https://doi.org/10.1145/3802827

coordination mechanisms, and benchmark enhancement. This review provides a consolidated and evidence-driven foundation for future research, aiming to advance the development of scalable, reliable, and practically applicable LLM-driven unit testing techniques.

CCS Concepts: • **Software and its engineering → Software testing and debugging**.

Additional Key Words and Phrases: Large Language Models, Unit Test Generation, Literature Review

1 Introduction

Unit testing plays a crucial role in ensuring the correctness and reliability of software systems by verifying, as it verifies the behavior of individual components or units of code [148]. To improve productivity in test development and reduce human errors, a variety of automated unit test generation techniques have been introduced, including Randoop [135] and EvoSuite [42]. Hence, unit test generation has remained an active research topic, receiving extensive attention over the past decades and continuing to attract sustained interest from both academia and industry [7, 85, 111, 158, 167, 198]. Recent advances in Large Language Models (LLMs) have fundamentally transformed the paradigm of automated test generation [5, 157, 174]. Unlike traditional methods, LLMs possess the ability to comprehend code semantics, enabling the automatic generation of more meaningful and context-aware test cases [72, 98, 175]. For instance, Chen et al. [20] employed GPT-3.5 to automatically generate unit tests. By leveraging large-scale training corpora and inferring the underlying intent of code snippets, LLM-based techniques provide enhanced flexibility, scalability, and accuracy in test generation [133, 171]. By automating the creation of diverse, high-coverage, and bug-revealing test cases, these approaches address long-standing challenges in software quality assurance and improve the efficiency of modern development workflows.

Although research on leveraging LLMs for unit test generation has advanced rapidly, to the best of our knowledge, there is still no comprehensive survey that currently exists that systematically synthesizes state-of-the-art methods, identifies current limitations, and delineates promising future directions. To bridge this gap and provide actionable insights for the research community, we conducted a systematic literature review encompassing 116 primary studies published between January 2021 and December 2025. Specifically, this paper first summarizes the range of LLMs employed in these studies and examines the specific components of unit tests targeted for generation. We then categorize the adaptation strategies adopted to tailor LLMs for test generation and review the datasets and benchmarks used for empirical evaluation. Finally, we analyze the limitations of existing approaches and outline future research opportunities, thereby charting potential pathways for advancing the field of LLM-based unit test generation.

Recent survey studies have examined the application of LLMs across various software engineering (SE) tasks, including code generation [80], program repair [203], and vulnerability detection [161]. These works underscore the growing impact of LLMs in automating and advancing traditional SE practices. For example, Jiang et al. [80] conducted a systematic literature review of 235 papers on the use of LLMs for code generation. Hou et al. [68] provided a broader systematic literature review on LLM applications in SE. However, given the wide scope of SE tasks, their review did not provide a fine-grained categorization of LLM utilization in software testing. In contrast, our study focuses on unit test generation with LLMs, offering a more detailed and targeted synthesis of this emerging research area.

Several survey papers have been published on topics related to software testing, each addressing different aspects of the field [80, 162, 180, 218, 227]. For instance, Zhang et al. [227] conducted the first comprehensive survey dedicated to testing deep learning (DL) libraries. Their work defines DL library bugs, categorizes testing properties, and summarizes testing methodologies across three key components: DL frameworks, DL compilers, and DL hardware libraries. They systematically review 93 studies, highlighting techniques such as differential testing, fuzzing, and metamorphic testing. Similarly, Wang et al. [180] provided an extensive review of 102 studies on the application of LLMs to software testing. Their survey categorizes applications across the software testing

lifecycle, with particular emphasis on unit test generation, test oracle generation, system test input generation, bug analysis, debugging, and program repair. Zhang et al. [218] offered another comprehensive survey of testing techniques for machine learning systems.

These surveys establish a solid foundation for understanding testing methodologies in diverse contexts. However, they also reveal a notable gap: the absence of a comprehensive review dedicated specifically to the application of LLMs in unit test generation. To address this, our literature review synthesizes studies published up to December 2025, and focuses on LLMs, encompassing 116 primary studies that employ 37 distinct LLMs. More recently, Zhang et al. [221] surveyed LLM-based unit testing, which covered unit test generation as part of its study scope. Distinct from their work, our review jointly categorizes the literature based on unit test sub-tasks (input generation, oracle generation, test evolution, etc.) and LLM adaptation strategies (fine-tuning, prompt engineering, agentic workflows). Besides, our survey systematically extracts challenges from empirical evidence reported in 116 studies, quantifying trends (e.g., technique adoption rates, dataset characteristics, metric usage) and linking challenges to observed performance issues. We propose solution-oriented strategies, including multi-level dataset alignment, context compression, MoE-style routing, agent coordination mechanisms, and benchmark redesign. Specifically, this survey first organizes existing studies according to the test scenarios in which LLMs have been applied, including test generation, test input generation, oracle generation, and test evolution. This scenario-driven perspective reflects the core activities required to construct and maintain executable test suites. Previous work [221] also organizes the literature according to similar scenario-based categories. Following this, we analyze the model adaptation strategies employed to support these scenarios, categorizing existing techniques into three dominant paradigms: fine-tuning, prompt engineering, and agent-based methods. This categorization highlights how researchers adapt LLMs to unit test generation tasks. We then examine the evaluation practices adopted in prior work, including the metrics and benchmarks used to assess the quality, robustness, and practicality of LLM-generated tests. This analysis allows us to identify gaps in opportunities for improving evaluation standardization and reproducibility. Finally, we synthesize the main challenges and potential solutions in LLM-based unit test generation to inform future work and guide the development of more effective, scalable, and trustworthy LLM-driven testing techniques.

In summary, this study makes the following contributions:

- We conduct a systematic review of 116 primary studies that investigate the utilization of LLMs for unit test generation.
- We map the task landscape of LLM-assisted unit testing, spanning test generation, input generation, oracle generation, and test evolution.
- We provide an in-depth summary of the LLMs employed in the literature and categorize the diverse techniques used to adapt these models for unit test generation.
- We analyze the limitations of existing LLM-based approaches and propose several promising avenues for future research.

Survey Structure. Figure 1 presents the overall organization of this paper. Section 2 introduces the background and preliminaries. Section 3 describes our survey methodology and surveys the LLMs employed for unit test generation. Section 4 analyzes LLM usage from different unit test scenarios. Section 5 reviews techniques for adapting and enhancing LLMs to this task. Section 6 examines datasets and evaluation benchmarks in LLM-based unit test generation. Section 7 outlines the limitations of current studies and identifies opportunities for future research. Section 8 presents a discussion that compares LLM-based test generation with traditional techniques, outlines key implications for research and practice, and describes the limitations and threats to validity of this review. Finally, Section 9 summarizes our contributions and outlines future research directions of this paper.

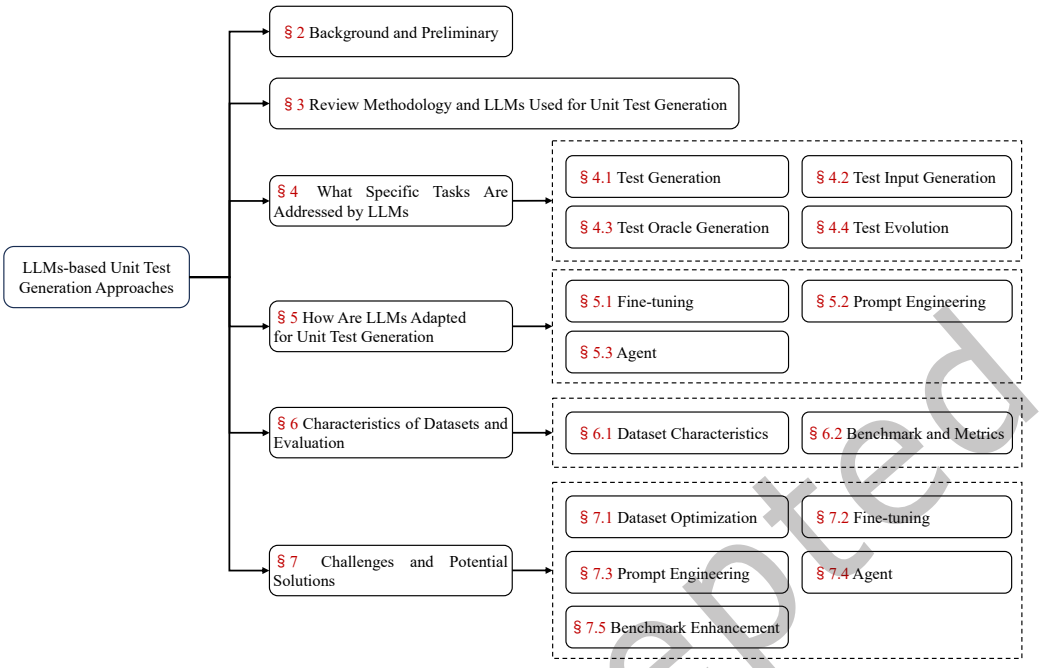

Fig. 1. The structure of this paper.

2 Background and Preliminaries

In this section, we introduce the core concepts relevant to this work, including unit testing, LLMs, and related surveys.

2.1 Unit Test Generation Formulation

Unit test generation can be formally defined as the process of constructing, for a target program under test P and a function or method $f \in P$, a set of test cases $T = \{t_1, t_2, \ldots, t_n\}$. Each test case t_i comprises three fundamental components: (1) an input configuration I_i, which specifies the concrete values or objects supplied to f; (2) an invocation statement C_i, which executes f with the given inputs; and (3) an expected oracle O_i, which specifies the anticipated output or the observable program state after execution. The generation task can be conceptualized as a mapping

$$G : (P, f) \mapsto T,$$

where G is a unit test generator that produces a set of test cases satisfying predefined adequacy criteria, such as statement coverage, branch coverage, or bug detection capability.

Traditional approaches rely on search-based strategies or symbolic execution techniques to explore the input space and automatically infer test oracles [42, 135]. In contrast, recent advances reformulate unit test generation as a conditional code generation problem, where the generator G is parameterized by the weights of an LLM and guided through prompts or fine-tuned objectives. This paradigm shift facilitates the synthesis of test cases that not only achieve structural coverage but also leverage the semantic understanding embedded within large-scale code corpora.

2.2 Large Language Models (LLMs)

Large Language Models (LLMs) are large-scale neural networks trained on massive text or code corpora using self-supervised objectives such as next-token prediction or masked language modeling [231]. With billions of parameters, they capture complex statistical and semantic patterns, enabling generalization across a wide range of tasks [180, 240]. Based on architectural design, LLMs can be broadly categorized into three paradigms: encoder-only, decoder-only, and encoder–decoder models [138].

Encoder-only LLMs employ only the Transformer encoder to generate bidirectional contextual representations [36]. Representative encoder-only LLMs include CodeBERT [41], GraphCodeBERT [57], and CuBERT [84], which focus on learning rich bidirectional code representations. Recent variants such as VulBERTa [58], CCBERT [241], and SOBERT [61] extend this paradigm to specialized software engineering tasks. **Encoder–decoder LLMs** combine both encoder and decoder components, where the encoder generates semantic representations and the decoder produces conditioned outputs [177]. Common examples of encoder-decoder LLMs include PLBART [2], T5 [147], CodeT5 [188], and UniXcoder [56]. **Decoder-only LLMs** rely solely on the Transformer decoder and are trained autoregressively [146]. Representative decoder-only LLMs include the GPT series (e.g., GPT-2 [146], GPT-3 [13], GPT-3.5 [53], and GPT-4 [132]), which demonstrate strong general-purpose text generation capabilities. In the domain of code intelligence, notable models include CodeGPT [119], Codex [19], Polycoder [200], Incoder [44], the CodeGen series [128, 129], Copilot [49], CodeLlama [152], and StarCoder [103]. These three architectural paradigms illustrate the diversity of design strategies that underpin the functionality and application scope of LLMs.

The lifecycle of LLMs comprises four stages: pre-training, post-training, inference, and evaluation [235]. **Pre-training** constitutes the foundation, where models are trained on large-scale text or multimodal corpora using self-supervised objectives such as masked language modeling or next-token prediction [177]. **Post-training** adapts LLMs for specific tasks through supervised fine-tuning on labeled datasets [70] or alignment methods such as reinforcement learning from human feedback (RLHF) [23]. **Inference** denotes the deployment phase, during which models generate outputs via decoding strategies (e.g., greedy search [155] and beam search [17]), potentially enhanced by advanced prompting [165] or reasoning techniques [74]. Finally, **evaluation** measures model performance across multiple dimensions, including task accuracy, robustness, fairness, and efficiency [73]. These four stages define the end-to-end pipeline by which LLMs are trained, adapted, deployed, and assessed in practice.

2.3 Related Surveys

Recent years have witnessed a growing number of surveys and systematic reviews on the use of large language models in software testing, reflecting increasing academic and industrial interest in AI-driven automation. In reviewing the existing literature, we categorize related work into two broad streams, as shown in Table 1. The first line of research comprises surveys that focus on software engineering tasks broadly enabled by LLMs. These works provide high-level overviews of how LLMs are being integrated into software development workflows, but they typically treat testing as only one of many downstream applications, resulting in limited task-specific depth. For example, Wang et al. [181] provide a comprehensive investigation into automated unit test case generation, with a particular emphasis on search-based software testing (SBST) techniques and evolutionary algorithms. The review synthesizes 63 selected primary studies and analyses three major dimensions of automated testing research: (1) the test adequacy criteria used to evaluate generated tests, (2) the automated test generation techniques and tool implementations, and (3) the performance, improvements, and persistent challenges associated with these methods. Wang et al. [180] review 102 papers published between 2019 and 2023. They analyze software testing from two complementary perspectives: (1) testing tasks, and (2) LLM techniques. From the testing perspective, the authors categorize and examine how LLMs are applied across a wide range of tasks, including unit test case

generation, test oracle generation, and system test input generation. From the LLM perspective, the authors analyze the types of models, prompt engineering strategies, input formats, and auxiliary techniques. Zhang et al. [221] conduct a comprehensive systematic literature review of LLMs applied to unit testing, analyzing 105 papers from 2020 to 2025. It surveys a broad spectrum of unit testing activities, including test generation, oracle generation, test evolution, test repair, test readability, test smell detection, and test-to-code traceability. Further, the authors examine how LLMs are utilized through techniques such as fine-tuning, prompt engineering, and hybrid integration with traditional program analysis tools. Chu et al. [24] analyze 115 studies published between 2021 and 2025. They examine multiple aspects of LLM-enabled software testing, including test prefix and oracle generation, test evolution, validation and repair, mutation-based enhancement, and test smell detection, and further investigate how software engineering techniques such as static/dynamic analysis, feedback loops, and hybrid integration with traditional tools are used to improve test effectiveness. This review also identifies key challenges related to bug detection capability, data and benchmark limitations, and alignment with industrial practice, outlining future research opportunities. Celik et al. [16] examine how LLMs are integrated with traditional testing workflows, highlighting their impact on test development efficiency, bug detection effectiveness, and automated fault localization. This review identifies opportunities for improving software testing reliability and productivity using LLMs. Prates et al. [123] examine how LLMs are applied across several key software testing activities. It also investigates the role of LLMs in improving bug detection accuracy, debugging support, and fault localization, emphasizing their contribution to the efficiency and quality of testing workflows.

The second line of research includes surveys and systematic reviews that specifically target software testing with LLMs, particularly automated unit test generation. These studies examine how LLMs support test input generation, oracle construction, or test case synthesis, and begin to explore their strengths, limitations, and evaluation practices within testing scenarios. For instance, Fan et al. [40] investigate how LLMs are applied across diverse software engineering tasks, with an emphasis on automating and assisting activities throughout the development lifecycle. It reviews applications of LLMs in areas such as code generation and refinement, bug detection and program repair, fault localization, test generation, and specification mining. Zheng et al. [237] review a wide range of techniques that leverage LLMs to enhance software development productivity, reliability, and reasoning ability, often combining them with static or dynamic program analysis. This study also highlights emerging opportunities and challenges in applying LLMs to real-world SE practices, including scalability, robustness, and human–AI collaboration. Hou et al. [68] provide a comprehensive review of how LLMs are being adopted in software engineering tasks, with coverage spanning code generation, program repair, bug detection, fault localization, and test automation. It further examines enabling techniques such as prompting, fine-tuning, and hybrid integration with program analysis, evaluating their effectiveness, limitations, and impact on productivity and reliability. Zhang et al. [222] systematically categorize 947 studies covering 112 code-related tasks, ranging from code generation, summarization, fault localization, and program repair to testing tasks such as fuzzing, mutation testing, and vulnerability detection. In addition, the review discusses emerging practices for evaluation, benchmarking, security, reliability, domain adaptation, and model compression, highlighting both trends and open challenges for integrating LLMs into SE workflows. Wang et al. [191] review 115 studies and analyze the integration of LLM-based agents across multiple SE dimensions, including input modalities, memory, and action. They further examine applications to a broad range of SE tasks and identify key challenges and future opportunities for advancing LLM-driven agents within SE.

Our survey distinguishes itself from prior work by providing the first comprehensive and task-oriented synthesis of LLM-based unit test generation, covering the full spectrum of unit testing activities. In contrast, earlier surveys often focus on only one or two components of the testing pipeline, with most reviews examining either test generation alone or broader software engineering/testing tasks where unit testing is treated peripherally. Moreover, while existing surveys primarily emphasize fine-tuning and prompt engineering techniques, our review systematically analyzes multiple LLM adaptation strategies (i.e., fine-tuning, prompt engineering, and agent-based

approaches). Besides, we examine how they differ in objectives, capabilities, and limitations. Additionally, unlike earlier work that reports findings at a high level, our survey integrates evidence-based insights from 116 studies (2021–2025), identifying cross-cutting challenges and proposing solution-oriented research directions grounded in empirical observations.

Table 1. Comparison of related surveys and our work.

Surveys	Year	#Papers	Covered Unit Test Component				Covered LLM Techniques			Time Frame
			Test Input Generation	Test Oracle Generation	Unit Test Generation	Unit Test Evolution	Fine-Tuning	Prompt Engineering	Agentic	
Wang et al. [181]	2025	63	✓	✓	✓	×	×	×	×	1990-2023
Wang et al. [180]	2024	102	✓	✓	✓	×	✓	✓	×	2019-2023
Zhang et al. [221]	2025	105	×	✓	✓	✓	✓	✓	×	2020-2025
Chu et al. [24]	2025	115	×	×	✓	×	✓	✓	×	2021-2025
Celik et al. [16]	2025	84	×	×	✓	×	✓	✓	×	2022-2025
Prates et al. [123]	2025	20	×	×	✓	×	✓	✓	×	2021-2024
Fan et al. [40]	2023	-	×	×	✓	✓	✓	✓	×	2007-2023
Zheng et al. [237]	2023	123	×	×	✓	✓	✓	✓	×	2022-2024
Hou et al. [68]	2023	395	×	×	✓	×	✓	✓	×	2017-2023
Zhang et al. [222]	2024	1009	✓	✓	✓	×	✓	✓	×	2017-2024
Wang et al. [191]	2025	115	✓	×	×	×	×	×	✓	2020-2024
Our survey	2025	119	✓	✓	✓	✓	✓	✓	✓	2021-2025

3 Review Methodology

This section first introduces the central research questions guiding our study, outlines the search strategy for identifying relevant literature, and specifies the criteria for assessing the eligibility and quality of selected studies. We then analyze the collected papers from multiple perspectives, including their temporal distribution and dissemination across publication venues, to provide a comprehensive overview of the research landscape.

3.1 Research Question

In this paper, we investigate three Research Questions (RQs) designed to provide a structured examination of the role of LLMs in unit test generation. Specifically, we seek to answer the following questions.

- **RQ1: What specific tasks within unit test generation have been addressed by these models?**
- **RQ2: How have LLMs been adapted or applied to the unit test generation process?**
- **RQ3: What datasets and evaluation strategies have been employed in existing studies?**

These questions establish a comprehensive framework for examining the current state of research and identifying directions for future advancement.

3.2 Search Strategy

Our search methodology builds on the work of Zhang et al. [215], who introduced the first code-oriented LLM, CodeBERT, in November 2020. Given this milestone, we focus on papers published between January 2021 and December 2025. To ensure coverage of high-quality studies, we first manually reviewed influential, peer-reviewed conferences and journals in the SE domain, including seven major conferences (ICSE, FSE, ASE, ISSTA, ICSME, MSR, and ISSRE) and four top journals (TOSEM, TSE, IST, and JSS), as shown in Table 2. A manual search across these venues yields 133 papers directly relevant to our research objectives. We then conduct automated searches across four widely used academic databases: IEEE Xplore [31], ACM Digital Library [107], Web of Science [130], and arXiv [9]. The search queries for this automated process are formulated based on the relevant

papers identified during the manual search, as listed in Table 3. In contrast to previous surveys [215], which primarily rely on keyword searches or restrict their scope to earlier time periods, our approach combines venue-driven identification, systematic database querying, and iterative refinement of queries based on seed papers, allowing us to retrieve a substantially larger and more current set of candidate studies (8,347 initial records).

Table 2. Publication venues for conference proceedings and journal articles for manual search.

Conference/Journal	Venue	Acronym
Conference	International Conference on Software Engineering	**ICSE**
	ACM International Conference on the Foundations of Software Engineering	**FSE**
	International Conference on Automated Software Engineering	**ASE**
	International Symposium on Software Testing and Analysis	**ISSTA**
	IEEE International Conference on Software Maintenance and Evolution	**ICSME**
	International Mining Software Repositories Conference	**MSR**
	IEEE International Symposium on Software Reliability Engineering	**ISSRE**
Journal	Transactions on Software Engineering	**TSE**
	Transactions on Software Engineering and Methodology	**TOSEM**
	Information and Software Technology	**IST**
	Journal of Systems and Software	**JSS**

Table 3. Keywords related to LLMs and unit test generation task for automated search.

Keywords Related to LLMs	Keywords Related to Unit Test Generation Task
large language model, LLM, language model, LM, pre-trained language model, PLM, transformer, GPT-3, GPT-4, LLaMA, CodeLlama, PaLM, CodeT5, CodeGen	unit test, test case, test generation, test repair, test evolution, assert, oracle, test reduction, test selection

3.3 Study Selection

To ensure the rigor and relevance of the reviewed literature, we follow a systematic study selection procedure, as shown in Figure 2. After retrieving papers through the predefined search strategy, we remove duplicates and exclude non-English or non-peer-reviewed publications. Titles and abstracts are then screened to eliminate studies unrelated to LLMs or unit test generation. Finally, the remaining papers are evaluated through a full-text assessment against predefined inclusion and exclusion criteria to determine their eligibility.

Inclusion Criteria. Studies are included if they meet all of the following conditions: (1) The paper is written in English; (2) The full text is accessible; (3) The paper is a peer-reviewed, full-length research article published in a recognized conference proceeding or journal; and (4) The study explicitly employs LLM techniques to address unit test generation tasks or investigates evaluation methodologies specifically designed for LLMs-based unit test generation.

Exclusion Criteria. Studies are excluded if any of the following applied: (1) The paper contains fewer than five pages; (2) The publication type is a book, keynote, panel summary, technical report, thesis, tool demonstration, editorial, or originated from a non–peer-reviewed venue; (3) The paper is a literature review or survey; (4) The study is a duplicate or a substantially similar work authored by the same research group; (5) The approach does not involve LLMs; (6) LLMs are mentioned only in the discussion or future work without integrating them into the methodology; or (7) The study does not address unit test generation tasks.

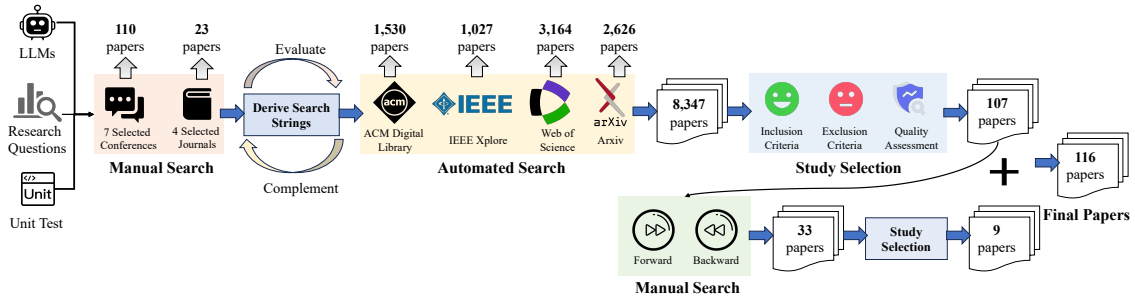

Fig. 2. Study identification and selection process.

This multi-stage filtering process yields a final corpus of 117 primary studies, providing a representative and comprehensive foundation for analysis. Preprints on arXiv were retained if recently released and not yet formally published.

Quality Assessment. To reduce bias from low-quality studies, we adopt five quality assessment criteria (QACs) based on prior work [221, 240]:

- QAC1: Is the study published in a reputable journal or conference?
- QAC2: Does the study contribute to either academia or industry?
- QAC3: Does the study clearly describe the workflow and implementation of the proposed method?
- QAC4: Are the experimental details, including datasets, baselines, and evaluation metrics, clearly reported?
- QAC5: Do the experimental results support the main claims of the study?

Each criterion is scored from 0 to 3 (poor, fair, good, excellent). Studies scoring at least 12 (i.e., 80% of the maximum score) are included. Preprints on arXiv not yet formally accepted receive a score of 1 for QAC1 but remain eligible if their overall score meets the threshold. Applying these criteria, we identify 107 studies, comprising 62 formally published papers and 45 high-quality preprints.

Forward and Backward Snowballing. To enhance the completeness and reproducibility of our literature search, we applied both backward and forward snowballing following established SLR guidelines [196]. After completing our initial venue-based and keyword-based searches, we followed previous work [240] and conducted backward snowballing by manually examining the reference lists of all papers that passed full-text screening. This process allowed us to identify earlier or foundational studies that may not have been indexed under our search keywords or published outside the targeted venues. Next, we performed forward snowballing using Google Scholar and Semantic Scholar to retrieve all subsequent studies that cite the selected papers. This step ensures the inclusion of newer or emerging works that build on the identified literature but may not yet appear in curated venues or databases. For both phases, each candidate paper was subjected to the same screening protocol, including title/abstract inspection, duplicate removal, and full-text eligibility checks. In total, the snowballing process produced 33 additional candidate papers, of which 9 met the inclusion criteria and were incorporated into the final dataset. This expanded and rigorously validated search procedure strengthens the coverage and methodological robustness of our systematic review. Table 10 shows the details of the collected papers.

Compared with prior surveys, our study adopts a substantially more comprehensive and methodologically rigorous approach. First, earlier reviews primarily rely on keyword-based searches or restrict their scope to a small set of publication venues. We integrate three complementary search strategies: a curated venue-based search, systematic queries across four major academic databases, and both forward and backward snowballing, ensuring broader coverage and reducing the likelihood of missing relevant studies. Second, our study employs a

fine-grained inclusion and exclusion framework tailored specifically to LLM-based unit test generation, enabling the precise filtering of studies that genuinely utilize LLMs for test generation tasks, rather than merely discussing LLMs or testing. Third, we introduce a five-dimensional quality assessment rubric that evaluates publication venue, methodological clarity, contribution significance, completeness of experimental reporting, and validity of results.

3.4 Data Extraction and Analysis

Figure 3 (a) illustrates the temporal distribution of models released between 2021 and 2025. The number of models remains very limited in the early years, with only one model in 2021 and two in 2022. This number increases modestly in 2023 (9 models), followed by a sharp growth in 2024 (35 models). The most pronounced surge occurs in 2025, with 69 models reported, indicating a rapid acceleration of research activity in recent years and highlighting the growing momentum of LLM-related work in software testing. This growing number of publications highlights the rising research interest in utilizing LLMs for unit test generation. In addition, we analyze the publication venues of the included studies. Figure 3 (b) shows the distribution of publications across different venues. The majority of studies are published on arXiv (47 papers), reflecting the fast-paced and preprint-driven nature of the field. Among peer-reviewed venues, ASE (13 papers) and FSE (8 papers) represent the most prominent conferences, followed by ISSTA (8 papers) and TOSEM (6 papers). Other well-established software engineering venues, such as ICSE (5 papers), TSE (4 papers), IST (4 papers), and JST (3 papers), also contribute a notable number of studies. The remaining publications are distributed across a wide range of conferences and journals, each with one or two papers, including ASE Journal, EMSE, IEEE Software, Internetware, and OOPSLA. This pattern suggests that early dissemination of LLM-based models often occurs through preprint platforms like arXiv, enabling rapid sharing in a fast-evolving research area. The strong presence of ICSE, ASE, and FSE reflects the growing integration of LLM research into core software engineering communities, where methodological rigor and empirical validation are emphasized.

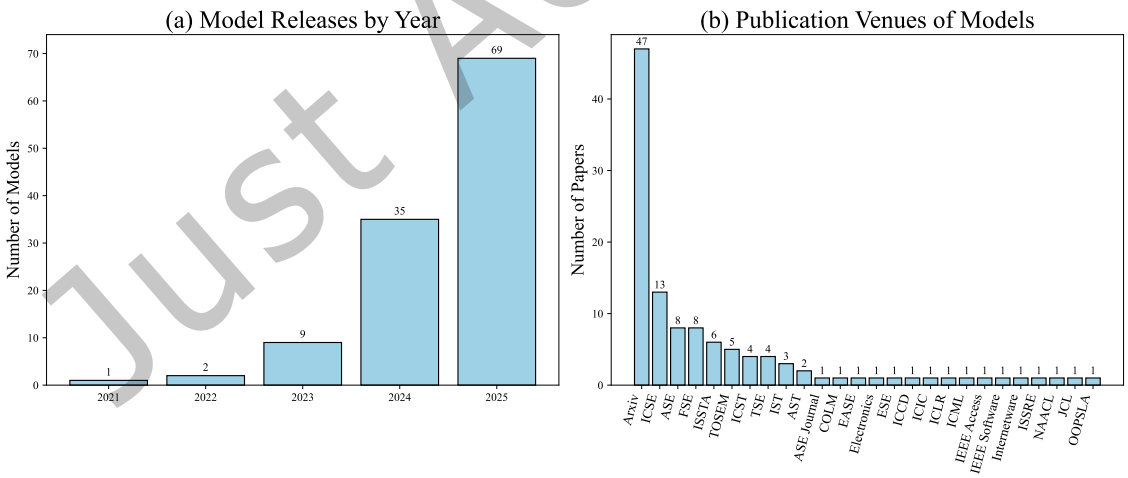

Fig. 3. Distribution of publications per year and venues.

3.5 Utilized LLMs of Surveyed Papers

Among the 116 included studies, we identify 37 distinct LLMs. Figure 4 shows their distribution. We count the occurrence of LLMs used in each study, noting that a single study may employ multiple LLMs. GPT-4 is the most frequently used, representing 17.95% (49/273) of all LLMs, followed by GPT-3.5 at 15.02%. Llama, CodeLlama, Deepseek, and Codex are also widely adopted, together accounting for more than 27.47% of total usage. In contrast, models such as GraphCodeBERT [57], T5 [188], ReasonFlux [207], TinyLlama [220], Magicoder [194], CodeGemma [234], and CodeFuse [37] appear only once each, indicating limited adoption.

Regarding the categories of LLMs, decoder-only LLMs dominate with 88.28% (241/273) of occurrences, reflecting the prevalence of GPT and LLaMA families. Encoder–decoder models contribute 9.52% (26/273), while encoder-only models account for just 2.20% (6/273). Commercial models with undisclosed architectures, notably GPT-3.5 and GPT-4, represent 32.97% (90/273). This distribution underscores the field's strong preference for decoder-only architectures, with encoder–decoder and encoder-only models playing relatively minor roles.

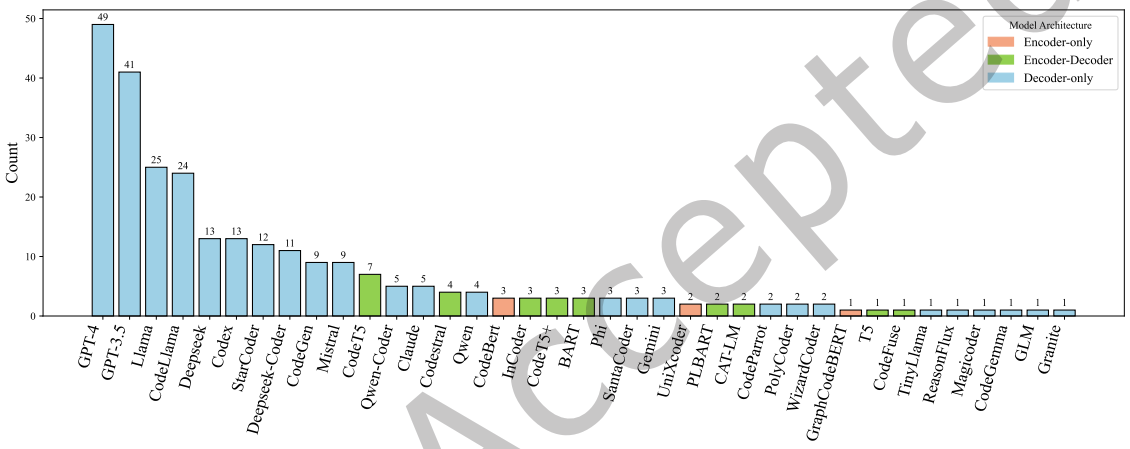

Fig. 4. Distribution of LLMs for unit test generation.

The predominance of decoder-only architectures in current research on LLM-based unit test generation can be attributed to their alignment with the generative nature of programming tasks. Decoder-only models (e.g., GPT-3.5, GPT-4, CodeLlama, and StarCoder) are autoregressive language models optimized for next-token prediction, which directly maps to the core objective of code generation. Their scalable transformer architecture and strong few-shot and in-context learning capabilities have further facilitated rapid adoption. In contrast, encoder-only models (e.g., CodeBERT) are primarily designed for representation learning, excelling at discriminative tasks such as code search, clone detection, or defect prediction, rather than open-ended generation. Their bidirectional masked-language modeling objective is less suited to producing long, coherent sequences, particularly source code that requires structural consistency and dependency tracking. Consequently, encoder-only architectures are rarely chosen as the backbone for generative pipelines and often appear only as auxiliary components for embedding extraction or program understanding, leading to significantly lower representation in code generation studies.

3.6 Explored Programming Languages of Surveyed Papers

Table 4 shows details of the explored programming language. The explored studies predominantly target single programming languages, with Java and Python being by far the most represented. Java-related research dominates

the landscape because of its long-standing role in software engineering research and its well-established, large-scale benchmarks such as Defects4J [83], Bugs.jar [154], QuixBugs [108], SF110 [43], and more recent datasets specifically adapted for LLM testing [189]. Java's strong typing system, explicit semantics, and standardized unit testing frameworks (e.g., JUnit) make it highly suitable for systematic test generation and evaluation. Moreover, a substantial portion of industrial software and academic benchmarks are written in Java, enabling easy comparison across studies and fostering cumulative research progress.

Table 4. Details of the explored programming language.

Type	Programming Languages	Paper Reference
Single Language	Java	[5, 6, 12, 18, 38, 46–48, 52, 54, 55, 60, 65–67, 72, 85, 86, 90, 92, 93, 106, 111, 116, 118, 124, 126, 133, 134, 144, 151, 159, 160, 164, 168, 171, 174, 174, 175, 192, 205, 206, 208–210, 213, 216, 217, 223–226, 230, 233, 238, 239]
	Python	[1, 3, 4, 8, 21, 30, 32, 45, 77, 78, 87, 95, 97–99, 101, 102, 104, 115, 125, 141, 142, 149, 153, 163, 169, 179, 186, 190, 198, 199, 204]
	JavaScript	[143, 158, 228]
	Verilog	[112, 114, 121]
	C\C++	[11, 109, 228]
	Kotlin	[7, 59]
	Rust	[22, 25]
	Golang	[211]
	RV64GC	[33]
Multiple Languages	Java and Python	[137, 162, 167, 212, 232]
	C++ and C#	[136]
	Java, Python, and Go	[51]
	Java, Python, and C/C++	[81]
	Python, Java, and JavaScript	[189]
	Java, Python, C++, JavaScript, and Go	[62]

Python emerges as the second most explored language, owing largely to its ubiquity in AI, data science, scripting, and educational contexts. Benchmarks like HumanEval [19], MBPP [10], and multiple Python-based bug datasets [110, 113, 201] have been widely adopted for evaluating LLM capabilities. Python's dynamic nature and minimal boilerplate allow LLMs to generate syntactically valid and easily executable test cases with relatively low context, making it an attractive target for early-stage or model-agnostic experimentation. The abundance of open-source Python repositories further contributes to data availability for training and evaluation.

Despite the presence of multi-language studies spanning combinations such as Java–Python [137, 162, 167, 212, 232], C/C++ [136], JavaScript [189], and even larger bundles (e.g., Java, Python, C++, JavaScript, Go) [62], most research remains single-language. This is primarily because unit test generation is highly language-specific: differences in type systems, build tools, semantics, memory models, and testing frameworks require specialized prompting strategies, context extraction, and validation pipelines. For example, generating a JUnit test suite for Java fundamentally differs from producing PyTest tests for Python. Ensuring compilability and runtime correctness in multiple languages dramatically increases engineering effort, limiting the feasibility of broad multi-language coverage within a single study.

Languages such as Golang [211], RV64GC [33], Rust [22, 25], and Kotlin [7, 59] appear far less frequently for several reasons. Golang and Kotlin, though increasingly popular, lack mature, widely-used testing benchmarks

analogous to Defects4J [83] or HumanEval [19]. Their unit test ecosystems are also relatively smaller, making them less attractive for systematic evaluation. Rust presents unique challenges due to its strict ownership and borrowing rules, which make LLM-generated code prone to compilation failures. The complexity of its type system, lifetimes, and memory-safety guarantees requires deeper semantic reasoning than current LLMs reliably provide. RV64GC, being an instruction-set architecture rather than a high-level programming language, introduces additional challenges: tests must target low-level semantics, rely on specialized execution environments, and require domain expertise rarely encoded in general-purpose language models.

Overall, the dominance of Java and Python, and the relative scarcity of research on languages such as Rust, Golang, RV64GC, and Kotlin, reflects a combination of benchmark availability, language characteristics, tooling maturity, dataset accessibility, and the practical constraints faced when generating, compiling, and validating LLM-produced tests across heterogeneous programming ecosystems.

3.7 Replicability for Surveyed Papers

To evaluate the replicability of existing studies on LLM-based unit test generation, we systematically examined whether each paper provided sufficient artifacts to enable independent reproduction of its experiments. Specifically, a study was labeled Replicable if it released at least one of the following materials: (1) source code of the proposed method or tool; (2) an accompanying benchmark or dataset; (3) detailed experimental scripts or configuration files; or (4) publicly accessible replication instructions, typically hosted on GitHub, Zenodo, or institutional repositories. In contrast, papers that lacked publicly available artifacts, relied solely on internal datasets, or offered only high-level methodological descriptions without actionable implementation details were classified as Non-Replicable. This assessment was conducted consistently across conference papers, journal articles, and arXiv preprints to ensure a uniform standard of evaluation.

The results summarized in Table 5 reveal that replicability across the surveyed literature is generally high but varies slightly by publication venue. While the field is moving toward strong reproducibility practices, there remains meaningful room for improvement. Overall, 74% of the studies (85 papers) provide some form of replication package, indicating a healthy trend toward transparency and reproducibility in the LLM-based testing community. Journal articles exhibit the highest proportion of replicable studies (0.78), likely reflecting stricter publication policies regarding artifact availability and reproducibility standards. Conferences follow closely at 0.73, suggesting that artifact release is increasingly becoming a norm in software engineering venues, especially those that emphasize empirical evaluation. ArXiv preprints demonstrate a comparable level of replicability (0.72), which is notable given that preprints are not subject to mandatory artifact review; this suggests that many authors proactively share code to enhance visibility, facilitate early adoption, or promote community validation of their approaches. Despite these positive trends, 26% of the papers (31 studies) remain non-replicable. These works typically either rely on proprietary datasets, omit implementation details due to space constraints, or involve prototype systems that were not publicly released. The lack of accessible artifacts not only limits empirical verification but also restricts downstream comparative studies and inhibits progress in understanding how LLM-based techniques perform under diverse settings. Table 11 shows details of replication packages for each study.

Table 5. The proportion of replicability for surveyed papers.

	Conference	Journal	Arxiv	Percent
Replicable	0.73	0.78	0.72	0.74
Non-Replicable	0.27	0.22	0.28	0.26

4 RQ1: What Specific Tasks Have Been Addressed by LLMs?

This section provides a comprehensive analysis from the perspective of specific tasks in unit testing. LLMs have been applied to four main tasks: test generation, test input generation, test oracle generation, and test evolution. One study may solve multiple unit test generation tasks. Figure 5 illustrates the distribution of studies across these tasks. Test generation dominates with 84 papers, underscoring its central role and the highest level of research focus. Test oracle generation and test evolution account for 13 studies and 18 studies, reflecting moderate interest. In contrast, input generation (5 studies) receives relatively limited attention. Overall, the distribution highlights a strong emphasis on generating test cases while revealing clear research gaps in evolution and oracle generation. The following subsections present detailed analyses of each task.

Most existing work focuses on test generation, which is consistent with the dominant research goal of leveraging LLMs to automatically generate executable test cases that improve developer productivity and software quality. Recent advances in LLMs have demonstrated strong capabilities in synthesizing code, test methods, and assertions from natural language descriptions or source code. As a result, this task has attracted significant attention from both academia and industry, leading to a large number of studies. In contrast, test input and oracle generation have received substantially less attention. These tasks are inherently more constrained and technically challenging. Generating valid and diverse test inputs often requires a precise understanding of program semantics, input constraints, data structures, and domain-specific invariants, which LLMs struggle to model reliably. Moreover, many existing test generation systems implicitly include input generation as part of producing test cases, reducing the incentive to study it as an independent research problem. Consequently, researchers tend to prioritize broader tasks such as test generation or test oracle generation, resulting in relatively few studies on input generation.

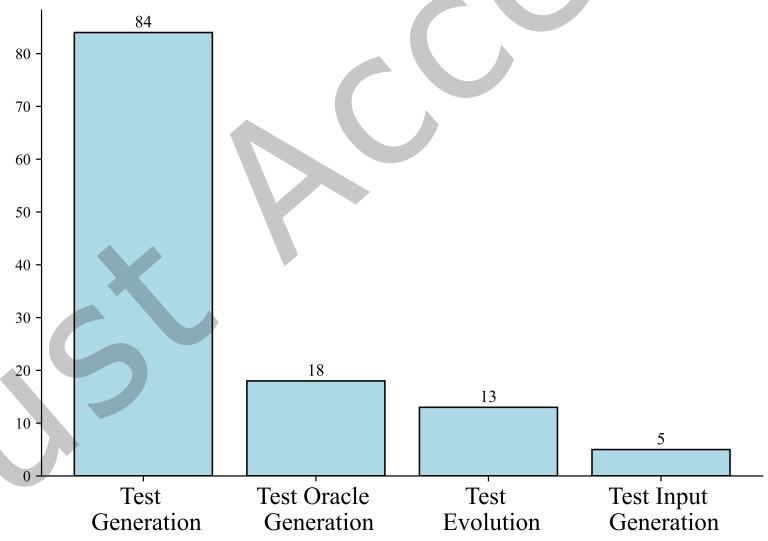

Fig. 5. Distribution of papers for unit test generation with LLMs.

4.1 Test Generation

The workflow of LLM-based unit test generation can be described as a transformation from source code to executable tests. The process starts with providing the function, class, or project under test, optionally augmented

with documentation, comments, or usage examples. Candidate unit tests are then generated, consisting of input values, execution logic, and test oracles. These cases can be refined through feedback loops, where compilation or execution results are used to repair or optimize the generated tests. In the following subsections, we present related work on unit test generation, organized by project-level, class-level, and function-level perspectives.

4.1.1 Project-Level Test Generation. Recent work has begun to explicitly tackle the challenges of project-level unit test generation, where tests must account for multi-file dependencies, project-specific APIs, and realistic execution environments. ProjectTest [189] systematically advances this direction by introducing ProjectTest, the first large-scale benchmark specifically designed for evaluating LLMs on project-level unit test generation across Python, Java, and JavaScript. The study demonstrates that, compared to function- or class-level settings, project-level testing exposes substantially harder challenges, including compilation failures, cascade errors, and limited coverage caused by complex inter-file dependencies. By conducting extensive error analyses and evaluating both manual and LLM-based self-fixing mechanisms, ProjectTest highlights that effective project-level unit test generation requires not only raw generation capability but also robust error-repair pipelines that enable tests to compile, execute, and meaningfully exercise project logic. Besides, IntentionTest [144] generates project-level tests from a semantic and requirement-oriented perspective, arguing that practical tests are fundamentally driven by developers' validation intentions rather than coverage alone. IntentionTest converts high-level validation intentions into executable project-specific tests via a retrieval-and-edit paradigm, reusing existing tests within the same project and enriching generation with project-specific APIs, setup idioms, and assertions. Through iterative refinement and error correction, IntentionTest effectively bridges the gap between natural-language test intent and executable project-level tests, yielding higher semantic relevance, stronger bug-detection capability, and better alignment with real-world testing practices. These two works establish complementary foundations for project-level unit test generation.

4.1.2 Class-Level Test Generation. Class-level test generation aims to automatically generate tests for a whole class, considering the relationships among its methods, shared attributes, and internal state transitions to achieve broader structural coverage. At the class level, Zhang et al. [225] introduced TestBench, the first benchmark for evaluating class-level unit test generation. It includes 108 Java programs from nine large-scale open-source projects and explores the impact of context richness. Gu et al. [54] proposed TestART, which improves test quality through iterative co-evolution of generation and repair, repeatedly compiling, executing, and regenerating test cases. Lops et al. [118] presented AGONETEST, an end-to-end system for generating and evaluating class-level unit tests in Java projects. It integrates test generation, prompt engineering, and automated evaluation using metrics such as coverage and test smells, addressing the need for scalable automation in class-level test generation and assessment.

4.1.3 Function-Level Test Generation. Function-level test generation has become the dominant focus within automated unit testing research, with studies spanning multiple methodological paradigms. Most works (64 studies) employ prompt-inference strategies, demonstrating the popularity of prompt-based approaches. Fine-tuning is also well represented, with 15 studies adapting LLMs for test generation. Three hybrid approaches combine fine-tuning with prompt inference to balance specialization and flexibility. More recent innovations include agent-based methods (seven studies). Overall, the distribution confirms prompt-based approaches as the mainstream paradigm. Hybrid and agent-based methods are emerging to address challenges in error correction, coverage, and robustness.

Fine-tuning LLMs for Test Generation. Fine-tuning technique applies supervised learning on domain-specific datasets, pairing source code with unit tests to generate executable test cases. This process aligns model behavior with software testing conventions and refines pre-trained parameters. Early works, particularly with medium-scale models such as T5 [147] and CodeT5 [188], relied heavily on this paradigm due to limited

generalization ability. For instance, Tufano et al. [174] introduced ATHENATEST, which generates unit tests from developer-written cases and focal methods through a two-step process: pretraining on English and Java corpora, followed by supervised fine-tuning. A3Test [5] incorporated assertion knowledge and is further fine-tuned for test generation. Beyond standard pipelines, researchers explored parameter-efficient fine-tuning (PEFT) [169] and domain adaptation [164]. Storhaug et al. [169] evaluated PEFT as a cost-effective alternative to full fine-tuning. Shin et al. [164] applied task- and project-level adaptation to CodeT5, addressing domain shift and improving project-specific test generation. Besides, Yin et al. [212] proposed AUGER, which unifies bug detection and triggering to guide models in producing bug-revealing tests. CleanTest [216] improves the quality of the fine-tuning datasets and boosts the LLMs' test generation performance. Rehan et al. [151] demonstrate how fine-tuning the Llama-2-7B model with QLoRA on a curated focal-method dataset enables automated, context-aware unit test generation for Java programs, improving the model's ability to produce relevant and structurally meaningful test cases. RLSQM [168] uses static analysis–based automatic feedback as rewards to optimize test quality. By fine-tuning LLMs with a learned reward model, this approach reduces test smells and produces higher-quality, syntactically correct unit tests compared with standard LLM prompting or supervised fine-tuning. UTRL [97] also introduces an adversarial reinforcement learning framework and teaches large models to automatically generate high-quality unit tests. In addition to the above approaches, ATGEN [102] introduces an adversarial reinforcement learning framework that trains an LLM-based test generator against a code generator producing increasingly subtle buggy programs. SF-UTG [145] enhances LLMs for unit test generation by integrating AST and DFG code structures into both encoder attention and decoder prediction tasks. CURE [190] co-evolves an LLM coder and an LLM unit tester to generate high-quality unit tests without ground-truth code and use these tests as reliable reward signals to improve code generation accuracy.

In addition, fine-tuning has also been applied in unit test generation of specialized domains. For instance, Paduraru et al. [136] automated unit test generation for game development, targeting the Unity engine and C++/C#. CPP-UT-Bench [11] provides 2,653 C++ source–test pairs from 14 open-source projects to benchmark LLM performance. In finance, Xue et al. [202] introduced LLM4Fin, which combines LLMs with algorithmic methods to generate acceptance tests from natural language business rules, addressing the strict requirements of FinTech systems such as stock trading.

Prompting LLMs for Test Generation. Unlike fine-tuning, prompt-based methods generate unit tests directly by designing prompts without modifying model parameters. Tasks are framed using natural language instructions or code templates, often enriched with function signatures, documentation, or usage examples. By leveraging pre-trained knowledge, prompt strategies enable zero-shot or few-shot test generation at low adaptation cost. Recent extensions include in-context learning [39], which embeds examples in the prompt, and retrieval-augmented generation, which supplies external knowledge. Prompting is flexible and scalable because it avoids task-specific retraining.

Early prompt-based methods relied on carefully crafted prompts integrating multiple forms of context information [7, 22, 33, 46, 47, 106, 109, 112, 115, 124, 209, 211, 232, 233, 238, 239]. For instance, APT [232] used property-based retrieval augmentation to incorporate both general and task-specific context, improving test accuracy and maintainability. TestGen-LLM [7] extended Kotlin test suites with additional edge-case tests to increase coverage. JSONTESTGEN [238] generated bug-exposing tests for the fastjson2 library by combining historical bug cases, mutation strategies, and differential testing. LLM-TG [33] applied GPT-4 to processor verification, reducing manual effort in test generation. MAPS [46] leverages LLMs to generate diverse, error-aware, and context-enhanced prompts, guiding LLMs to generate more accurate and higher-coverage unit tests. RUG [22] leverages a semantic-aware, bottom-up LLM pipeline, guided by static type-dependency analysis and enhanced with coverage-guided fuzzing, to automatically generate compilable, high-coverage Rust unit tests. IntUT [124] leverages LLMs to generate high-coverage and executable unit tests by integrating program-analysis–derived branch conditions, mock behaviors, and expected outcomes into structured prompts.

Most prompt-based techniques follow a generation-refinement paradigm, where initial tests are iteratively improved using dynamic execution feedback such as coverage and failure data [3, 12, 25, 30, 45, 51, 52, 55, 59, 78, 87, 90, 95, 98, 99, 125, 134, 137, 141, 142, 153, 192, 204, 205, 208, 209, 228, 228, 233]. For example, HITS [192] decomposed methods into slices for stepwise test generation, improving coverage through chain-of-thought prompting, context retrieval, and self-debugging. TELPA [204] applied program analysis and iterative feedback to address hard-to-cover branches. CODAMOSA [98] combined search-based testing with targeted test generation to escape coverage plateaus. SymPrompt [153] decomposed test generation into path-specific stages guided by control-flow constraints. MuTAP [30] augmented prompts with mutation testing to improve bug detection and correctness. Biagiola et al. [12] integrated search-based test generation (EvoSuite) with LLMs-based readability improvements. Overall, prompt-based methods remain the dominant paradigm for function-level test generation, with ongoing efforts to enhance coverage, correctness, and readability through refinement and hybrid strategies. ASTER [137] introduces a static-analysis-guided LLM pipeline that generates natural, high-coverage unit tests by providing rich contextual information to the model and iteratively repairing and augmenting the produced tests. By integrating program analysis, hybrid retrieval, and knowledge-driven prompting, CITYWALK [228] allows LLMs to behave as expert developers, producing compilable, correct, and high-coverage unit tests far beyond standard LLM prompting. LspRag [51] leverages LLMs together with LSP-guided precise context retrieval and compile-free self-repair, enabling models to generate high-coverage, valid tests across multiple programming languages.

Agent for Test Generation. Agent-based approaches decompose unit test generation into modular subtasks, each handled by a specialized agent. Rather than relying on a single model, these frameworks coordinate multiple agents that operate autonomously yet exchange outputs and feedback within iterative loops. By integrating retrieval, reasoning, and self-correction, they improve robustness and adaptability while reducing non-executable code and hallucinated outputs. This paradigm is particularly effective in complex testing scenarios. For example, Sherifi et al. [162] proposed a multi-agent system for automated test generation, execution, and reporting, integrating agents such as a test generation agent and a test executor to minimize human involvement. Li et al. [101] introduced TestChain, including the designer and calculator agents. The designer agent creates diverse test inputs. The calculator agent, via a React chain, maps them to outputs using a Python interpreter, reducing complexity and errors. Arcadinho et al. [8] developed a framework for evaluating models in customer support scenarios, using intermediate graphs to generate high-coverage, realistic test datasets while minimizing fabricated responses. AutoRestTest [89] combined multi-agent reinforcement learning with a semantic property dependency graph to optimize testing, assigning tasks such as endpoint selection, dependency identification, and parameter generation to different agents working collaboratively. TestForge [77] introduces a feedback-driven, agentic LLM framework that treats unit-test generation as an iterative refinement process, where an LLM generates initial tests and progressively improves them using execution errors, coverage gaps, and mutation feedback. AgentTester [18] constructs structured prompts, distills high-quality test-generation instructions, and iteratively repairs faulty outputs, guiding LLMs to generate more accurate, executable, and high-coverage unit tests. RTED [21] introduces a type-aware unit test generation framework that leverages LLMs guided by step-by-step type-constraint analysis and a multi-agent reflection mechanism, enabling the model to synthesize realistic, error-revealing tests rather than coverage-oriented ones.

4.1.4 Empirical Study. Beyond technical advances, researchers have conducted extensive empirical studies to evaluate the effectiveness of LLMs in unit test generation. These studies cover diverse aspects, including fine-tuning [160], prompt engineering [133, 158, 160, 163, 206], integration with traditional techniques [198], contextual factors [160], quality assessment [167, 199], and ChatGPT-specific evaluations [133, 171, 213].

For example, Shang et al. [160] conducted a large-scale study on fine-tuning, evaluating test generation across five benchmarks and eight metrics. Ouédraogo et al. [133] compared four LLMs (GPT-3.5, GPT-4, Mistral 7B,

and Mixtral) with five prompt engineering techniques on datasets including SF110 [43] and Defects4J [83]. Yang et al. [206] assessed five open-source LLMs and compared them with GPT-4 and EvoSuite, emphasizing the impact of prompt design. Xiong et al. [199] examined the functional diversity of generated tests, while Siddiq et al. [167] studied JUnit test generation in strongly typed languages. Yuan et al. [213] evaluated ChatGPT along four dimensions: correctness, sufficiency, readability, and usability. Tang et al. [171] systematically compared ChatGPT and EvoSuite in terms of correctness, readability, coverage, and bug detection. Finally, Santos et al. [156] conducted a survey of 83 professionals, revealing practical applications of LLMs in test generation, bug reporting, debugging, and regression testing. Shin et al. [163] presented a retrieval-augmented LLM framework that substantially improves unit test generation for ML/DL APIs by grounding LLM outputs in domain-specific knowledge such as API documentation, GitHub issues, and StackOverflow discussions.

4.1.5 Combined with Traditional Test Generation Approaches. Among the collected studies, four papers combine LLM-based unit test generation with traditional test generation techniques. These hybrid approaches leverage the semantic reasoning and code understanding capabilities of large language models while integrating established methods such as search-based testing, evolutionary algorithms, and constraint-guided input exploration. For instance, BLAST [90] first prompts an LLM to synthesize a semantically meaningful seed test that encodes critical information from the issue description and the patch (e.g., magic values, triggering inputs). This seed is then statically deserialized into an SBST-compatible canonical form, enabling SBST to begin its evolutionary search from an informed starting point rather than an uninformed random initialization. CodaMOSA [98] enhances traditional SBST by integrating LLMs as a complementary guidance mechanism within the MOSA evolutionary search pipeline. CodaMOSA periodically invokes an LLM to generate semantically meaningful candidate tests that explore new program behaviors. Deljouyi et al. [32] implicitly enhance SBST workflows by complementing their structural generation capabilities with semantic refinement provided by LLMs. Xiao et al. [198] inject LLM-generated tests at different phases of the MOSA search algorithm, during initial population construction, iterative population updates, and coverage-plateau periods. LLMs provide semantically meaningful inputs that help the search escape local optima and reach program states that are difficult for random mutation and crossover to discover. Overall, while LLMs offer clear quantitative advantages in coverage, diversity, and the speed of test creation, traditional approaches remain more reliable in generating structurally consistent and execution-safe test suites. These findings indicate that LLMs can meaningfully complement, but not yet fully replace, established automated and manual testing practices.

4.2 Test Input Generation

LLMs have shown strong potential in generating diverse, context-aware input data for unit tests [81, 85, 104, 114, 134]. They identify patterns and produce inputs that cover valid, invalid, and edge cases. This diversity is crucial for comprehensive testing, as it ensures code behavior is evaluated under a wide range of scenarios.

Approaches to input generation aim to produce meaningful values that represent normal, boundary, and exceptional cases, enabling broad coverage of execution paths. For example, Jiang et al. [81] proposed LLMSym, a hybrid method combining LLMs with constraint-based tools. LLMs generate candidate inputs, while constraint solvers refine them when precision is required. AID [114] integrated LLMs with differential testing by generating program variants and test inputs, then detecting discrepancies in outputs. Li et al. [104] introduced differential prompting to address the difficulty of distinguishing subtle differences between correct and buggy programs. LIBRO [85] generated test cases from bug reports, automatically reproducing reported bugs through targeted prompts. BRMiner [134] combines traditional literal-extraction techniques with LLM–based semantic filtering to mine relevant test inputs from bug reports and produce more effective and bug-revealing unit tests.

4.3 Test Oracle Generation

A test oracle determines whether the outputs of a system under test are correct by comparing them with expected results. Recent work explores how LLMs can support oracle generation by leveraging their ability to recognize patterns in code and reason over both natural language and program logic. Two common categories are assertion oracles, which validate correctness against expected values, and exception oracles, which detect erroneous states or program failures (e.g., division by zero or invalid inputs) [60, 65, 92]. Assertions verify program outputs against predefined criteria such as ranges, integrity, or edge cases [6, 65, 67, 121, 143, 160, 175]. With the rise of LLMs, oracle generation has been studied through both fine-tuning and prompting strategies.

Whole Oracle Generation. Dinella et al. [38] introduced TOGA, the first transformer-based approach for oracle generation. TOGA fine-tunes CodeBERT to (1) determine whether a test prefix raises an exception and (2) rank candidate assertions. Liu et al. [116] later identified three limitations in TOGA: reliance on correct program versions for prefix generation, unrealistic evaluation metrics, and the absence of a clear baseline. They re-evaluated TOGA under improved conditions by reducing duplicates, removing noise, and adding a ranking step for failed cases. Hossain et al. [66] further replicated and extended TOGA across 25 Java systems, 223.5K test cases, and 51K injected faults, assessing its applicability, generalizability, precision, and bug detection. Recent work has shifted from using CodeBERT as a component to adopting fine-tuned or prompt-based LLMs as full backbones for oracle generation. ChatAssert [60] enhances prompts with both static information (e.g., code summaries) and dynamic feedback from executions. It iteratively generates, repairs, and refines oracles, employing static repair for compilation errors and dynamic repair for runtime failures. AugmenTest [86] focuses on EvoSuite-generated test prefixes, prompting models with documentation and developer comments to infer intended behavior. Doc2OracLL [67] leverages fine-tuned LLMs to generate accurate test oracles by using Javadoc comments as the primary behavioral specification.

In addition to the above novel techniques, several empirical studies have assessed the feasibility of LLMs in oracle generation. Hossain et al. [65] evaluated assertion and exception oracles, showing that fine-tuned approaches outperform earlier methods in correctness and diversity. Konstantinou et al. [92] examined whether generated oracles align with developer expectations rather than merely reflecting actual program behavior, highlighting the importance of intent-aware oracle design.

Assertion Oracle Generation. Beyond whole-oracle generation, researchers have explored assertion generation using both fine-tuning [175, 179] and prompt engineering [179, 224]. Tufano et al. [175] proposed a method that generates assert statements with a transformer model, improving unit test generation. Shang et al. [160] conducted a large-scale empirical study of assertion tasks, evaluating multiple LLMs across five benchmarks and eight metrics to compare fine-tuning with prompt-based methods. Other approaches leverage prompt engineering. Nashid et al. [126] applied few-shot learning with Codex to generate assertions from focal methods and test prefixes. Wang et al. [179] developed CLAP, which generates assertions through chain-of-thought prompting and iteratively refines results using both a model and a Python interpreter. Zhang et al. [226] provided the first comprehensive study of assertion generation across five LLMs, two benchmarks, and two metrics, demonstrating that automated assertion generation can substantially reduce developer effort in practice. Building on this, they introduced RetriGen [224], which integrates a hybrid retriever combining lexical and semantic similarity for retrieval-augmented assertion generation. They further proposed AG-RAG [223], a joint training strategy that optimizes both retriever and generator in a unified framework. AsserT5 [143] generates precise test assertions by learning from real test–focal method pairs. AssertFix [121] automatically locates, diagnoses, and repairs incorrect assertions generated by LLM-based test-generation systems.

Exceptional Oracle Generation. In contrast, only one study has addressed exception oracle generation. Zhang et al. [217] proposed exLong, a framework built on CodeLlama that generates exceptional behavior test

cases. exLong combines program analysis with reasoning to extract execution traces, guard conditions, and non-exceptional test cases, enabling automated detection of exception-throwing scenarios.

4.4 Test Evolution

As software evolves through updates, bug fixes, or feature enhancements, corresponding test cases must also be updated to remain valid. This process, known as test evolution, ensures that tests continue to verify system correctness. LLMs show potential in supporting this process by generating or repairing test cases that reflect code changes [32, 48, 72, 93, 111, 141, 149, 210, 230].

Frequent software modifications create challenges for co-evolving production and test code, particularly under limited regression testing resources and rapid release cycles. CEPROT [72] addressed this by detecting and updating obsolete test cases in response to production code edits. It leverages fine-grained edit sequences and applies CodeT5 to classify and update outdated tests. CoverUp [141] introduced an iterative, coverage-guided approach to regression testing in Python. It prompts LLMs to generate additional tests targeting under-covered code regions and refines them through repeated coverage analysis and feedback.

Other approaches focus on automated repair of broken test cases. For example, Liu et al. [111] proposed SYNTER, which builds repair-oriented contexts through static analysis, ranks them, and uses GPT-4 to generate repaired tests. Yaraghi et al. [210] introduced TARGET, which adapts CodeT5+ to test repair by framing the task as language translation. It constructs input–output pairs from broken tests and applies fine-tuning to generate corrected versions. Several works also adopt LLMs to optimize test evolution and improve the readability and maintainability of test code. UTGen [32] enhances test clarity by refining variable names, adding comments, and generating descriptive test names after initial test creation with tools like EvoSuite. Liu et al. [111] extended this direction by repairing tests affected by syntactic breaking changes, such as modified method parameters or return types. Their approach collects and reranks contextual code information before prompting an LLM for accurate repair. UTFix leverages LLMs to automatically repair unit tests by analyzing code diffs, execution failures, and static/dynamic slices of the focal method. TestWeaver [95] leverages LLMs guided by statistical structural testing signals to automatically generate high-quality unit tests. By fusing LLM-based semantic reasoning with coverage-driven feedback, TestWeaver [95] synthesizes more effective and executable test cases than traditional prompting or SBST techniques. YATE [93] introduces a multi-stage repair pipeline, combining chain-of-thought test planning, LLM-guided compilation fixing, oracle correction, and coverage-driven augmentation, to transform near-miss or faulty LLM outputs into high-quality, executable tests. TESTUPDATER [230] updates and enhances unit tests by using LLMs to collect semantic context, generate updated test logic, and iteratively refine errors based on compiler and runtime feedback. Beyond technical solutions, Santos et al. [156] surveyed 83 software professionals on the adoption of LLMs across testing tasks, including generation, bug reporting, debugging, and regression testing. Together, these studies highlight the growing role of LLM-based methods in supporting robust and adaptive test evolution.

4.5 Unexplored Unit Testing Tasks

Despite the rapid progress of LLM-based unit test generation, several important unit testing tasks remain unexplored. First, test prioritization, test selection, and test minimization, which are crucial for regression testing and continuous integration pipelines, have received almost no attention. These tasks require global reasoning across large test suites and historical execution data, which current LLMs are not designed to process efficiently. Second, test adequacy assessment, evaluating whether a test suite sufficiently covers intended behaviors, has not been explored. We think the reason is the difficulty of mapping natural-language model reasoning to formal adequacy metrics such as path coverage or requirement traceability. Third, domains requiring system-level tests, performance tests, concurrency tests, embedded or hardware-oriented tests, or security-focused unit tests remain

underrepresented because they demand specialized execution environments, domain knowledge, and high-fidelity program behavior modeling that LLMs currently struggle to reproduce. Overall, these gaps arise not only from the technical limitations of current LLMs, such as incomplete semantic understanding, limited long-context reasoning, and challenges in ensuring executable correctness, but also from the lack of benchmarks, datasets, and standardized evaluation frameworks.

Answer to RQ1: From the perspective of unit testing, our systematic analysis reveals the following key findings: (1) LLMs have been applied to four primary tasks in unit test generation. Research is heavily concentrated on test generation, while test evolution and input generation remain underexplored. (2) Test generation dominates the field, with 84 studies focusing on generating executable tests at the function, class, or project level, often refined through feedback loops and hybrid strategies. (3) Test input generation (5 studies) explores creating diverse, context-aware inputs. (4) Test oracle generation (18 studies) addresses assertion and exception oracles, with approaches ranging from fine-tuning to retrieval-augmented prompting. (5) Test evolution (13 studies) investigates repairing or updating obsolete tests in response to code changes, supporting regression testing and maintainability.

5 RQ2: How Are LLMs Adapted for Unit Test Generation?

In RQ2, we shift our focus to examining the specific adaptation techniques of LLMs for unit test generation. Regarding the usage of LLMs, three primary strategies emerge: 1) fine-tuning [160], which updates model parameters using labeled datasets; 2) prompt engineering [14], which designs prompts to elicit relevant outputs without parameter updates; and 3) agent-based methods [183], which employ multiple autonomous components to plan, generate, and refine tests collaboratively. Among the included studies, 20.97% adopt fine-tuning, 79.03% use prompt engineering, and 6.45% apply agent-based approaches. The following subsections provide a detailed analysis of these categories.

The large proportion of studies adopting prompt engineering can be explained by its low barrier to entry, flexibility, and strong compatibility with existing LLM capabilities. Prompt-based methods require no model retraining or labeled data, making them cost-effective and easy to experiment with, especially when working with large proprietary models such as GPT-3.5 or GPT-4. Moreover, test generation can often be framed as a direct code synthesis problem, where LLMs are prompted to produce test cases, assertions, or inputs in a single step. This aligns well with the natural strengths of contemporary LLMs and allows researchers to obtain reasonably strong baseline performance with minimal infrastructure or tooling. In contrast, agent-based approaches are less common because they introduce substantially more complexity. These methods require decomposing the testing process into coordinated subtasks (e.g., analysis, generation, execution, repair), establishing communication protocols between agents, and often integrating external tools such as compilers, interpreters, or symbolic engines. This results in high engineering overhead, greater system fragility, and more challenging evaluation. Consequently, researchers tend to adopt agent architectures only when single-step prompting methods reach performance bottlenecks, such as limited coverage, low executability, or the absence of repair mechanisms. The dominance of prompt engineering reflects a pragmatic and resource-efficient approach to leveraging current LLM strengths, whereas the relatively small number of agent-based studies highlights the technical, operational, and research maturity constraints associated with more sophisticated multi-agent systems.

5.1 Fine-tuning

Fine-tuning is one of the earliest and most widely used techniques for adapting LLMs to unit test generation. In this paradigm, LLMs are trained on paired datasets of source methods and developer-written tests to capture structural and semantic mappings between program logic and testing specifications. By formulating the task

as sequence-to-sequence generation, fine-tuning adjusts parameters to learn domain-specific patterns such as assertion construction, input–output relationships, and exception handling. Some approaches rely solely on this strategy [62, 174, 175]. Hossain et al. [65] investigate the potential of fine-tuning LLMs in generating correct, diverse, and robust test oracles. In this subsection, we focus on studies that propose optimizations to fine-tuning, categorizing existing research into parameter-efficient fine-tuning, multi-stage fine-tuning, and domain adaptation.

Parameter-Efficient Fine-Tuning (PEFT). PEFT mitigates the high cost of full fine-tuning by updating only a small set of parameters while freezing most of the pre-trained weights [182]. Techniques such as Low-Rank Adaptation (LoRA) [71], prompt tuning [82], and adapter modules [187] reduce memory use and training time while maintaining or improving test generation quality. These methods preserve general knowledge from pretraining while enabling specialization for tasks such as assertion construction and boundary-value testing. For instance, Storhaug et al. [169] evaluate LoRA, $(IA)^3$, and prompt tuning across multiple LLMs, finding that PEFT reduces resource demands while delivering comparable or better test quality. Paduraru et al. [136] apply LoRA to instruction-tuned test generation in code models. Rehan et al. [151] explored how fine-tuning the Llama-2-7B model with QLoRA on a curated focal-method dataset for Java programs. By integrating focal methods with parameter-efficient LLM adaptation, this study establishes a scalable framework for leveraging LLMs to streamline and enhance unit test generation.

Multi-Stage Fine-Tuning. This strategy adapts LLMs through a sequence of intermediate training phases rather than a single end-to-end fine-tuning step. An initial stage trains on large auxiliary datasets to capture general testing knowledge, followed by fine-tuning on smaller domain-specific datasets to specialize in executable test generation. For example, Hu et al. [72] use CodeT5 [188] in two stages: first, to classify whether a test has become obsolete after code changes; second, to generate updated tests based on modified program logic. Similarly, LLM4Fin [202] applies staged fine-tuning for financial testing, addressing rule filtering (classification) and rule element extraction (sequence labeling) to handle domain-specific business constraints. SF-UTG [145] enhances LLMs for unit test generation by integrating AST and DFG code structures into both encoder attention and decoder prediction tasks. By infusing syntactic and semantic code representations into the model's training process, SF-UTG substantially improves the correctness, coverage, and fault-detection capability of LLM-generated unit tests.

Domain Adaptation. Domain adaptation fine-tuning narrows the gap between generic pretraining and project-specific requirements. This type of approach incorporates domain resources such as developer-written unit tests, API documentation, or industry rules. By aligning with the vocabulary, coding style, and conventions of the target domain, domain adaptation improves correctness, maintainability, and contextual relevance. For example, A3Test [5] first pretrains PLBART [2] on the Atlas dataset [193] to capture assertion patterns. Then, A3Test fine-tunes on the Methods2Test dataset [173] to learn mappings from focal methods to generate unit tests. A3Test achieves better alignment with practical testing needs, leading to improved correctness and coverage in generated test cases. Besides, Shin et al. [164] also apply domain-specific fine-tuning to adapt LLMs for the specialized requirements of financial software testing.

5.2 Prompt Engineering

Prompt engineering has emerged as a central paradigm for LLM-based unit test generation, offering an alternative to resource-intensive fine-tuning. In this setting, models are guided by carefully crafted prompts that encode task-specific instructions, input–output formats, and context information. Existing approaches can be categorized as zero-shot, few-shot, chain-of-thought (CoT) prompting, and retrieval-augmented generation (RAG).

Zero-Shot Prompting. Zero-shot prompting relies solely on natural language instructions, allowing unit tests to be produced without task-specific training. Prompt design often incorporates the following components:

- *Role Description.* Many approaches incorporate role descriptions into prompts to guide test generation [4, 12, 32, 54, 111, 114, 133, 141, 213, 242]. In several studies, the role is explicitly specified, such as "*You are a professional software engineer*" [133] or "*You are an expert Python test developer.*" [141]. Role descriptions appear in different forms: system role prompting, which assigns the model the role of a tester [54, 111, 242]; role-based instructions, which emphasize expertise in software engineering or unit testing [4, 114, 213]; and persona-style prompts, which frame the system as an experienced developer to enhance reliability [12, 32]. Role descriptions in prompts provide contextual framing that constrains output style and scope. By assigning roles such as software engineer or test developer, they help align generated unit tests with domain-specific expectations, thereby improving correctness, coverage, and reliability.
- *Task Description.* Task descriptions are a fundamental element of prompt design. In many studies [7, 12, 30, 33, 81, 96, 100, 153, 186, 186, 192, 198, 199, 213], prompts explicitly specify objectives such as "*Write a unit test for the following function*" [186], "*Generate an input that can trigger an edge case*" [81], or "*Check the correctness of the program output.*" [199]. By explicitly defining the task, such descriptions guide models to generate more targeted and executable unit tests, thereby improving alignment with developer needs and evaluation standards.
- *Program-Analysis Auxiliary Information.* Some approaches [4, 6, 7, 12, 22, 33, 35, 46, 47, 67, 92, 96, 106, 111, 112, 124, 137, 141, 144, 153, 167, 192, 198, 204–206, 209, 213, 214, 219, 225, 232, 239, 242] incorporate program-analysis auxiliary information into prompts to improve reasoning about code behavior. Such information may include structural artifacts, such as control-flow graphs (CFGs) [96, 153] and program slices [192, 204], which provide an explicit view of code semantics. Other designs embed dependency information (e.g., inter-method or variable dependencies) or data-flow relations to reveal execution paths and potential error conditions. More advanced approaches integrate static analysis or symbolic reasoning outputs into the prompt alongside source code, creating semantic scaffolding that supports the generation of precise and fault-revealing unit tests. Program-analysis auxiliary information may also include focal method context, such as class names, method signatures, constructor definitions, field declarations, and related method bodies [4, 7, 12, 33, 35, 92, 111, 167, 198, 206, 213, 214, 219, 225, 232, 242]. Enriching prompts with this contextual or analysis-derived information provides a more comprehensive semantic view of the program under test and improves the quality of generated unit tests.
- *Feedback-Related Information.* Several approaches incorporate feedback-related information into prompts to improve test generation [25, 33, 35, 45, 48, 51, 54, 59, 60, 60, 78, 81, 85, 90, 93–95, 99, 121, 125, 141, 149, 185, 192, 199, 204, 204, 213, 214, 230]. This information often includes **execution results** such as failure messages, error logs, or system outputs, which provide direct signals about incorrect behavior [54, 60, 94, 185, 192, 213, 214]. In other cases, prompts embed coverage metrics (e.g., statement or branch coverage) to guide the generation of additional tests that improve coverage [33, 54, 141, 204]. Several studies also utilize bug reports or mutation testing results [35, 85, 204], where textual bug descriptions or mutant detection feedback help refine test generation and focus on fault-revealing inputs. Besides, some frameworks incorporate **repair-oriented feedback** [54, 60, 81, 199], in which previously unsuccessful test cases and error traces are reused to guide the production of corrected tests. Overall, these strategies demonstrate that integrating feedback signals, from coverage statistics to execution failures, can iteratively enhance the quality, correctness, and bug-detection capability of zero-shot test generation.
- *Tool-Related Information.* Several approaches incorporate tool-calling information into prompt design, guiding LLMs to interact with external components during test generation. In many studies [35, 96, 100, 153, 197, 214, 219], prompts provide API-level specifications such as function headers, parameter constraints, or annotated buggy APIs, enabling the construction of valid test cases for specific library calls. Other studies embed explicit tool invocation requirements [33, 60, 141], instructing LLMs to consider symbolic executors, solvers, or compilers when reasoning about program behavior. In addition, some frameworks integrate runtime interaction details [34, 111], such as API execution procedures or constraint satisfaction rules, to ensure that outputs align

with tool-supported validation. By embedding such tool-calling information, prompts enhance the precision of generated tests and improve their executability within real development frameworks, thereby increasing their effectiveness in revealing subtle bugs.

Few-Shot Prompting. Few-shot prompting for unit test generation employs a small set of curated code–test exemplars to illustrate the mapping from a focal method and its context to executable tests, eliminating the need for additional training [4, 30, 35, 85, 100, 133, 192, 204]. For example, Deng et al. [35] adopted a strategy where prompts include historical bug-triggering examples consisting of an API name, bug description, and the corresponding triggering code. Likewise, Kang et al. [85] incorporated exemplar pairs of bug reports and reproducing test cases, enabling more accurate bug reproduction. Yang et al. [204] proposed counter-example guided prompting, which adds previously generated but invalid tests as negative examples. By explicitly showing unsuccessful coverage attempts, this method directs generation toward more diverse and effective tests for hard-to-cover branches.

Chain-of-Thought (CoT) Prompting. Chain-of-Thought (CoT) prompting is a technique that guides LLMs to explicitly generate intermediate reasoning steps before generating the final output [21, 60, 91, 93, 114, 208, 209]. Rather than directly mapping inputs to outputs, CoT prompts instruct LLMs to "*think step by step*", decomposing the problem into logical sub-tasks [60, 91]. In unit test generation, CoT prompting typically involves sequential reasoning steps such as analyzing program semantics, deriving input–output relations, and constructing tests [32, 35, 100, 114, 206]. For example, Deng et al. [35] added the bug description step to the prompt, requiring the LLM to first explain the potential bug in natural language before generating test code, which enhances the effectiveness of bug-triggering tests. Other studies employ multi-step or decomposition-style prompts [12, 94, 104, 111, 192, 204, 232], where the task is explicitly divided into subgoals such as semantic analysis, input–output derivation, and test construction. Overall, CoT-enhanced prompting reduces spurious outputs, improves logical consistency, and increases the bug-revealing ability and coverage of the generated tests. YATE [93] introduces a multi-stage repair pipeline, combining chain-of-thought test planning, LLM-guided compilation fixing, oracle correction, and coverage-driven augmentation, to transform near-miss or faulty LLM outputs into high-quality, executable tests. IntelliUnitGen [209] adopts structured static-analysis–driven prompts and chain-of-thought reasoning to generate high-quality unit tests.

RAG. Retrieval-Augmented Generation (RAG) is a prompting paradigm that enhances LLMs by integrating external knowledge retrieved from relevant sources into the generation process. Instead of relying solely on the model's parametric memory, RAG dynamically retrieves context information. This retrieval step grounds the model's reasoning in accurate, task-relevant evidence, thereby reducing hallucinations and improving precision. In the context of unit test generation, RAG enables LLMs to leverage authoritative references to generate more executable, semantically consistent, and bug-revealing tests that align closely with domain-specific requirements [100, 109, 111, 115, 185, 206, 211, 232, 233]. For example, Liu et al. [111] combine a static collector with a neural reranker to identify and prioritize the most relevant contextual snippets. The static collector performs a broad retrieval of candidate information, such as method signatures, documentation, or historical test cases, while the neural reranker refines this pool by ranking candidates according to semantic similarity with the query. Zhang et al. [232] go beyond retrieving only the source code context related to the method under test by also incorporating existing test cases into the prompt. By combining relevant code snippets with prior test examples and framing them in a property-driven format, the LLM is better guided to generate consistent, semantically meaningful, and executable unit tests that align with established testing practices. RATester [211] augments LLMs with precise, dynamically retrieved global code context, including definitions and documentation fetched via the Go language server, to eliminate hallucinations and enable the model to generate more correct, compilable, and comprehensive unit tests. TypeTest [115] overcomes the long-standing type-inference bottleneck

in dynamically typed languages by retrieving call-instance and behavioral evidence to guide LLMs in constructing type-correct test inputs and assertions, thereby substantially improving coverage and test quality.

5.3 Multi-Agent Unit Test Generation

Multi-agent approaches to unit test generation decompose the testing workflow into specialized sub-tasks handled by autonomous or collaborative modules, rather than relying on a single monolithic prompt. In these architectures, different components interact through iterative feedback loops, exchanging execution results, error traces, or coverage metrics to guide subsequent refinements [8, 18, 21, 77, 89, 101, 162]. For example, Li et al. [101] introduce a system with distinct roles, where one module generates candidate inputs, and another derives expected outputs, ensuring a clear separation between input generation and oracle construction. Arcadinho et al. [8] design a multi-component pipeline in which modules are assigned dedicated functions such as file locating, test generation, execution, and reporting, enabling end-to-end automation. More advanced frameworks incorporate multi-agent reinforcement learning, where specialized components coordinate actions such as API selection, parameter assignment, and value generation through joint optimization guided by coverage- or failure-based rewards [89]. In addition, some systems treat LLMs as interactive modules capable of calling external tools or compilers, with generated test cases refined through repeated feedback [162]. Overall, these modular architectures illustrate how decomposing the testing process into coordinated functional units can enhance scalability, reduce errors, and produce higher-quality, bug-revealing test cases. RTED [21] leverages LLMs guided by step-by-step type-constraint analysis and a multi-agent reflection mechanism to synthesize realistic, error-revealing tests rather than coverage-oriented ones. By integrating AutoPrompting, multi-temperature prompt distillation, and validation-repair, AgentTester [18] leverages LLM reasoning capabilities to overcome limitations of existing test-generation tools and improve correctness and coverage performance.

Answer to RQ2: From the perspective of adaptation techniques, our systematic analysis reveals the following key findings: (1) Three adaptation strategies dominate: prompt engineering (75%), fine-tuning (22.41%), and agent-based frameworks (5.17%). (2) Prompt engineering steers pretrained models via zero/few-shot instructions, CoT reasoning, RAG, and feedback-augmented prompts (e.g., coverage, failures, bug reports), often within generate-and-refine loops. (3) Fine-tuning aligns models to code–test pairs, with optimizations such as PEFT (e.g., LoRA), multi-stage pipelines, and domain adaptation for specialized contexts (e.g., finance, games). (4) Agent approaches decompose the workflow into coordinated subtasks—input synthesis, oracle derivation, repair—connected by iterative feedback and, in some cases, reinforcement learning.

6 RQ3: What Are the Characteristics of Datasets and Evaluation?

The previous RQs primarily focus on the model construction or adaptation. In this RQ, we focus on examining the characteristics of the datasets used, as well as the evaluation benchmark strategies employed in LLM-based unit test generation studies.

6.1 Dataset

High-quality datasets are essential for unit test generation, as they directly influence the ability of large models to capture accurate patterns and produce meaningful test cases. Well-curated datasets expose models to diverse code structures, functions, edge cases, and error-handling scenarios, thereby improving their understanding of programming language features and testing requirements.

In our analysis of datasets used for LLM-based unit test generation, we emphasize two dimensions: **(1) dataset curation** and **(2) dataset quality**. **(1) Dataset Curation.** Datasets can be categorized into two levels of granularity: complete test cases and individual assert statements. The first type involves full test cases. For

example, Tufano et al. [173] introduced the Methods2Test dataset, which contains 780,944 focal method–test case pairs from 91,385 real-world GitHub projects. He et al. [62] proposed UniTSyn, a multilingual dataset of 2.7 million focal–test pairs across five programming languages. The second type focuses on assert statements, capturing core assertion logic. Watson et al. [193] created the Atlas dataset by mining 9,000 GitHub projects using JUnit, producing 2,502,623 focal–assert pairs. This divergence in dataset granularity can be attributed to the distinct research objectives and methodological requirements underlying different strands of work in automated unit testing. Datasets containing complete test cases are designed to support end-to-end test generation tasks, where models must learn to synthesize executable code that includes input construction, invocation logic, and assertions. In contrast, datasets consisting of individual assert statements focus on the core challenge of reasoning about expected program behavior. These datasets isolate assertions as atomic units of test logic to facilitate learning of semantic correctness, condition evaluation, and bug detection mechanisms.

(2) Dataset Quality. Most datasets rely on heuristic-based methods to link tests with focal methods, using patterns such as naming conventions or structural similarity. These heuristics often fail in complex systems, where they cannot capture dynamic behaviors or context-specific relationships. To address this, Zhang et al. [216] examined dataset noise and proposed a noise-cleaning framework to improve reliability. Kicsi et al. [88] introduced ReCover, a dataset with fine-grained coverage data, including build outputs, execution reports, and statement-level metrics, enabling reproducible experiments. Higo et al. [64] constructed a dataset of functionally equivalent Java methods by analyzing 36 million lines of code from 2,500 projects. Methods with identical parameters and return types were grouped, and tests generated by EvoSuite were executed across groups; mutual pass results were used to confirm functional equivalence. Overall, the evolution from heuristic-based linking toward execution-based validation and semantic analysis reflects the growing recognition that dataset quality directly constrains the reliability, interpretability, and scientific value of empirical studies in LLM-based test generation. While heuristics enable large-scale dataset construction, their inability to model dynamic behaviors has led to a new wave of research efforts focused on noise reduction, behavior-aware validation, and reproducible benchmarking.

6.2 Evaluation Benchmark and Metrics

6.2.1 Benchmark. Benchmarks serve as standardized frameworks for evaluating the performance, efficiency, and effectiveness of testing tools and techniques. In the absence of dedicated benchmarks for test code evolution, objective comparisons of different methods for test suite maintenance remain difficult.

Figure 6 shows the distribution of benchmarks used in existing studies. Our analysis of existing studies reveals several trends in the benchmarks employed for evaluating LLM-based unit test generation: (1) Newly constructed datasets are the most prevalent, appearing in 39 papers, followed by Defects4J [83] with 30 studies. (2) Defects4J [83] is the second most frequently used benchmark, reflecting its importance for assessing test generation models on real-world, bug-prone Java programs. Its prominence highlights the value of comprehensive, practical datasets for realistic evaluation. (3) HumanEval [19] and its variants (HumanEval-X [236]) are also widely adopted. These benchmarks target small, isolated functions, providing insight into a model's ability to generate precise unit tests at a fine-grained level. (4) Other benchmarks, including DynaMOSA [150], BugsInPy [195], Evosuite SF110 [43], and Pynguin [120], are used in three to five studies each. Their recurring use underscores their relevance for evaluating test generation in Python and other programming ecosystems. (5) 22 benchmarks appear in only a single study, such as APPS [63] and Quixbugs [108]. Their limited use suggests specialization or relative novelty, while also indicating potential for introducing unique challenges and emerging evaluation perspectives.

The diversity of benchmarks demonstrates active experimentation in the field, with researchers assessing LLMs across different programming languages, testing tasks, and levels of granularity. Recent benchmarks such as CPP-UT-Bench [11], TDD-Bench [4], TESTEVAL [186], and TestBench [225] extend this trend by targeting

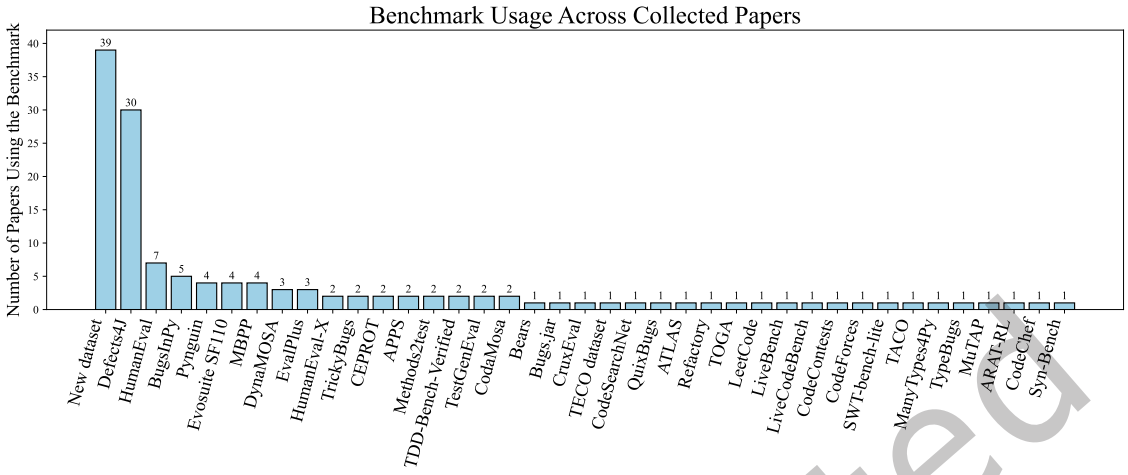

Fig. 6. The distribution of benchmarks used in existing studies.

specific contexts, including C++ unit test generation and class-level testing. We summarize the statistics for these benchmarks in Table 6.

Existing benchmarks are often insufficiently aligned with the emerging capabilities of LLMs, particularly in terms of scale, diversity, and coverage of modern programming languages and paradigms. As a result, researchers frequently build customized datasets to support specific evaluation needs, such as large-scale test synthesis, multi-language support, or fine-grained behavioral analysis. In contrast, Defects4J [83] remains the second most widely used benchmark because it is a long-standing, high-quality dataset of real-world, bug-prone Java programs that enables rigorous and repeatable evaluation of test generation techniques. Its prominence reflects the continued emphasis on realistic, defect-focused testing scenarios and the research community's reliance on well-established, trusted benchmarks to demonstrate empirical validity. The popularity of HumanEval and its variants can be attributed to their lightweight, function-level design, which simplifies experimental setup and allows researchers to obtain clear, interpretable results on fine-grained test generation capabilities. By contrast, benchmarks such as DynaMOSA [150], BugsInPy [195], SF110 [43], and Pynguin [120], while less widely used, are adopted recurrently because they provide high-quality domain-specific evaluation environments (e.g., Python ecosystems, search-based testing), appealing to researchers working in these areas. Finally, benchmarks that appear only once, such as APPS [63] and QuixBugs [108], tend to be either recently introduced, highly specialized, or associated with narrow problem settings, which limits broad adoption but simultaneously signals emerging opportunities for novel evaluation tasks and perspectives. Overall, the distribution of benchmarks reveals a trade-off between using established datasets that ensure comparability and credibility and new or specialized datasets that better match the complexity and diversity of modern testing challenges.

6.2.2 Metrics. Evolution metrics assess the extent to which unit tests adapt to changes in the underlying software, thereby reflecting their robustness and long-term maintainability. Figure 7 presents the distribution of metrics used in existing studies. Overall, the evaluation landscape demonstrates a strong reliance on traditional software testing criteria—such as coverage, bug detection, and compilation—while gradually expanding toward quality-oriented and domain-specific measures. This diversity highlights the need for more unified and comprehensive evaluation frameworks to benchmark LLM-based unit test generation approaches. Based on our analysis, five key findings emerge:

Table 6. The detailed statistics of commonly-used benchmarks used in evaluating LLMs for unit test generation.

Benchmark Name	Size	#Program Language	Date	Link	Domains
Defects4J [83]	854 bugs	Java	2014	https://github.com/rjust/defects4j	Reproducible Bugs
HumanEval [19]	164 tests	Python	2021	https://github.com/openai/human-eval	Python Programming Code Generation
Pynguin [120]	118 modules	Python	2022	https://github.com/se2p/pynguin	Test Case Generation
Evosuite SF110 [43]	8,784 classes	Java	2011	https://www.evosuite.org/experimental-data/sf100/	Test Case Generation
BugsInPy [195]	493 bugs	Python	2020	https://github.com/soarsmu/BugsInPy	Python Program Repair
DynaMOSA [150]	346 classes	Java	2018	https://www.evosuite.org/experimental-data/evolutionary-algorithm-study/	Test Case Generation
HumanEval-X [236]	820 pairs	Multiple PLs	2023	https://github.com/THUDM/CodeGeeX	Code Generation
MBPP [10]	964 pairs	Python	2021	https://huggingface.co/datasets/google-research-datasets/mbpp	Python Programming Code Generation
CruxEval [201]	800	Python	2024	https://crux-eval.github.io/	Python Code Generation
Bugs.jar [154]	21K functions	Java	2018	https://github.com/bugs-dot-jar/bugs-dot-jar	Java Program Repair
Bears [122]	251 bugs	Java	2019	https://github.com/bears-bugs/bears-benchmark	Java Program Repair
TECO dataset [127]	131K tests	Java	2013	https://github.com/EngineeringSoftware/teco	Code Complete
EvalPlus [110]	1,138 tasks	Python	2023	https://github.com/evalplus/evalplus	Code Generation
TrickyBugs [113]	3,043 bugs	Python	2024	https://zenodo.org/records/10252289	Bugs in Plausible Programs
CEPROT [72]	5,196	Java	2023	https://github.com/CEPROTest/CEPROT	Test Case Generation
CodeSearchNet [75]	6,452,446 functions	Multiple PLs	2020	https://github.com/github/CodeSearchNet	Code Generation
APPS [63]	131,777 tests	Python	2021	https://github.com/hendrycks/apps	Code Generation
Methods2test [173]	780,944 tests	Java	2021	https://github.com/microsoft/methods2test	Method-Level Test Case Generation
Quixbugs [108]	40 tests	Java and Python	2017	https://github.com/jkoppel/QuixBugs	Program Repair
ATLAS [193]	188,154 assertions	Java	2020	https://sites.google.com/view/atlas-nmt/home	Assert Statement Generation
CPP-UT-Bench [11]	2,653 pairs	C++	2024	https://huggingface.co/datasets/Nutanix/CPP-UNITTEST-BENCH	C++ Unit Test Generation
TDD-Bench [4]	13 datasets	Python	2025	https://github.com/zzh9568/TDDBench	Data Detection
TESTEVAL [186]	210	Python	2025	https://github.com/LLM4SoftwareTesting/TestEval	Test Case Generation
TestBench [225]	108 programs	Java	2024	https://github.com/iSEngLab/TestBench	Class-Level Test Case Generation

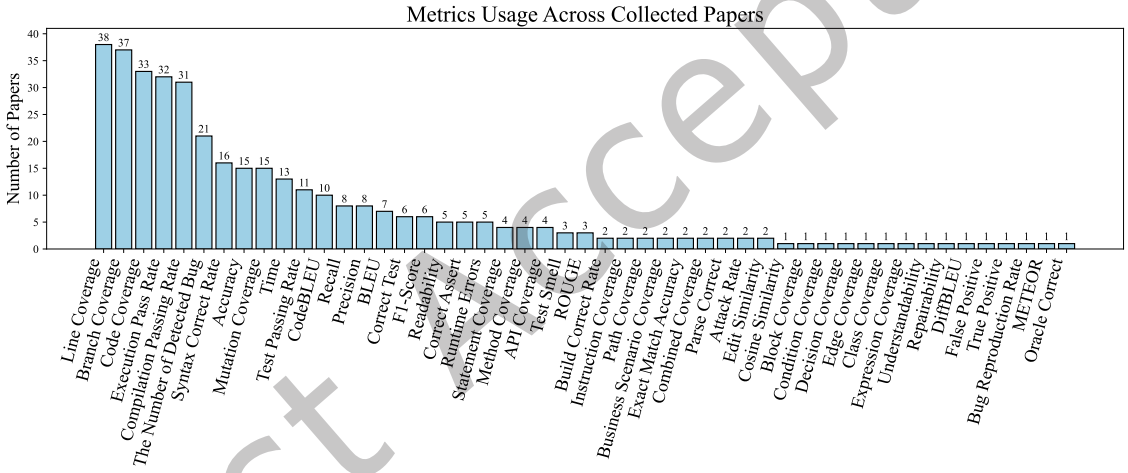

Fig. 7. The distribution of evaluation metrics used in existing studies.

(1) The analysis of existing studies reveals a diverse set of evaluation metrics employed to assess LLM-based unit test generation methods. The most widely used are coverage-based metrics, including line coverage (38 studies), branch coverage (37 studies), and code coverage (33 papers), which measure how thoroughly generated tests exercise the program under test. In addition, the execution passing rate (32 studies) and compilation passing rate (31 studies) are frequently adopted to evaluate execution and compilation correctness, respectively. (2) Accuracy (15 studies), CodeBLEU (11 studies), and mutation coverage (15 studies) further provide quantitative assessments of semantic alignment, code similarity, and robustness. The number of detected bugs (21 studies) focuses on test quality with respect to bug detection. (3) Classic machine learning metrics such as precision, recall, and F1-score (6-8 studies each) have also been adopted, especially when test generation is framed as a classification or retrieval problem. (4) Less common but notable metrics include build correctness, instruction

coverage, business scenario coverage, and repairability, which capture aspects of test execution, domain-specific adequacy, and adaptability to software evolution. Text similarity metrics such as BLEU, ROUGE, cosine similarity, and DiffBLEU appear in a small number of studies to measure alignment between generated and reference test cases.

The current metric landscape reflects an evolutionary trajectory: from reliance on traditional, coverage-oriented criteria that emphasize structural adequacy, toward a more heterogeneous and semantically rich set of measures that address behavioral correctness, human-centric quality attributes, and domain-specific requirements. Coverage-based metrics remain dominant because they are long-established, objective, and easily automatable measures of structural adequacy. They provide a standardized way to quantify how thoroughly tests exercise program behavior, enabling direct comparison across systems, techniques, and benchmarks. Similarly, metrics such as bug detection and compilation rate persist because they offer practical, outcome-oriented evidence of test effectiveness and syntactic validity, aligning with long-standing goals of automated testing research. In contrast, metrics such as accuracy, CodeBLEU, and mutation coverage have gained traction more recently because LLM-generated tests introduce challenges that traditional criteria do not fully capture. These metrics reflect an effort to evaluate not just whether tests execute code, but whether they represent meaningful, high-quality verification artifacts that align with program semantics and reveal faults beyond shallow structural coverage. The emergence of readability, maintainability, and test smell metrics indicates a broader shift toward evaluating LLM outputs from a human-centered and software engineering quality perspective, acknowledging that automatically generated tests must remain understandable, maintainable, and reusable within real development workflows. The use of classic ML metrics (precision, recall, F1-score) reflects the increasing tendency to frame sub-problems, such as assertion generation or test retrieval, as classification or ranking tasks, thereby importing evaluation practices from machine learning. Finally, the presence of rare and domain-specific metrics, such as business scenario coverage, repairability, and instruction coverage, highlights the growing diversification of LLM-based testing applications. These less common metrics tend to arise in specialized or high-stakes domains, where generic criteria cannot adequately capture domain adequacy or operational reliability.

6.2.3 Evaluation Tools. Across the 33 collected studies that explicitly report quality-evaluation tools, researchers validate LLM-generated unit tests using a diverse set of measurement instruments targeting coverage adequacy, fault-revealing power, test smells, static correctness, and compilability. Code coverage is the most widely used criterion, with tools such as JaCoCo [76], Clover [26], Cobertura [27], LLVM [117], and SlipCover [140] assessing line, branch, and instruction coverage across languages, including Java, Python, and C/C++. To evaluate the effectiveness of tests in detecting faults, studies [54, 87, 92, 115, 118, 145, 225] employ mutation testing tools such as PiTest [28], mut.py, and Mutmut [50], which quantify how many injected mutants the generated tests can kill. Several works [48, 118, 137, 167] further assess test quality through test smell detection using TSDetect [139], and through static analysis frameworks like WALA [178], javalang [79], and Tree-Sitter [172], which help identify semantic errors or structural inconsistencies. Finally, tools such as test-compile are used to verify whether the generated tests compile and execute successfully. For semantic or structural quality, studies rely on CodeBLEU, ROUGE, or diff-based similarity measures. These instruments provide a multi-dimensional validation pipeline that allows researchers to rigorously measure the correctness, robustness, readability, and practical usefulness of LLM-generated unit tests. The detailed statistics of the commonly used tools are shown in Table 7.

Answer to RQ3: LLM-based unit test generation studies primarily rely on large-scale but often noisy datasets (e.g., Methods2Test, UniTSyn, Atlas) and widely adopted benchmarks such as Defects4J and HumanEval, complemented by newer domain-specific resources. Evaluation is dominated by traditional metrics (coverage, bug detection, compilation), with emerging attention to correctness, readability, and maintainability, underscoring the need for higher-quality datasets and more unified evaluation frameworks.

Table 7. The detailed statistics of commonly-used tools used in evaluating LLM-generated unit tests.

Quality Type	Tool Name	Programming Languages	Paper Reference
Code Coverage	JaCoCo [76]	Java	[72, 93, 99, 106, 109, 118, 137, 167, 171, 192, 206, 225, 232, 233, 238, 239]
	coverage.py	Python	[87, 137]
	Clover [26]	Java and Groovy	[164]
	Cobertura [27]	Java	[216]
	LLVM [117]	Rust and C\C++	[22, 228]
	SlipCover [140]	Python	[141]
Mutation Testing Tool	PiTest [28]	Java	[54, 92, 118, 145, 225]
	mut.py	Python	[87]
	Mutmut [50]	Python	[115]
Test Smell	TSDetect [139]	Java	[48, 118, 137, 167]
Static Analysis Tool	WALA [178]	Java	[137]
	javalang [79]	Java	[85, 225]
	Tree-Sitter [172]	Multiple PLs	[137, 216]
Compile Tool	test-compile	Java	[225]

7 Challenges and Potential Solutions

In this section, we first discuss the limitations of current studies and then introduce potential solutions, illustrating the research trajectory shaped by prior work and highlighting future directions for exploration. Table 8 synthesizes the key challenges and solution strategies identified in LLM-based unit test generation, categorizing them across five dimensions: dataset optimization, fine-tuning, prompt engineering, agent-based methods, and benchmark enhancement, along with corresponding empirical evidence and representative studies.

7.1 Dataset Optimization

❗ **Challenge I: Limited coverage across a small number of programming languages.** As shown in Section 6.1, only UniTSyn [62] focuses on multilingual programming languages. Other studies [173, 193] focus on Java and Python, indicating a strong language concentration that limits generalizability. While these languages are popular in software development, they represent only a small fraction of the broader programming ecosystem. As a result, LLMs trained on such limited datasets often struggle to generalize effectively to other programming languages. This lack of diversity in training data hampers their ability to generate accurate unit tests across different programming languages, limiting their applicability to less common or domain-specific languages.

🚀 ***Potential Solution: Synthesize test code.*** This challenge presents an opportunity to improve software testing datasets for domain-specific languages. There are several techniques employed to generate domain-specific or infrequently used unit test datasets. (1) Researchers can analyze open-source projects and existing software within the target domain to extract and adapt relevant unit tests. (2) LLMs can be leveraged to generate unit tests by combining real-world test cases with synthetically generated ones, enhancing dataset diversity. (3) Training multilingual models and applying cross-language transfer learning can help LLMs generalize across different programming languages.

🚀 ***Potential Solution: Develop scalable, language-agnostic approaches to dataset construction, model adaptation, and evaluation.*** One potential solution is to create cross-language benchmark suites that systematically sample real-world projects from emerging ecosystems (e.g., Go, Rust, Kotlin), thereby enabling more diverse and representative evaluation. In parallel, researchers can explore transfer learning and multilingual fine-tuning strategies that leverage knowledge learned from high-resource languages to bootstrap performance in low-resource ones, reducing data scarcity barriers. To improve generalization, future systems may also adopt

Table 8. The evidence-based mapping of challenges and solution strategies in LLM-driven unit test generation.

Type	Challenges	Evidence Observed in Studies	Representative Studies	Solutions
Dataset Optimization	Limited coverage across a small number of programming languages	Few studies focus on multilingual programming languages	[62]	Synthesize test code
				Develop scalable, language-agnostic approaches to dataset construction, model adaptation, and evaluation
	Lack of multi-level alignment	Existing datasets typically capture method-level mappings	[62, 173, 193]	Create multi-level alignment dataset
				Optimize LLMs to capture structural and semantic relationships within software artifacts
	Lack of reasoning process in test input	Most datasets contain millions of method-test pairs	[62, 173, 193]	Create dataset with "thought process"
				Leverage multi-modal data sources
				Construct datasets with positive and negative samples
Fine-tuning	Suboptimal performance of parameter efficiency in fine-tuning LLMs	Only 20.97% of studies adopt fine-tuning	Section 5	Advancing PEFT techniques
				Incorporate Mixture-of-Experts style routing mechanisms
	Suboptimal performance of adapting traditional fine-tuning techniques	Fine-tuned models typically struggle with program-level reasoning	[126, 160, 165]	Develop specialized fine-tuning techniques
Prompt Engineering	Balancing context richness with input length	79.03% of studies focus on prompt engineering	[126, 153, 192, 204]	Adaptive prompt combined with program analysis tools
				Design context compression techniques
	Dependency on precise prompting	Existing studies are highly sensitive to manual prompt design choices	[126, 153]	Feedback of prompt quality
				Develop robust, adaptive prompting strategies
	Non-determinism in outputs	Existing studies produce inconsistent or unstable outputs	[111, 210]	Maintaining consistency in output
Agent	Complex dependency management	Existing studies attempt to mitigate environmental and dependency-related failures	[8, 89, 101, 162]	Incorporate advanced coordination mechanisms
				Model and leverage repository-level dependency graphs
				Adopt reinforcement learning to optimize repository dependency information
	Handling ambiguity and uncertainty requirements	The difficulty of handling ambiguity and uncertainty in requirements is reflected in existing agent-based studies	[8, 89, 101, 162]	Integrate symbolic reasoning and constraint-solving techniques
Benchmark Enhancement	Insufficient interpretability and reliability evaluation	Existing evaluations prioritize functional correctness and coverage-based metrics	[19, 83]	Enhancing benchmarks for interpretability and reliability
				Incorporate agent-based architectures
	Complexity of real-world code	Existing benchmarks focus on small or simplified program fragments	[10, 19, 63, 236]	Add benchmarks of test code update and test input generation tasks
	Generated tests with invalid syntax and hallucinated assertions	Existing approaches struggle to generate tests with high syntactic accuracy	[89, 171]	Combine LLM generation with constraint-aware postprocessing and program analysis
		Existing approaches are prone to hallucination-related errors	[133, 171]	Multi-round or agent-based test generation

intermediate representations or language-independent abstractions, such as AST graphs, API schemas, or IR-based execution traces.

🚀 ***Potential Solution: Broaden the linguistic and ecosystem diversity of LLM-based unit test generation.*** Several important languages, particularly C/C++, Rust, Go, and Kotlin, require deeper investigation due to their growing industrial relevance and the unique technical challenges they pose. For example, C/C++ and Rust underpin systems programming, embedded software, and safety-critical applications, yet their manual memory management, strict typing, pointer semantics, and concurrency models make automated test generation substantially more difficult for current LLMs. Similarly, Go's widespread adoption in cloud-native and distributed systems introduces challenges related to goroutines, channel-based communication, and nondeterministic execution, which existing benchmarks and models rarely capture. Kotlin, despite its central role in Android and modern backend development, is limited by a lack of representative unit-test datasets and insufficient LLM training exposure. Addressing these gaps will require not only the construction of new multilingual benchmarks and industrial-scale datasets, but also the development of LLM–program-analysis hybrid methods capable of reasoning about complex type systems, memory safety, and concurrency constructs. Beyond C/C++, Rust, Go, and Kotlin, several other programming languages remain underexplored in LLM-based unit test generation and warrant deeper investigation, such as JavaScript, TypeScript, Swift, PHP, Ruby, Scala, and other languages.

❗ **Challenge II: Lack of multi-level alignment between tests and the corresponding code being tested.** In Section 6.1, our dataset analysis revealed that existing datasets typically capture method-level mappings and rarely include class or requirement-level links, resulting in insufficient semantic traceability between code and tests. This misalignment hinders LLMs from fully understanding expected behaviors and generating tests that

comprehensively cover critical aspects. For example, a test case may verify the correctness of a function but not be explicitly linked to the specific function or method it tests, resulting in gaps in test coverage.

🚀 ***Potential Solution: Create multi-level alignment dataset.*** The development of a multi-level alignment dataset presents a promising research opportunity in LLM-based test generation. Multi-level alignment refers to structuring data across method, class, project, and requirement levels. By aligning these levels, researchers can better capture the complex relationships between software's logic and corresponding test cases, enhancing the overall test generation process. For instance, at the method level, alignment could focus on code structure, mapping specific code elements to generated test cases, ensuring coverage of branches, loops, and edge cases. At the requirement level, alignment can ensure that generated test cases correspond to specified software requirements, covering all critical business logic.

🚀 ***Potential Solution: Optimize LLMs to capture structural and semantic relationships within software artifacts.*** One direction is to equip LLMs with multi-granular representations that jointly encode method-level signatures, class hierarchies, and high-level requirements, allowing LLMs to reason about behavioral intent before generating test cases. Model optimization can also leverage multi-task training objectives, where the LLM learns not only to generate unit tests but also to predict focal methods, identify enclosing classes, and summarize expected behaviors, thereby internalizing alignment as part of its training trajectory. Besides, integrating code-aware pre-training signals, such as static analysis features, coverage annotations, or execution traces, can strengthen LLM's ability to infer which components of the program a test should exercise. Advanced prompting strategies, including hierarchical planning, retrieval-augmented generation, and role-based prompting, further encourage LLMs to reason stepwise about the linkage between requirements, classes, and methods. Finally, optimizing LLMs to incorporate structural and behavioral cues will enable the generation of tests that are not only syntactically correct but also accurately aligned with the intended code artifacts and their functional responsibilities.

❗ **Challenge III: Lack of reasoning process in test input.** Our systematic review shows that large-scale datasets such as Methods2Test [173] and UniTSyn [62] contain millions of focal method–test pairs, but they primarily encode outcomes rather than the process by which inputs are derived, including constraint extraction, boundary identification, or semantic justification. Similarly, datasets focusing on assertion statements, such as Atlas [193], offer rich coverage of assertion logic but omit contextual cues about how specific inputs were selected to exercise particular behaviors. Hence, LLMs typically generate test cases based on patterns and statistical relationships in the code rather than understanding the reasoning or intent behind the inputs. In contrast, manual testing relies on developers' insights into system behavior, including potential edge cases and input boundaries. However, existing datasets lack this reasoning process, resulting in test cases that cover a broad range of inputs but lack meaningful diversity and relevance.

🚀 ***Potential Solution: Create dataset with "thought process".*** Creating a dataset with the "thought process" offers a transformative research opportunity to enhance the quality and interpretability of generated test cases. The "thought process" refers to capturing and documenting the reasoning behind each step of test generation, including LLMs' decision-making when selecting test cases, identifying edge cases, and determining coverage requirements. Such a dataset would include not only the test cases themselves but also detailed explanations of their rationale, enabling LLMs to learn the context and underlying logic necessary for generating more effective and meaningful unit tests.

🚀 ***Potential Solution: Leverage multimodal data sources.*** Rather than relying solely on static source code, future datasets could integrate information extracted from execution logs, API documentation, UI interactions, configuration files, and domain-specific artifacts, providing LLMs with richer signals about how inputs are derived, validated, and consumed in real systems. Dynamic artifacts such as runtime traces, error logs, and debugging sessions can reveal causal relationships between specific input values and program behaviors, while natural-language artifacts such as developer discussions, issue reports, or requirement documents can expose high-level rationale and semantic intent behind input design choices. Incorporating such heterogeneous data

into training corpora would allow LLMs to learn implicit reasoning patterns grounded in real execution contexts, rather than inferring inputs from code structure alone. Ultimately, multimodal data can supply the missing bridge between semantic intent, behavioral constraints, and concrete input values, enabling test generation models to produce inputs that are not only executable but also logically motivated and aligned with how software actually behaves in practice.

🚀 ***Potential Solution: Construct datasets with positive and negative samples.*** Positive samples can demonstrate valid, semantically meaningful inputs that trigger intended program behaviors. Negative samples can showcase boundary violations, type mismatches, invalid states, or cases that fail to satisfy functional requirements. By exposing models to both successful and unsuccessful input–behavior pairs, such datasets allow LLMs to learn discriminative patterns and develop an implicit reasoning ability to differentiate good inputs from bad ones, rather than relying on memorized templates or random guessing. Furthermore, labeling negative samples with failure causes, exception messages, or coverage gaps provides actionable supervision that teaches models why a given input is incorrect and how to refine it. Incorporating adversarial, corner-case, or mutation-induced negative examples can further encourage models to explore diverse input spaces and reason about edge conditions. Ultimately, datasets that encode contrastive learning signals across positive and negative samples can help LLMs internalize the semantic and behavioral constraints governing input validity, enabling the generation of test inputs that are not only executable and diverse but also derived through informed reasoning aligned with program semantics.

7.2 Fine-tuning

❗ **Challenge I: Suboptimal performance of parameter efficiency in fine-tuning LLMs.** The suboptimal parameter efficiency of fine-tuning LLMs is strongly reflected in our review findings. Only 20.97% of studies adopt fine-tuning. Fine-tuning LLMs for unit test generation is resource-intensive, requiring substantial computational power and large amounts of labeled data. This makes the fine-tuning process both costly and time-consuming. Several researchers are exploring parameter-efficient fine-tuning (PEFT) methods [69, 105], which aim to achieve comparable or superior performance with fewer parameters and lower resource consumption. However, maintaining the same level of accuracy and test generation completeness remains a significant challenge in this approach. Moreover, fine-tuning strategies in existing studies tend to optimize for task-specific performance, leading to models that perform well on the targeted training corpus but exhibit poor generalization to unseen projects, programming languages, or testing frameworks. This specialization amplifies the parameter inefficiency problem, as new tasks or datasets frequently necessitate separate fine-tuned models, resulting in storage overhead and model proliferation.

🚀 ***Potential Solution: Advancing PEFT techniques.*** Integrating transfer learning from LLMs and leveraging meta-learning strategies could enable LLMs to adapt to diverse unit testing tasks with minimal data and computational resources. Besides, exploring PEFT approaches may lead to more efficient test generation models that maintain high performance across various scenarios without requiring the extensive resources typically associated with fine-tuning.

🚀 ***Potential Solution: Incorporate Mixture-of-Experts (MoE)–style routing mechanisms.*** Rather than updating a single, monolithic parameter space, MoE architectures partition the model into multiple lightweight "experts," each responsible for learning different aspects of unit test generation, such as input construction, assertion inference, or domain-specific API usage. A router network determines which experts to engage for a given input based on semantic features, thereby ensuring that only the most relevant parameters are trained and executed. This fine-grained specialization offers two key advantages: (1) it reduces computational and memory cost, since most parameters remain inactive for each forward/backward pass, and (2) it enhances generalization, as experts can independently learn specialized skills without interfering with each other's parameter space.

Moreover, MoE routing can be implemented in a parameter-efficient manner, for example, by combining low-rank adaptation (LoRA) modules with expert layers, enabling the model to incorporate new capabilities without modifying the base model. In the context of unit test generation, MoE-based architectures may also support task-aware routing, where experts are dynamically assigned based on code complexity, programming language, or type of test artifact being generated.

❗ **Challenge II: Suboptimal performance of adapting traditional fine-tuning techniques.** In existing studies [126, 160, 165], fine-tuned models typically struggle with program-level reasoning, assertion synthesis, and coverage-driven behavior, resulting in outputs that are syntactically fluent but semantically incomplete, particularly when evaluating corner cases or environment-dependent logic. Hence, the adaptation of fine-tuning techniques for unit test generation often leads to suboptimal performance. These techniques were originally developed for natural language processing (NLP) tasks, where the primary objective is to adapt pre-trained models to process text-based data. However, code possesses a distinct structure, syntax, and semantics that differ significantly from natural language. For instance, code comprises specific programming constructs such as variables, functions, loops, and conditional statements, each of which must be interpreted and manipulated within a logical context. Consequently, directly applying conventional fine-tuning methods to code-related tasks may not yield optimal results.

🚀 ***Potential Solution: Develop specialized fine-tuning techniques.*** It is crucial to design specialized fine-tuning techniques that take into account the peculiarities of programming languages, including their syntax, semantics, and the specific logic of the code being tested. These techniques would enable models to generate tests that are not only syntactically correct but also logically sound, ensuring they effectively verify the functionality of the code under test.

7.3 Prompt Engineering

❗ **Challenge I: Balancing context richness with input length.** In Section 5.2, 79.03% of studies focus on prompt engineering. However, several studies [126, 153, 192, 204] report that when prompts contain rich contextual descriptions necessary for reasoning about complex code, the input frequently exceeds token limits, leading to aggressive truncation or selective omission of essential details. Conversely, when prompts are shortened to fit within allowable lengths, they often lose semantic cues needed for accurate test construction, resulting in incomplete or trivial test cases. This trade-off is particularly evident in evaluations conducted on large code units or multi-module projects. As a result, prompt-based approaches exhibit performance degradation on tasks that require deep contextual understanding, especially when compared to tasks involving short, self-contained functions. Therefore, the primary challenge is to strike an optimal balance, offering enough detail for accurate test generation while staying within LLMs' input size constraints to avoid losing critical information.

Moreover, the quality of generated tests depends not only on the amount of context provided but also on its relevance and conciseness. Including extraneous or irrelevant information can overwhelm the model, leading to inefficient or inaccurate test generation. Hence, the key challenge is to optimize context length while ensuring it remains detailed and focused on the critical aspects necessary for generating effective unit tests. Achieving this balance requires a careful assessment of the model's limitations and a deep understanding of unit testing requirements.

🚀 ***Potential Solution: Adaptive prompt combined with program analysis tools.*** A possible solution involves integrating program analysis tools with prompt optimization strategies. This approach could ensure that relevant context is captured and effectively communicated to LLMs. For instance, researchers employ static analysis tools to automatically analyze focal methods. These tools can identify key components such as functions, variables, classes, dependencies, and control flow structures. This information can then be used to generate concise but rich contextual summaries of the code that emphasize the most critical aspects needed for unit test

generation. By doing so, LLMs receive relevant context without exceeding input size constraints. Besides, concise and structured test intent information can be incorporated into the context. Key code features most relevant to the test generation task should be highlighted, while unnecessary code or comments that do not contribute to the process can be excluded. This ensures that the prompt remains focused and concise while still providing LLMs with all essential information.

🚀 ***Potential Solution: Design context compression techniques.*** One approach is to perform hierarchical summarization, where code, documentation, and prior test examples are compressed into multi-level representations, such as high-level intent descriptions, class-level summaries, and method-level logic flows. This can be further enhanced by static and dynamic analysis–driven filtering, which automatically extracts key semantic signals (e.g., control-flow edges, API dependencies, exception-handling logic) and discards low-impact tokens such as formatting, boilerplate code, or repeated patterns. Additionally, designing learned compression modules that map long code segments into compact embeddings or symbolic abstractions enables the model to retain contextual cues without requiring full textual input. Beyond structural compression, task-aware prioritization, deciding which information is essential for generating meaningful tests (e.g., boundary conditions, invariants, data types), ensures that compression aligns with downstream objectives rather than generic summarization.

❗ **Challenge II: Dependency on precise prompting.** The dependency on precise prompting is clearly reflected in our review findings. Prompt engineering is the dominant strategy in existing work, being adopted by 79.03% of studies, while only a small fraction explores alternative adaptation mechanisms such as fine-tuning or agent-based approaches. Across these studies [126, 153], performance is repeatedly reported as highly sensitive to manual prompt design choices, including prompt structure, specificity, and ordering of contextual information, with minor changes often leading to substantial performance fluctuations. Vague, ambiguous, or insufficiently detailed prompts can lead to suboptimal test cases that are incomplete, irrelevant, or misaligned with the intended testing objectives. A poorly crafted prompt may produce only basic test cases that cover typical scenarios while failing to address edge cases or complex behaviors. Therefore, crafting precise and detailed prompts is crucial for guiding LLMs in generating comprehensive and effective unit tests. This underscores the importance of understanding both the capabilities and limitations of LLMs and tailoring prompts to the specific requirements of the testing task.

🚀 ***Potential Solution: Feedback of prompt quality.*** To mitigate the challenge of dependency on precise prompting in unit test generation using LLMs, an effective solution is to implement a feedback mechanism that evaluates prompt quality and enhances clarity while aligning with the LLMs' capabilities. If the generated test cases fail to meet the required standards, the feedback mechanism can provide corrective input, which can then be used to refine the prompt. This iterative process helps guide the LLM toward generating improved test cases in subsequent iterations.

🚀 ***Potential Solution: Develop robust, adaptive prompting strategies.*** One direction is to employ automatic prompt optimization, where the system iteratively refines prompts through gradient-free search, reinforcement learning, or evolutionary algorithms, using coverage, correctness, or execution feedback as optimization signals. Another promising avenue is to design task-aware prompt templates that encode domain-specific knowledge, such as assertion patterns, API usage constraints, or typical fault scenarios, thereby providing structured guidance that is less reliant on human-crafted instructions. In addition, meta-prompting frameworks can be introduced to allow models to reason about their own instructions, detect failure modes, and autonomously adjust their prompting strategies based on observed outcomes. Finally, integrating prompting with modular architectures or agent-style pipelines, where specialized components collaborate to generate, validate, and repair outputs, can further mitigate the impact of imperfect prompts by distributing reasoning across multiple dedicated processes.

❗ **Challenge III: Non-determinism in outputs.** The challenge of non-determinism in outputs is reflected in our review findings, which show that LLM-based test generation frequently produces inconsistent or unstable outputs across repeated executions, even when models are provided with identical prompts and identical code

contexts. Several studies [111, 210] report that generated tests vary in structure, assertion logic, and coverage, leading to unpredictable performance, fluctuating correctness, and difficulty reproducing results across trials. This instability is particularly evident in prompt-based approaches, which dominate the literature (79.03%), as small variations in sampling parameters or model states can lead to substantial semantic drift, resulting in tests that compile in one run but fail in another, or that detect different sets of bugs in repeated trials. If LLMs generate different test cases each time they are prompted, it becomes difficult to ensure that the tests are reliable and repeatable. Besides, non-determinism complicates the process of identifying the most appropriate or effective test cases for a given scenario. This increases the need for human intervention to review and select the best tests from multiple generated options, ultimately reducing the efficiency and automation of the testing process.

🚀 ***Potential Solution: Maintaining consistency in output.*** A promising approach to improving output consistency is controlled prompting or template-based prompting, where predefined structures or templates guide LLMs to generate unit tests in a standardized format. Besides, incorporating constraint-based rules or specific conditions can further minimize inconsistency by restricting the output space and reducing variations in generated test cases.

7.4 Agent

❗ **Challenge I: Complex dependency management.** Existing agent-based research [8, 89, 101, 162] attempts to mitigate environmental and dependency-related failures through multi-step execution, verification, and repair workflows, yet still reports frequent breakdowns when processing real-world repositories. Hence, managing complex dependencies among agents is a major challenge in multi-agent systems for LLM-based test generation. These agents collaborate to generate different components of a test case, such as defining operations, selecting parameters, and determining input values. However, they do not operate in isolation. Their decisions are highly interdependent. Choices made by one agent can directly affect the validity of another's decisions, creating a risk of inconsistencies. For example, a change in an agent's selected parameter could invalidate previously generated function calls, leading to incomplete or erroneous test cases. Addressing these dependencies is crucial to ensuring coherence and accuracy in agent-based test generation systems.

🚀 ***Potential Solution: Incorporate advanced coordination mechanisms.*** To manage these complex dependencies, multi-agent systems must incorporate advanced coordination mechanisms. Approaches such as dependency graphs and real-time synchronization protocols can enable agents to share information, resolve conflicts, and maintain the integrity of generated tests. These mechanisms help align agent actions, minimize errors in test case generation, and enhance the efficiency of the overall test creation process.

🚀 ***Potential Solution: Model and leverage repository-level dependency graphs.*** One potential solution is to perform static analysis (via Tree-Sitter and related tooling) and construct a fine-grained dependency graph over functions, classes, modules, and their interrelations (calls, inheritance, imports). The dependency graph captures not only direct imports but also transitive library dependencies, version constraints, and module-level coupling. This graph can then be used to (1) identify self-contained modules whose dependencies can be resolved in isolation, (2) detect and cluster heavily coupled components that require integrated test environments, and (3) generate dependency-aware prompts that provide the LLM with precise information about available APIs, required initialization order, and external services for each focal method. Hierarchical decomposition can further partition large systems into dependency-aware "slices" (e.g., per service, package, or bounded context), enabling the test generation pipeline to configure environments, install libraries, and scaffold mocks at the right granularity instead of naively guessing imports.

🚀 ***Potential Solution: Adopt reinforcement learning to optimize repository dependency information.*** One potential solution can treat the repository and its ecosystem as an environment, where the state encodes a partial view of the dependency graph, and actions correspond to concrete search or analysis steps—such as

inspecting a new config file, querying a package registry, parsing a build script, or attempting a test build in a sandbox. The RL agent's goal is to learn a search policy that efficiently discovers the minimal set of dependency and environment information required to successfully compile and execute tests for a given project. Reward signals can be designed to balance correctness and efficiency, for example, by giving positive rewards for successful builds, resolved imports, and executable tests, while penalizing unnecessary file scans, failed installations, or redundant environment configurations. Over time, the agent can learn project- and ecosystem-specific patterns and adapt its search strategy accordingly, leading to faster, more reliable, and less ad-hoc dependency resolution. This RL-optimized search layer can feed precise, up-to-date dependency context into LLM-based test generation pipelines, significantly improving the robustness and reproducibility of generated tests in complex, real-world repositories.

❗ **Challenge II: Handling ambiguity and uncertainty requirements.** The difficulty of handling ambiguity and uncertainty in requirements is reflected in existing agent-based studies [8, 89, 101, 162], which attempt to distribute reasoning across multiple specialized agents but still struggle to interpret vague, incomplete, or underspecified behavioral expectations. While agent pipelines introduce roles such as planner, generator, verifier, and refiner, these agents largely operate on explicit cues extracted from code structure and execution traces, and provide limited support for inferring latent requirements or disambiguating developer intent. Ambiguities in requirements, such as vague boundary conditions or contradictory constraints, can cause confusion among agents. This can result in generated test cases that are either irrelevant or fail to cover critical aspects of the code. Besides, unclear instructions can make it difficult for agents to interpret the intended software behavior, reducing the effectiveness of test generation. Addressing these challenges requires refining requirement specifications and improving prompt clarity to ensure more precise and meaningful test outputs.

🚀 ***Potential Solution: Integrate symbolic reasoning and constraint-solving techniques.*** To address ambiguity in agent-based test generation, systems can incorporate mechanisms to manage uncertainty and improve test reliability. Strategies may include providing more explicit instructions, implementing fallback mechanisms for handling ambiguous cases, and introducing robust validation steps to ensure tests align with specified requirements. Besides, integrating symbolic reasoning and constraint-solving techniques could enhance agents' ability to process edge cases and ambiguous scenarios. These methods would improve the system's understanding of the problem domain, reducing uncertainty and increasing the quality and relevance of generated tests.

7.5 Benchmark Enhancement

❗ **Challenge I: Insufficient interpretability and reliability evaluation.** The challenge of insufficient interpretability and reliability evaluation is reinforced by our benchmark analysis in Section 6.2, which shows that existing evaluation suites overwhelmingly prioritize functional correctness and coverage-based metrics, while offering limited mechanisms to assess test interpretability, behavioral robustness, or long-term reliability. Widely used benchmarks such as Defects4J [83] and HumanEval [19] primarily measure line or branch coverage, compilation success, and basic correctness, but do not capture whether generated tests provide meaningful explanations, adhere to semantic invariants, or remain stable under code evolution. LLMs are often considered "black-box" models due to their opaque reasoning and decision-making processes. This lack of transparency undermines developers' trust in generated test cases. Without insight into why a particular test is generated or how it aligns with the intended software behavior, ensuring correctness becomes challenging. Moreover, current benchmarks do not evaluate interpretability and reliability, both of which are crucial for validating the practical applicability of LLMs in real-world testing scenarios. Addressing these gaps is essential to increasing confidence in LLM-based test generation.

🚀 ***Potential Solution: Enhancing benchmarks for interpretability and reliability.*** There is a growing need for evaluation frameworks that assess both the interpretability and reliability of LLM-generated test cases. Future benchmarks should incorporate mechanisms to provide insights into the LLM's reasoning and decision-making process when generating tests. Besides, strategies must be developed to improve the consistency and predictability of test generation, ensuring that the produced test cases are not only interpretable but also reliable. Strengthening these evaluation criteria will enhance trust in LLM-based unit test generation for real-world applications.

🚀 ***Potential Solution: Incorporate agent-based architectures.*** Instead of relying on a single LLM, multi-agent systems can assign specialized roles to separate agents, such as a generation agent for producing candidate tests, a verification agent for executing and validating them, and an explanation agent for analyzing decision steps and generating human-readable rationales. The verification agent can perform runtime checking, mutation analysis, or symbolic execution, and feed structured feedback back to the generation agent, creating interpretable, evidence-based refinement loops. Meanwhile, the explanation agent can track intermediate decisions, key assumptions, and confidence estimates, producing post-hoc explanations that clarify why certain inputs, assertions, or test strategies were selected, as well as documenting the causes of failures. By distributing responsibilities, agent systems transform opaque one-shot generation into observable, auditable, and self-evaluating workflows, providing both operational robustness and actionable interpretability.

❗ **Challenge II: Complexity of real-world code.** The challenge of addressing the complexity of real-world code is reflected in our benchmark analysis, which shows that existing evaluation suites predominantly consist of small, self-contained functions or simplified program fragments, such as HumanEval [19], HumanEval-X [236], MBPP [10], and APPS [63]. Datasets like Defects4J [83], despite being widely used, are often applied in ways that focus on individual classes or isolated faults, rather than the broader system interactions that characterize industrial codebases. As a result, existing studies achieve promising results in controlled settings but also performance degradation or tool failures when models are applied to larger, real-world repositories. Besides, real-world software projects are frequently updated, with ongoing refactoring, bug fixes, and new feature additions. These continuous changes introduce increasing complexities. Generated tests must adapt to these dynamic changes, yet benchmarks for tasks like test input generation and test code modification remain scarce. The lack of such benchmarks makes it challenging to evaluate how effectively LLMs adapt to code changes, such as refactoring or new feature integrations, and generate test inputs that adequately cover both existing and modified code paths.

🚀 ***Potential Solution: Add benchmarks of test code update and test input generation tasks.*** The development of benchmarks that accurately capture the relationship between tested code and corresponding test case changes presents a valuable opportunity. Such benchmarks would enable models to be trained and evaluated in real-world, evolving testing conditions, allowing them to adapt to code modifications, including bug fixes, refactoring, or feature extensions. By incorporating the dynamic nature of software evolution into test generation tasks, these benchmarks would provide a more comprehensive assessment of a model's ability to generate meaningful and contextually appropriate tests in response to code updates. Besides, they would facilitate precise evaluation and comparison of large models, supporting their continuous improvement and optimization.

❗ **Challenge III: Generated tests with invalid syntax and hallucinated assertions.** LLMs often generate syntactically invalid tests, incomplete or hallucinated assertions, and semantically infeasible paths, limiting the reliability and maintainability of the produced test suites. For instance, the generated tests by SPDG-enhanced [89] are not only more comprehensive but also more effective in revealing faults. Tang et al. [171] found that while ChatGPT can generate more readable and human-like tests, these tests often lack semantic correctness, contain invalid API usages, and include weak or missing assertions. Ouédraogo et al. [133] also found that LLM-generated tests still suffer from issues such as hallucinated assertions and incorrect API usage.

🚀 ***Potential Solution: Combine LLM generation with constraint-aware postprocessing and program-analysis.*** Syntactic invalidity can be mitigated through automated repair pipelines and grammar-constrained decoding, while hallucinated or incomplete assertions can be reduced by leveraging oracle templates, specification extraction, or mutation-based validation to enforce semantic correctness. Additionally, integrating static and dynamic analyses, such as path feasibility checking, branch-predicate extraction, and symbolic execution, helps prevent the generation of semantically impossible execution paths, thereby increasing the reliability and maintainability of the resulting test suites.

🚀 ***Potential Solution: Multi-round or agent-based test generation.*** In an agent-based setting, the test generation process is decomposed into specialized, cooperative agents with clearly defined responsibilities. A code analyzer agent extracts ground-truth program context, including target method signatures, dependency graphs, accessible classes, and available assertion libraries. Based on this verified context, a test generator agent synthesizes candidate tests. A syntax/build checker agent then validates compilability through static parsing or compilation, immediately filtering tests with invalid syntax or unresolved references. Subsequently, an execution and oracle checker agent executes the tests and inspects assertion validity, identifying hallucinated assertions that lack execution evidence, such as assertions over unobserved return values, unreachable states, or unused mocks. Finally, a test repair agent applies localized modifications guided by concrete compiler diagnostics, runtime exceptions, and execution traces, iteratively refining the test until it becomes both compilable and behaviorally consistent.

8 Discussion

In this section, we first present a comparative analysis of LLM-based test generation and traditional automated testing techniques. We then discuss the implications for research and practice. Next, we articulate the limitations of this review. Finally, we examine the threats to validity associated with our systematic review methodology, covering internal and external validity concerns.

8.1 Comparing LLM-Based Test Generation with Traditional Techniques

Table 9. Traditional test generation methods adopted in the collected LLM-based unit test generation studies.

Type	Traditional Methods	Paper Reference
Unit Test Generation	**Evosuite** [42]	[18, 32, 38, 54, 66, 81, 106, 133, 134, 137, 144, 159, 167, 171, 198, 206, 209, 213, 216, 217, 233]
	Pynguin [120]	[30, 90, 98, 104, 137, 153, 204]
	Randoop [135]	[106, 134, 216, 217]
	RustyUnit [176]	[25]
	RuMono [229]	[25]
	MuTAP [30]	[30]
	NxtUnit [184]	[211]
	Pyinder [131]	[21]
	SunwiseAUnit [170]	[109]
Test Input Generation	**KLEE** [15]	[81]
	Angr [166]	[81]
	CrossHair [29]	[81]

Among the collected studies, a total of 32 papers conduct empirical comparisons against traditional unit test generation methods. Specifically, 22 papers compare their approaches with EvoSuite, making it the most frequently used baseline, while 7 papers include comparisons with Pynguin [120]. In addition, a smaller number of studies evaluate against other established tools, including RustyUnit [176] and RuMono [229] (1 paper), Randoop [135] (4 papers), MuTAP [30] (1 paper), NxtUnit [184] (1 paper), Pyinder [131] (1 paper), and SunwiseAUnit [170] (1 paper). Furthermore, one study compares against symbolic or constraint-based testing tools, including KLEE [15],

Angr [166], and CrossHair [29]. This diversity of baselines reflects the breadth of traditional unit test generation techniques used for evaluating LLM-based approaches. Table 9 shows traditional test generation methods adopted in the collected studies.

LLM-based test generation differs fundamentally from traditional automated testing approaches in both methodology and capability. Traditional techniques, such as search-based testing, symbolic execution, and evolutionary algorithms, systematically explore program structures, providing strong guarantees in terms of soundness, reproducibility, and coverage-driven rigor, but they often struggle with generating meaningful test oracles, handling complex input domains, or inferring developer intent. In contrast, LLM-based approaches leverage semantic understanding, natural-language reasoning, and contextual pattern learning to produce human-like test cases and assertions, making them particularly effective at synthesizing realistic usage scenarios and high-level oracles. However, LLMs face persistent challenges related to reasoning reliability, determinism, dependency resolution, and execution robustness, areas where traditional techniques remain superior. Despite these differences, the two paradigms are highly complementary: LLMs can accelerate test design and generate rich candidate tests, while traditional tools can validate, refine, and optimize these tests through systematic analysis. This synergy points toward promising hybrid frameworks that combine semantic generation with structural verification.

Across the surveyed studies, LLM-based unit test generation methods exhibit clear qualitative distinctions from traditional techniques. LLMs provide strong semantic reasoning, enabling more readable, human-like, and contextually meaningful tests, and they can incorporate rich sources of information, such as bug reports [90, 134], repository-level context [211], path constraints [153], type constraints [153], differential behaviors [104], and structured seeds [109], to overcome long-standing limitations of search-based, random, and symbolic execution approaches. These enhancements allow LLM-based methods to trigger exceptional behaviors [217], reach deep or hard-to-cover branches [204], generate project-specific and intention-aligned tests [144, 211], and break through coverage plateaus that tools like EvoSuite [42], Randoop [135], KLEE [15], and Pynguin frequently miss. However, multiple studies also show that LLMs often suffer from issues rarely seen in mature SBST tools, including hallucinated assertions, invalid API usage, syntactic errors, missing or incorrect oracles, and reduced reliability in strict correctness or safety-critical scenarios.

Quantitatively, the comparison reveals a more nuanced landscape: many hybrid or enhanced LLM-based frameworks, such as BLAST [90], ASTER [137], CODAMOSA [98], EXLONG [217], PALM [25], SymPrompt [153], TestART [54], TestLoter [208], and RATester [211], substantially outperform traditional baselines by achieving higher line and branch coverage, generating more valid or fault-revealing tests, increasing oracle correctness, reducing error rates, and uncovering bugs missed by classic tools. Some systems achieve dramatic gains [98, 153, 216], including 9–10% average coverage improvements, 5× correctness boosts, 20–30% branch coverage increases, or multi-fold reductions in surviving mutants. Empirical studies also show that vanilla LLM prompting, without structural constraints, repair mechanisms, or hybrid integration, still underperforms traditional tools in key metrics such as compilation success, coverage, and mutation detection, with EvoSuite consistently outperforming GPT-based generators in raw structural adequacy. Taken together, these results highlight a consistent trend: LLMs alone are not yet a full replacement for traditional methods, but when augmented with program analysis, search, constraints, or repair mechanisms, LLM-based approaches can surpass traditional unit test generation both qualitatively and quantitatively.

8.2 Implications for Research and Practice

Insights for Research. Our review reveals several important insights for researchers seeking to advance LLM-based unit test generation. (1) The current research landscape is heavily concentrated on test generation, whereas more specialized tasks, such as input generation, oracle construction, and especially test evolution,

remain significantly underexplored. This indicates clear methodological gaps and opportunities for developing new techniques that go beyond direct code-to-test generation, particularly those requiring deeper semantic reasoning or integration with program analysis tools. (2) Our analysis shows that the field overwhelmingly relies on decoder-only models, while encoder-only and encoder–decoder architectures appear far less frequently. This highlights a need for research into how alternative architectures, multi-modal models, or hybrid LLM–analysis pipelines could improve consistency, reduce hallucinations, and better support tasks such as assertion inference or regression-aware test repair. (3) Current approaches suffer from limited robustness and reproducibility. Existing studies are sensitive to prompt design, suggesting that fundamental challenges in LLM alignment, dataset quality, and contextual understanding remain unresolved. (4) Our examination of datasets and benchmarks reveals heavy dependence on a small set of Java and Python benchmarks such as Defects4J, SF110, and HumanEval, while domains involving Rust, Go, Kotlin, or large-scale industrial systems are severely underrepresented. This lack of diversity restricts the generalizability of current evidence and highlights the urgent need for new, multi-language, context-rich, and industrially grounded benchmarks. (5) Although 74% of surveyed studies provide some form of replication package, a substantial portion still lacks accessible artifacts, limiting cross-study comparison and slowing scientific progress. Collectively, these insights indicate that future research should prioritize expanding task coverage, improving LLM adaptability and reliability, diversifying benchmarks, and strengthening reproducibility practices to build a more rigorous foundation for LLM-driven software testing research.

Insights for Practitioners. Practitioners adopting LLM-based unit test generation must recognize both the strengths and limitations of current technologies. (1) While LLMs can substantially accelerate test creation and improve developer productivity, they still face persistent challenges in reasoning reliability, determinism, dependency resolution, and execution robustness. Traditional techniques remain superior in these aspects, suggesting that practitioners should treat LLM-generated tests as assistive artifacts rather than fully reliable replacements for established automated testing tools. Practitioners could adopt hybrid workflows, where LLMs are used to propose candidate tests and traditional static or dynamic analysis tools refine, validate, and verify those tests to ensure correctness and executability. (2) Most existing empirical evaluations have been conducted on simplified benchmarks or small-scale datasets, which may not reflect the complexity of real industrial systems. Therefore, practitioners should be cautious when generalizing reported results and should integrate additional validation steps when deploying LLM-based testing in production environments. (3) The rapid evolution of LLMs means that tool performance can vary significantly over time. Hence, practitioners should regularly reassess model behavior, stay aware of emerging capabilities, and adopt workflows that allow for iterative improvements rather than one-time integration.

8.3 Limitations of This Review

While this review provides a comprehensive synthesis of recent advances in LLM-based unit test generation, several limitations should be acknowledged. (1) The scope is constrained to unit testing and does not include system-level or end-to-end testing approaches, which may limit the applicability of some insights to broader testing scenarios. However, unit test generation represents a fundamentally different problem space from system-level testing, particularly in terms of input granularity, test oracle design, dependency management, and model reasoning requirements. System-level tests typically involve end-to-end workflows, external services, APIs, GUIs, or environmental configurations that are beyond the reasoning capabilities of current LLMs. Including such methods would broaden the scope substantially and dilute the depth of analysis needed to understand the technical challenges unique to LLM-driven unit test generation. Besides, unit test generation has reached a critical mass of scientific contributions, 116 papers between 2021 and 2025 in our survey, enabling a deep, evidence-based synthesis. (2) The reviewed studies exhibit substantial heterogeneity in terminology, evaluation

metrics, benchmarks, and experimental setups, which restricts our ability to make direct quantitative comparisons or derive stronger generalizable claims. (3) Despite our systematic search strategy, relevant studies may still have been missed due to variations in keyword usage or unpublished industrial work. (4) Many included studies rely on simplified benchmarks or small-scale datasets, which may not reflect the complexity of real-world software systems. Therefore, the synthesized findings may not fully generalize to large, highly interconnected codebases. (5) The rapid evolution of LLMs means that some conclusions may become outdated as new LLM architectures and agent-based paradigms emerge. These limitations highlight the need for ongoing updates and more standardized methodologies to ensure cumulative progress in this fast-moving research area.

8.4 Threats to Validity

Internal Validity. One threat to internal validity lies in the selection and screening process of the surveyed papers. Despite our systematic methodology, there is a risk of inadvertently omitting relevant studies due to limitations in search terms, database coverage, or manual filtering. To mitigate this threat, our review employed a rigorous and multi-stage study selection process, including duplicate removal, manual screening, automated database search, quality assessment, and forward/backward snowballing, thereby reducing the risk of selection bias and ensuring that only studies meeting predefined criteria were included. The use of explicit inclusion and exclusion criteria, along with a five-dimensional quality assessment rubric (e.g., publication venue, methodological clarity, experimental detail, and empirical support), further strengthens confidence in the correctness, consistency, and reliability of the extracted findings. Moreover, data extraction followed a structured framework aligned with the research questions (tasks addressed, LLM adaptation strategies, datasets/metrics), which minimizes subjective interpretation and helps maintain reproducibility.

External Validity. The generalizability of our findings may be limited by the scope of the selected venues and time frame. To mitigate this threat, our review focused on prominent software engineering venues (ICSE, FSE, ASE, ISSTA, TSE, TOSEM, IST, JSS) and major digital libraries (ACM, IEEE Xplore, Web of Science, arXiv) across the 2021–2025 time frame, ensuring that the selected studies represent the current state of research on LLM-based unit test generation. However, the generalizability of the conclusions may be limited by the focus on unit testing tasks, the predominance of Java and Python in published datasets, and the sparse availability of large-scale benchmarks covering other languages or system-level testing. Besides, because the field is rapidly evolving, with many emerging studies released as preprints, the results reflect the best available evidence at the time of review.

9 Conclusion

The application of LLMs to unit test generation is a rapidly developing field with significant potential to automate testing and enhance software quality. This paper presents a systematic literature review of 116 primary studies on LLMs for unit test generation. This review begins by analyzing the types of LLMs used in primary studies, shedding light on researchers' preferences for different LLMs. Subsequently, we categorized a variety of techniques for adapting LLMs. Through our analysis, this review also identifies the limitations in this field and proposes a research roadmap outlining promising avenues for future exploration. In future work, we aim to expand this research to other types of testing, including performance and security testing.

Acknowledgments

This research is supported by the Fundamental Research Funds for the Central Universities (No.226-2025-00171)

References

[1] Saarang Agarwal, Pengyu Nie, and Meiyappan Nagappan. 2025. What Inputs Drive Effective LLM-based Unit Test Generation? *IEEE Software* 0, 0 (2025), 0.

[2] Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Unified Pre-training for Program Understanding and Generation. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2021, Online, June 6-11, 2021*. Association for Computational Linguistics, 2655–2668. doi:10.18653/V1/2021.NAACL-MAIN.211

[3] Toufique Ahmed, Jatin Ganhotra, Rangeet Pan, Avraham Shinnar, Saurabh Sinha, and Martin Hirzel. 2025. Otter: Generating Tests from Issues to Validate SWE Patches. In *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025 (Proceedings of Machine Learning Research, Vol. 267)*. PMLR / OpenReview.net, 0. https://proceedings.mlr.press/v267/ahmed25b.html

[4] Toufique Ahmed, Martin Hirzel, Rangeet Pan, Avraham Shinnar, and Saurabh Sinha. 2024. TDD-Bench Verified: Can LLMs Generate Tests for Issues Before They Get Resolved? *CoRR* abs/2412.02883 (2024). doi:10.48550/ARXIV.2412.02883

[5] Saranya Alagarsamy, Chakkrit Tantithamthavorn, and Aldeida Aleti. 2024. A3Test: Assertion-Augmented Automated Test case generation. *Information and Software Technology* 176 (2024), 107565. doi:10.1016/J.INFSOF.2024.107565

[6] Ahmed Aljohani, Anamul Haque Mollah, and Hyunsook Do. 2025. Assertion Messages with Large Language Models (LLMs) for Code. *CoRR* abs/2509.19673 (2025). doi:10.48550/ARXIV.2509.19673

[7] Nadia Alshahwan, Jubin Chheda, Anastasia Finogenova, Beliz Gokkaya, Mark Harman, Inna Harper, Alexandru Marginean, Shubho Sengupta, and Eddy Wang. 2024. Automated Unit Test Improvement using Large Language Models at Meta. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering, FSE 2024, Porto de Galinhas, Brazil, July 15-19, 2024*. ACM, 185–196. doi:10.1145/3663529.3663839

[8] Samuel Arcadinho, David Aparício, and Mariana Almeida. 2024. Automated Test Generation to Evaluate Tool-Augmented LLMs as Conversational AI Agents. *CoRR* abs/2409.15934 (2024). doi:10.48550/ARXIV.2409.15934

[9] arXiv Database. 2026. arXiv Database. https://arxiv.org.

[10] Jacob Austin, Augustus Odena, Maxwell I. Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie J. Cai, Michael Terry, Quoc V. Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. *CoRR* abs/2108.07732 (2021). https://arxiv.org/abs/2108.07732

[11] Vaishnavi Bhargava, Rajat Ghosh, and Debojyoti Dutta. 2024. CPP-UT-Bench: Can LLMs Write Complex Unit Tests in C++? *CoRR* abs/2412.02735 (2024). doi:10.48550/ARXIV.2412.02735

[12] Matteo Biagiola, Gianluca Ghislotti, and Paolo Tonella. 2025. Improving the Readability of Automatically Generated Tests Using Large Language Models. In *IEEE Conference on Software Testing, Verification and Validation, ICST 2025, Napoli, Italy, March 31 - April 4, 2025*. IEEE, 162–173. doi:10.1109/ICST62969.2025.10989020

[13] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*. https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html

[14] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*. https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html

[15] Cristian Cadar, Daniel Dunbar, and Dawson R. Engler. 2008. KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs. In *8th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2008, December 8-10, 2008, San Diego, California, USA, Proceedings*. USENIX Association, 209–224. http://www.usenix.org/events/osdi08/tech/full_papers/cadar/cadar.pdf

[16] Arda Celik and Qusay H Mahmoud. 2025. A Review of Large Language Models for Automated Test Case Generation. *Machine Learning and Knowledge Extraction* 7, 3 (2025), 97.

[17] Brian J. Chan, Mao Xun Huang, Jui-Hung Cheng, Chao-Ting Chen, and Hen-Hsen Huang. 2025. Efficient Beam Search for Large Language Models Using Trie-Based Decoding. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, EMNLP 2025, Suzhou, China, November 4-9, 2025*. Association for Computational Linguistics, 14795–14807. doi:10.18653/V1/2025.EMNLP-MAIN.748

[18] Honghui Chen, Kaiqing Chen, Fanlong Zhang, Tao Wang, and Lianglun Cheng. 2025. AgentTester: An LLM-Based Tool for Unit Test Generation with Automatically Generated Prompts. In *International Conference on Intelligent Computing*. 114–126.

[19] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. *CoRR* abs/2107.03374 (2021). https://arxiv.org/abs/2107.03374

[20] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2024. ChatUniTest: A Framework for LLM-Based Test Generation. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering, FSE 2024, Porto de Galinhas, Brazil, July 15-19, 2024*. ACM, 572–576. doi:10.1145/3663529.3663801

[21] Yang Chen, Ziqi Wang, Yanjie Jiang, Lin Yang, Yuteng Zheng, Jianyi Zhou, and Junjie Chen. 2025. Precisely Detecting Python Type Errors via LLM-based Unit Test Generation. *CoRR* abs/2507.02318 (2025). doi:10.48550/ARXIV.2507.02318

[22] Xiang Cheng, Fan Sang, Yizhuo Zhai, Xiaokuan Zhang, and Taesoo Kim. 2025. Rug: Turbo Llm for Rust Unit Test Generation. In *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 - May 6, 2025*. IEEE, 2983–2995. doi:10.1109/ICSE55347.2025.00097

[23] Paul F. Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. 2017. Deep Reinforcement Learning from Human Preferences. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*. 4299–4307. https://proceedings.neurips.cc/paper/2017/hash/d5e2c0adad503c91f91df240d0cd4e49-Abstract.html

[24] Bei Chu, Yang Feng, Kui Liu, Zifan Nan, Zhaoqiang Guo, and Baowen Xu. 2025. Large Language Models for Unit Test Generation: Achievements, Challenges, and the Road Ahead. *arXiv preprint arXiv:2511.21382* (2025).

[25] Bei Chu, Yang Feng, Kui Liu, Hange Shi, Zifan Nan, Zhaoqiang Guo, and Baowen Xu. 2025. Boosting Rust Unit Test Coverage through Hybrid Program Analysis and Large Language Models. *CoRR* abs/2506.09002 (2025). doi:10.48550/ARXIV.2506.09002

[26] Clover. 2026. OpenClover Code Coverage Platform for Java and Groovy. https://openclover.org/.

[27] Cobertura. 2026. Cobertura. https://cobertura.github.io/cobertura/.

[28] Henry Coles, Thomas Laurent, Christopher Henard, Mike Papadakis, and Anthony Ventresque. 2016. PIT: A Practical Mutation Testing Tool for Java (Demo). In *Proceedings of the 25th International Symposium on Software Testing and Analysis, ISSTA 2016, Saarbrücken, Germany, July 18-20, 2016*. ACM, 449–452. doi:10.1145/2931037.2948707

[29] CrossHair. 2026. CrossHair. https://github.com/pschanely/CrossHair.

[30] Arghavan Moradi Dakhel, Amin Nikanjam, Vahid Majdinasab, Foutse Khomh, and Michel C. Desmarais. 2024. Effective Test Generation Using Pre-trained Large Language Models and Mutation Testing. *Information and Software Technology* 171 (2024), 107468. doi:10.1016/J.INFSOF.2024.107468

[31] IEEE Xplore Database. 2026. IEEE Xplore Database. https://ieeexplore.ieee.org.

[32] Amirhossein Deljouyi, Roham Koohestani, Maliheh Izadi, and Andy Zaidman. 2025. Leveraging Large Language Models for Enhancing the Understandability of Generated Unit Tests. In *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 - May 6, 2025*. IEEE, 1449–1461. doi:10.1109/ICSE55347.2025.00032

[33] Yifei Deng, Renzhi Chen, Chao Xiao, Zhijie Yang, Yuanfeng Luo, Jingyue Zhao, Na Li, Zhong Wan, Yongbao Ai, Huadong Dai, and Lei Wang. 2024. LLM - TG: Towards Automated Test Case Generation for Processors Using Large Language Models. In *42nd IEEE International Conference on Computer Design, ICCD 2024, Milan, Italy, November 18-20, 2024*. IEEE, 389–396. doi:10.1109/ICCD63220.2024.00066

[34] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. 2023. Large Language Models Are Zero-Shot Fuzzers: Fuzzing Deep-Learning Libraries via Large Language Models. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2023, Seattle, WA, USA, July 17-21, 2023*. ACM, 423–435. doi:10.1145/3597926.3598067

[35] Yinlin Deng, Chunqiu Steven Xia, Chenyuan Yang, Shizhuo Dylan Zhang, Shujing Yang, and Lingming Zhang. 2024. Large Language Models are Edge-Case Generators: Crafting Unusual Programs for Fuzzing Deep Learning Libraries. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14-20, 2024*. ACM, 70:1–70:13. doi:10.1145/3597503.3623343

[36] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*. Association for Computational Linguistics, 4171–4186. doi:10.18653/V1/N19-1423

[37] Peng Di, Jianguo Li, Hang Yu, Wei Jiang, Wenting Cai, Yang Cao, Chaoyu Chen, Dajun Chen, Hongwei Chen, Liang Chen, Gang Fan, Jie Gong, Zi Gong, Wen Hu, Tingting Guo, Zhichao Lei, Ting Li, Zheng Li, Ming Liang, Cong Liao, Bingchang Liu, Jiachen Liu, Zhiwei Liu, Shaojun Lu, Min Shen, Guangpei Wang, Huan Wang, Zhi Wang, Zhaogui Xu, Jiawei Yang, Qing Ye, Gehao Zhang, Yu Zhang,

Zelin Zhao, Xunjin Zheng, Hailian Zhou, Lifu Zhu, and Xianying Zhu. 2024. CodeFuse-13B: A Pretrained Multi-lingual Code Large Language Model. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice, ICSE-SEIP 2024, Lisbon, Portugal, April 14-20, 2024*. ACM, 418–429. doi:10.1145/3639477.3639719

[38] Elizabeth Dinella, Gabriel Ryan, Todd Mytkowicz, and Shuvendu K. Lahiri. 2022. TOGA: A Neural Method for Test Oracle Generation. In *44th IEEE/ACM 44th International Conference on Software Engineering, ICSE 2022, Pittsburgh, PA, USA, May 25-27, 2022*. ACM, 2130–2141. doi:10.1145/3510003.3510141

[39] Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Jingyuan Ma, Rui Li, Heming Xia, Jingjing Xu, Zhiyong Wu, Baobao Chang, Xu Sun, Lei Li, and Zhifang Sui. 2024. A Survey on In-context Learning. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, November 12-16, 2024*. Association for Computational Linguistics, 1107–1128. doi:10.18653/V1/2024.EMNLP-MAIN.64

[40] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. 2023. Large Language Models for Software Engineering: Survey and Open Problems. In *IEEE/ACM International Conference on Software Engineering: Future of Software Engineering, ICSE-FoSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 31–53. doi:10.1109/ICSE-FOSE59343.2023.00008

[41] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020, Online Event, 16-20 November 2020 (Findings of ACL, Vol. EMNLP 2020)*. Association for Computational Linguistics, 1536–1547. doi:10.18653/V1/2020.FINDINGS-EMNLP.139

[42] Gordon Fraser and Andrea Arcuri. 2011. EvoSuite: Automatic Test Suite Generation for Object-Oriented Software. In *SIGSOFT/FSE'11 19th ACM SIGSOFT Symposium on the Foundations of Software Engineering (FSE-19) and ESEC'11: 13th European Software Engineering Conference (ESEC-13), Szeged, Hungary, September 5-9, 2011*. ACM, 416–419. doi:10.1145/2025113.2025179

[43] Gordon Fraser and Andrea Arcuri. 2015. 1600 Faults in 100 Projects: Automatically Finding Faults While Achieving High Coverage with EvoSuite. *Empirical Software Engineering* 20, 3 (2015), 611–639. doi:10.1007/S10664-013-9288-2

[44] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Scott Yih, Luke Zettlemoyer, and Mike Lewis. 2023. InCoder: A Generative Model for Code Infilling and Synthesis. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net. https://openreview.net/forum?id=hQwb-lbM6EL

[45] Jia Fu, Xinyu Yang, Hongzhi Zhang, Yahui Liu, Jingyuan Zhang, Qi Wang, Fuzheng Zhang, and Guorui Zhou. 2025. Klear-CodeTest: Scalable Test Case Generation for Code Reinforcement Learning. *CoRR* abs/2508.05710 (2025). doi:10.48550/ARXIV.2508.05710

[46] Shuzheng Gao, Chaozheng Wang, Cuiyun Gao, Xiaoqian Jiao, Chun Yong Chong, Shan Gao, and Michael R. Lyu. 2025. The Prompt Alchemist: Automated LLM-Tailored Prompt Optimization for Test Case Generation. *CoRR* abs/2501.01329 (2025). doi:10.48550/ARXIV.2501.01329

[47] Yi Gao, Xing Hu, Zirui Chen, Tongtong Xu, and Xiaohu Yang. 2025. Vulnerability-Triggering Test Case Generation from Third-Party Libraries. In *IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering, Forge@ICSE 2025, Ottawa, ON, Canada, April 27-28, 2025*. IEEE, 125–135. doi:10.1109/FORGE66646.2025.00021

[48] Yi Gao, Xing Hu, Xiaohu Yang, and Xin Xia. 2025. Automated Unit Test Refactoring. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 713–733. doi:10.1145/3715750

[49] GitHub. 2026. Github copilot. https://copilot.github.com.

[50] Milos Gligoric, Vilas Jagannath, and Darko Marinov. 2010. MuTMuT: Efficient Exploration for Mutation Testing of Multithreaded Code. In *Third International Conference on Software Testing, Verification and Validation, ICST 2010, Paris, France, April 7-9, 2010*. IEEE Computer Society, 55–64. doi:10.1109/ICST.2010.33

[51] Gwihwan Go, Quan Zhang, Chijin Zhou, Zhao Wei, and Yu Jiang. 2025. LSPRAG: LSP-Guided RAG for Language-Agnostic Real-Time Unit Test Generation. *CoRR* abs/2510.22210 (2025). doi:10.48550/ARXIV.2510.22210

[52] Daniele Gorla, Shivam Kumar, Pietro Nicolaus Roselli Lorenzini, and Alireza Alipourfaz. 2025. CubeTesterAI: Automated JUnit Test Generation Using the LLaMA Model. In *IEEE Conference on Software Testing, Verification and Validation, ICST 2025, Napoli, Italy, March 31 - April 4, 2025*. IEEE, 565–576. doi:10.1109/ICST62969.2025.10988978

[53] GPT-3.5. 2022. OpenAI. https://platform.openai.com/docs/models/gpt-3-5.

[54] Siqi Gu, Chunrong Fang, Quanjun Zhang, Fangyuan Tian, Jianyi Zhou, and Zhenyu Chen. 2024. TestART: Improving LLM-based Unit Test via Co-evolution of Automated Generation and Repair Iteration. *CoRR* abs/2408.03095 (2024). doi:10.48550/ARXIV.2408.03095

[55] Sijia Gu, Noor Nashid, and Ali Mesbah. 2025. LLM Test Generation via Iterative Hybrid Program Analysis. *CoRR* abs/2503.13580 (2025). doi:10.48550/ARXIV.2503.13580

[56] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified Cross-Modal Pre-training for Code Representation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022, Dublin, Ireland, May 22-27, 2022*. Association for Computational Linguistics, 7212–7225. doi:10.18653/V1/2022.ACL-LONG.499

[57] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin B. Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. 2021. GraphCodeBERT: Pre-training Code Representations with Data Flow. In *9th International Conference on Learning Representations, ICLR*

2021, Virtual Event, Austria, May 3-7, 2021. OpenReview.net. https://openreview.net/forum?id=jLoC4ez43PZ

[58] Hazim Hanif and Sergio Maffeis. 2022. VulBERTa: Simplified Source Code Pre-Training for Vulnerability Detection. In *International Joint Conference on Neural Networks, IJCNN 2022, Padua, Italy, July 18-23, 2022*. IEEE, 1–8. doi:10.1109/IJCNN55064.2022.9892280

[59] Mark Harman, Jillian Ritchey, Inna Harper, Shubho Sengupta, Ke Mao, Abhishek Gulati, Christopher Foster, and Hervé Robert. 2025. Mutation-Guided LLM-based Test Generation at Meta. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering, FSE Companion 2025, Clarion Hotel Trondheim, Trondheim, Norway, June 23-28, 2025*. ACM, 180–191. doi:10.1145/3696630.3728544

[60] Ishrak Hayet, Adam Scott, and Marcelo d'Amorim. 2025. ChatAssert: LLM-Based Test Oracle Generation With External Tools Assistance. *IEEE Trans. Software Eng.* 51, 1 (2025), 305–319. doi:10.1109/TSE.2024.3519159

[61] Junda He, Xin Zhou, Bowen Xu, Ting Zhang, Kisub Kim, Zhou Yang, Ferdian Thung, Ivana Clairine Irsan, and David Lo. 2024. Representation Learning for Stack Overflow Posts: How Far Are We? *ACM Trans. Softw. Eng. Methodol.* 33, 3 (2024), 69:1–69:24. doi:10.1145/3635711

[62] Yifeng He, Jiabo Huang, Yuyang Rong, Yiwen Guo, Ethan Wang, and Hao Chen. 2024. UniTSyn: A Large-Scale Dataset Capable of Enhancing the Prowess of Large Language Models for Program Testing. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024, Vienna, Austria, September 16-20, 2024*. ACM, 1061–1072. doi:10.1145/3650212.3680342

[63] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. 2021. Measuring Coding Challenge Competence With APPS. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, virtual*. https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/c24cd76e1ce41366a4bbe8a49b02a028-Abstract-round2.html

[64] Yoshiki Higo, Shinsuke Matsumoto, Shinji Kusumoto, and Kazuya Yasuda. 2022. Constructing Dataset of Functionally Equivalent Java Methods Using Automated Test Generation Techniques. In *19th IEEE/ACM International Conference on Mining Software Repositories, MSR 2022, Pittsburgh, PA, USA, May 23-24, 2022*. ACM, 682–686. doi:10.1145/3524842.3528015

[65] Soneya Binta Hossain and Matthew B. Dwyer. 2025. TOGLL: Correct and Strong Test Oracle Generation with LLMS. In *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 - May 6, 2025*. IEEE, 1475–1487. doi:10.1109/ICSE55347.2025.00098

[66] Soneya Binta Hossain, Antonio Filieri, Matthew B. Dwyer, Sebastian G. Elbaum, and Willem Visser. 2023. Neural-Based Test Oracle Generation: A Large-Scale Evaluation and Lessons Learned. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2023, San Francisco, CA, USA, December 3-9, 2023*. ACM, 120–132. doi:10.1145/3611643.3616265

[67] Soneya Binta Hossain, Raygan Taylor, and Matthew B. Dwyer. 2025. Doc2OracLL: Investigating the Impact of Documentation on LLM-Based Test Oracle Generation. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 1870–1891. doi:10.1145/3729354

[68] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 220:1–220:79. doi:10.1145/3695988

[69] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-Efficient Transfer Learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA (Proceedings of Machine Learning Research, Vol. 97)*. PMLR, 2790–2799. http://proceedings.mlr.press/v97/houlsby19a.html

[70] Jeremy Howard and Sebastian Ruder. 2018. Universal Language Model Fine-tuning for Text Classification. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics, ACL 2018, Melbourne, Australia, July 15-20, 2018, Volume 1: Long Papers*. Association for Computational Linguistics, 328–339. doi:10.18653/V1/P18-1031

[71] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net. https://openreview.net/forum?id=nZeVKeeFYf9

[72] Xing Hu, Zhuang Liu, Xin Xia, Zhongxin Liu, Tongtong Xu, and Xiaohu Yang. 2023. Identify and Update Test Cases When Production Code Changes: A Transformer-Based Approach. In *38th IEEE/ACM International Conference on Automated Software Engineering, ASE 2023, Luxembourg, September 11-15, 2023*. IEEE, 1111–1122. doi:10.1109/ASE56229.2023.00165

[73] Xing Hu, Feifei Niu, Junkai Chen, Xin Zhou, Junwei Zhang, Junda He, Xin Xia, and David Lo. 2025. Assessing and Advancing Benchmarks for Evaluating Large Language Models in Software Engineering Tasks. *CoRR* abs/2505.08903 (2025). doi:10.48550/ARXIV.2505.08903

[74] Yi Hu, Haotong Yang, Zhouchen Lin, and Muhan Zhang. 2023. Code Prompting: a Neural Symbolic Method for Complex Reasoning in Large Language Models. *CoRR* abs/2305.18507 (2023). doi:10.48550/ARXIV.2305.18507

[75] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. CodeSearchNet Challenge: Evaluating the State of Semantic Code Search. *CoRR* abs/1909.09436 (2019). http://arxiv.org/abs/1909.09436

[76] JaCoCo. 2026. JaCoCo. https://github.com/jacoco/jacoco.

[77] Kush Jain and Claire Le Goues. 2025. TestForge: Feedback-Driven, Agentic Test Suite Generation. *CoRR* abs/2503.14713 (2025). doi:10.48550/ARXIV.2503.14713

[78] Kush Jain, Gabriel Synnaeve, and Baptiste Rozière. 2025. TestGenEval: A Real World Unit Test Generation and Test Completion Benchmark. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net. https://openreview.net/forum?id=7o6SG5gVev

[79] Javalang. 2026. Javalang. https://github.com/c2nes/javalang.

[80] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2024. A Survey on Large Language Models for Code Generation. *CoRR* abs/2406.00515 (2024). doi:10.48550/ARXIV.2406.00515

[81] Zongze Jiang, Ming Wen, Jialun Cao, Xuanhua Shi, and Hai Jin. 2024. Towards Understanding the Effectiveness of Large Language Models on Directed Test Input Generation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE 2024, Sacramento, CA, USA, October 27 - November 1, 2024*. ACM, 1408–1420. doi:10.1145/3691620.3695513

[82] Can Jin, Ying Li, Mingyu Zhao, Shiyu Zhao, Zhenting Wang, Xiaoxiao He, Ligong Han, Tong Che, and Dimitris N. Metaxas. 2025. LoR-VP: Low-Rank Visual Prompting for Efficient Vision Model Adaptation. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net. https://openreview.net/forum?id=5btFIv2PNb

[83] René Just, Darioush Jalali, and Michael D. Ernst. 2014. Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs. In *International Symposium on Software Testing and Analysis, ISSTA '14, San Jose, CA, USA - July 21 - 26, 2014*. ACM, 437–440. doi:10.1145/2610384.2628055

[84] Aditya Kanade, Petros Maniatis, Gogul Balakrishnan, and Kensen Shi. 2020. Learning and Evaluating Contextual Embedding of Source Code. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event (Proceedings of Machine Learning Research, Vol. 119)*. PMLR, 5110–5121. http://proceedings.mlr.press/v119/kanade20a.html

[85] Sungmin Kang, Juyeon Yoon, and Shin Yoo. 2023. Large Language Models are Few-shot Testers: Exploring LLM-based General Bug Reproduction. In *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 2312–2323. doi:10.1109/ICSE48619.2023.00194

[86] Shaker Mahmud Khandaker, Fitsum Meshesha Kifetew, Davide Prandi, and Angelo Susi. 2025. AugmenTest: Enhancing Tests with LLM-Driven Oracles. In *IEEE Conference on Software Testing, Verification and Validation, ICST 2025, Napoli, Italy, March 31 - April 4, 2025*. IEEE, 279–289. doi:10.1109/ICST62969.2025.10988926

[87] Djamel Eddine Khelladi, Charly Reux, and Mathieu Acher. 2025. Unify and Triumph: Polyglot, Diverse, and Self-Consistent Generation of Unit Tests with LLMs. *CoRR* abs/2503.16144 (2025). doi:10.48550/ARXIV.2503.16144

[88] András Kicsi, László Vidács, and Tibor Gyimóthy. 2020. TestRoutes: A Manually Curated Method Level Dataset for Test-to-Code Traceability. In *MSR '20: 17th International Conference on Mining Software Repositories, Seoul, Republic of Korea, 29-30 June, 2020*. ACM, 593–597. doi:10.1145/3379597.3387488

[89] Myeongsoo Kim, Tyler Stennett, Saurabh Sinha, and Alessandro Orso. 2025. A Multi-Agent Approach for REST API Testing with Semantic Graphs and LLM-Driven Inputs. In *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 - May 6, 2025*. IEEE, 1409–1421. doi:10.1109/ICSE55347.2025.00179

[90] Konstantinos Kitsios, Marco Castelluccio, and Alberto Bacchelli. 2025. Automated Generation of Issue-Reproducing Tests by Combining LLMs and Search-Based Testing. *CoRR* abs/2509.01616 (2025). doi:10.48550/ARXIV.2509.01616

[91] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large Language Models are Zero-Shot Reasoners. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*. http://papers.nips.cc/paper_files/paper/2022/hash/8bb0d291acd4acf06ef112099c16f326-Abstract-Conference.html

[92] Michael Konstantinou, Renzo Degiovanni, and Mike Papadakis. 2024. Do LLMs Generate Test Oracles That Capture the Actual or the Expected Program Behaviour? *CoRR* abs/2410.21136 (2024). doi:10.48550/ARXIV.2410.21136

[93] Michael Konstantinou, Renzo Degiovanni, Jie M. Zhang, Mark Harman, and Mike Papadakis. 2025. YATE: The Role of Test Repair in LLM-Based Unit Test Generation. *CoRR* abs/2507.18316 (2025). doi:10.48550/ARXIV.2507.18316

[94] Nischal Ashok Kumar and Andrew S. Lan. 2024. Using Large Language Models for Student-Code Guided Test Case Generation in Computer Science Education. *CoRR* abs/2402.07081 (2024). doi:10.48550/ARXIV.2402.07081

[95] Cuong Chi Le, Cuong Duc Van, Tung Duy Vu, Thai Minh Pham Vu, Hoang Nhat Phan, Huy Nhat Phan, and Tien N. Nguyen. 2025. TestWeaver: Execution-aware, Feedback-driven Regression Testing Generation with Large Language Models. *CoRR* abs/2508.01255 (2025). doi:10.48550/ARXIV.2508.01255

[96] Tri Le, Thien Tran, Duy Cao, Vy Le, Tien N. Nguyen, and Vu Nguyen. 2024. KAT: Dependency-Aware Automated API Testing with Large Language Models. In *IEEE Conference on Software Testing, Verification and Validation, ICST 2024, Toronto, ON, Canada, May 27-31, 2024*. IEEE, 82–92. doi:10.1109/ICST60714.2024.00017

[97] Dongjun Lee, Changho Hwang, and Kimin Lee. 2025. Learning to Generate Unit Test via Adversarial Reinforcement Learning. *CoRR* abs/2508.21107 (2025). doi:10.48550/ARXIV.2508.21107

[98] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. 2023. CodaMosa: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models. In *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 919–931. doi:10.1109/ICSE48619.2023.00085

[99] Anji Li, Mingwei Liu, Zhenxi Chen, Zheng Pei, Zike Li, Dekun Dai, Yanlin Wang, and Zibin Zheng. 2025. KTester: Leveraging Domain and Testing Knowledge for More Effective LLM-based Test Generation. *CoRR* abs/2511.14224 (2025). doi:10.48550/ARXIV.2511.14224

[100] Jiageng Li, Zhen Dong, Chong Wang, Haozhen You, Cen Zhang, Yang Liu, and Xin Peng. 2025. LLM Based Input Space Partitioning Testing for Library APIs. In *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 - May 6, 2025*. IEEE, 1436–1448. doi:10.1109/ICSE55347.2025.00153

[101] Kefan Li and Yuan Yuan. 2024. Large Language Models as Test Case Generators: Performance Evaluation and Enhancement. *CoRR* abs/2404.13340 (2024). doi:10.48550/ARXIV.2404.13340

[102] Qingyao Li, Xinyi Dai, Weiwen Liu, Xiangyang Li, Yasheng Wang, Ruiming Tang, Yong Yu, and Weinan Zhang. 2025. ATGen: Adversarial Reinforcement Learning for Test Case Generation. *CoRR* abs/2510.14635 (2025). doi:10.48550/ARXIV.2510.14635

[103] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, Qian Liu, Evgenii Zheltonozhskii, Terry Yue Zhuo, Thomas Wang, Olivier Dehaene, Mishig Davaadorj, Joel Lamy-Poirier, João Monteiro, Oleh Shliazhko, Nicolas Gontier, Nicholas Meade, Armel Zebaze, Ming-Ho Yee, Logesh Kumar Umapathi, Jian Zhu, Benjamin Lipkin, Muhtasham Oblokulov, Zhiruo Wang, Rudra Murthy V, Jason T. Stillerman, Siva Sankalp Patel, Dmitry Abulkhanov, Marco Zocca, Manan Dey, Zhihan Zhang, Nour Fahmy, Urvashi Bhattacharyya, Wenhao Yu, Swayam Singh, Sasha Luccioni, Paulo Villegas, Maxim Kunakov, Fedor Zhdanov, Manuel Romero, Tony Lee, Nadav Timor, Jennifer Ding, Claire Schlesinger, Hailey Schoelkopf, Jan Ebert, Tri Dao, Mayank Mishra, Alex Gu, Jennifer Robinson, Carolyn Jane Anderson, Brendan Dolan-Gavitt, Danish Contractor, Siva Reddy, Daniel Fried, Dzmitry Bahdanau, Yacine Jernite, Carlos Muñoz Ferrandis, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. 2023. StarCoder: May The Source Be with You! *Transactions on Machine Learning Research* 2023 (2023). https://openreview.net/forum?id=KoFOg41haE

[104] Tsz On Li, Wenxi Zong, Yibo Wang, Haoye Tian, Ying Wang, Shing-Chi Cheung, and Jeff Kramer. 2023. Nuances are the Key: Unlocking ChatGPT to Find Failure-Inducing Tests with Differential Prompting. In *38th IEEE/ACM International Conference on Automated Software Engineering, ASE 2023, Luxembourg, September 11-15, 2023*. IEEE, 14–26. doi:10.1109/ASE56229.2023.00089

[105] Vladislav Lialin, Vijeta Deshpande, and Anna Rumshisky. 2023. Scaling Down to Scale Up: A Guide to Parameter-Efficient Fine-Tuning. *CoRR* abs/2303.15647 (2023). doi:10.48550/ARXIV.2303.15647

[106] Dianshu Liao, Xin Yin, Shidong Pan, Chao Ni, Zhenchang Xing, and Xiaoyu Sun. 2025. Navigating the Labyrinth: Path-Sensitive Unit Test Generation with Large Language Models. *CoRR* abs/2509.23812 (2025). doi:10.48550/ARXIV.2509.23812

[107] ACM Digital Library. 2026. ACM Digital Library. https://dl.acm.org.

[108] Derrick Lin, James Koppel, Angela Chen, and Armando Solar-Lezama. 2017. QuixBugs: A Multi-lingual Program Repair Benchmark Set Based on the Quixey Challenge. In *Proceedings Companion of the 2017 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity, SPLASH 2017, Vancouver, BC, Canada, October 23 - 27, 2017*. ACM, 55–56. doi:10.1145/3135932.3135941

[109] Jinwei Liu, Chao Li, Rui Chen, Shaofeng Li, Bin Gu, and Mengfei Yang. 2025. STRUT: Structured Seed Case Guided Unit Test Generation for C Programs using LLMs. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 2113–2135. doi:10.1145/3728970

[110] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*. http://papers.nips.cc/paper_files/paper/2023/hash/43e9d647ccd3e4b7b5baab53f0368686-Abstract-Conference.html

[111] Jun Liu, Jiwei Yan, Yuanyuan Xie, Jun Yan, and Jian Zhang. 2024. Fix the Tests: Augmenting LLMs to Repair Test Cases with Static Collector and Neural Reranker. In *35th IEEE International Symposium on Software Reliability Engineering, ISSRE 2024, Tsukuba, Japan, October 28-31, 2024*. IEEE, 367–378. doi:10.1109/ISSRE62328.2024.00043

[112] Kaibo Liu, Zhenpeng Chen, Yiyang Liu, Jie M. Zhang, Mark Harman, Yudong Han, Yun Ma, Yihong Dong, Ge Li, and Gang Huang. 2025. LLM-Powered Test Case Generation for Detecting Bugs in Plausible Programs. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, Vienna, Austria, July 27 - August 1, 2025*. Association for Computational Linguistics, 430–440. https://aclanthology.org/2025.acl-long.20/

[113] Kaibo Liu, Yudong Han, Yiyang Liu, Jie M. Zhang, Zhenpeng Chen, Federica Sarro, Gang Huang, and Yun Ma. 2024. TrickyBugs: A Dataset of Corner-case Bugs in Plausible Programs. In *21st IEEE/ACM International Conference on Mining Software Repositories, MSR 2024, Lisbon, Portugal, April 15-16, 2024*. ACM, 113–117. doi:10.1145/3643991.3644870

[114] Kaibo Liu, Yiyang Liu, Zhenpeng Chen, Jie M. Zhang, Yudong Han, Yun Ma, Ge Li, and Gang Huang. 2024. LLM-Powered Test Case Generation for Detecting Tricky Bugs. *CoRR* abs/2404.10304 (2024). doi:10.48550/ARXIV.2404.10304

[115] Runlin Liu, Zhe Zhang, Yunge Hu, Yuhang Lin, Xiang Gao, and Hailong Sun. 2025. LLM-based Unit Test Generation for Dynamically-Typed Programs. *CoRR* abs/2503.14000 (2025). doi:10.48550/ARXIV.2503.14000

[116] Zhongxin Liu, Kui Liu, Xin Xia, and Xiaohu Yang. 2023. Towards More Realistic Evaluation for Neural Test Oracle Generation. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2023, Seattle, WA, USA, July 17-21, 2023*. ACM, 589–600. doi:10.1145/3597926.3598080

[117] LLVM. 2026. LLVM. https://github.com/llvm/llvm-project.

[118] Andrea Lops, Fedelucio Narducci, Azzurra Ragone, Michelantonio Trizio, and Claudio Bartolini. 2025. A System for Automated Unit Test Generation using Large Language Models and Assessment of Generated Test Suites. In *IEEE International Conference on Software Testing, Verification and Validation, ICST 2025 - Workshops, Naples, Italy, March 31 - April 4, 2025*. IEEE, 29–36. doi:10.1109/ICSTW64639.2025.10962454

[119] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin B. Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, Ming Gong, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, and Shujie Liu. 2021. CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, virtual*. https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/c16a5320fa475530d9583c34fd356ef5-Abstract-round1.html

[120] Stephan Lukasczyk and Gordon Fraser. 2022. Pynguin: Automated Unit Test Generation for Python. In *44th IEEE/ACM International Conference on Software Engineering: Companion Proceedings, ICSE Companion 2022, Pittsburgh, PA, USA, May 22-24, 2022*. ACM/IEEE, 168–172. doi:10.1145/3510454.3516829

[121] Hongqin Lyu, Yunlin Du, Yonghao Wang, Zhiteng Chao, Tiancheng Wang, and Huawei Li. 2025. AssertFix: Empowering Automated Assertion Fix via Large Language Models. *CoRR* abs/2509.23972 (2025). doi:10.48550/ARXIV.2509.23972

[122] Fernanda Madeiral, Simon Urli, Marcelo de Almeida Maia, and Martin Monperrus. 2019. BEARS: An Extensible Java Bug Benchmark for Automatic Program Repair Studies. In *26th IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER 2019, Hangzhou, China, February 24-27, 2019*. IEEE, 468–478. doi:10.1109/SANER.2019.8667991

[123] Jorge Marques Prates, Bruno Oliveira Costa Jimez, Silvana Morita Melo, Igor Roberto Michalski Souza, and Rogério Eduardo Garcia. [n. d.]. Systematic Mapping of Empirical Research on LLM-based Test Generation: Research Gaps, Methodologies, and Emerging Trends. *Methodologies, and Emerging Trends* ([n. d.]).

[124] Zifan Nan, Zhaoqiang Guo, Kui Liu, and Xin Xia. 2025. Test Intention Guided LLM-Based Unit Test Generation. In *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 - May 6, 2025*. IEEE, 1026–1038. doi:10.1109/ICSE55347.2025.00243

[125] Noor Nashid, Islem Bouzenia, Michael Pradel, and Ali Mesbah. 2025. Issue2Test: Generating Reproducing Test Cases from Issue Reports. *CoRR* abs/2503.16320 (2025). doi:10.48550/ARXIV.2503.16320

[126] Noor Nashid, Mifta Sintaha, and Ali Mesbah. 2023. Retrieval-Based Prompt Selection for Code-Related Few-Shot Learning. In *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 2450–2462. doi:10.1109/ICSE48619.2023.00205

[127] Pengyu Nie, Rahul Banerjee, Junyi Jessy Li, Raymond J. Mooney, and Milos Gligoric. 2023. Learning Deep Semantics for Test Completion. In *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 2111–2123. doi:10.1109/ICSE48619.2023.00178

[128] Erik Nijkamp, Hiroaki Hayashi, Caiming Xiong, Silvio Savarese, and Yingbo Zhou. 2023. CodeGen2: Lessons for Training LLMs on Programming and Natural Languages. *CoRR* abs/2305.02309 (2023). doi:10.48550/ARXIV.2305.02309

[129] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2023. CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net. https://openreview.net/forum?id=iaYcJKpY2B_

[130] Web of Science Database. 2026. Web of Science Database. https://www.webofscience.com.

[131] Wonseok Oh and Hakjoo Oh. 2024. Towards Effective Static Type-Error Detection for Python. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1808–1820.

[132] OpenAI. 2023. GPT-4 Technical Report. *CoRR* abs/2303.08774 (2023). doi:10.48550/ARXIV.2303.08774

[133] Wendkûuni C. Ouédraogo, Abdoul Kader Kaboré, Haoye Tian, Yewei Song, Anil Koyuncu, Jacques Klein, David Lo, and Tegawendé F. Bissyandé. 2024. Large-scale, Independent and Comprehensive Study of the Power of LLMs for Test Case Generation. *CoRR* abs/2407.00225 (2024). doi:10.48550/ARXIV.2407.00225

[134] Wendkûuni C. Ouédraogo, Laura Plein, Abdoul Kader Kaboré, Andrew Habib, Jacques Klein, David Lo, and Tegawendé F. Bissyandé. 2025. Enriching Automatic Test Case Generation by Extracting Relevant Test Inputs from Bug Reports. *Empirical Software Engineering* 30, 3 (2025), 85. doi:10.1007/S10664-025-10635-Z

[135] Carlos Pacheco and Michael D. Ernst. 2007. Randoop: Feedback-Directed Random Testing for Java. In *Companion to the 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2007, October 21-25, 2007, Montreal, Quebec, Canada*. ACM, 815–816. doi:10.1145/1297846.1297902

[136] Ciprian Paduraru, Alin Stefanescu, and Augustin Jianu. 2024. Unit Test Generation using Large Language Models for Unity Game Development. In *Proceedings of the 1st ACM International Workshop on Foundations of Applied Software Engineering for Games, FaSE4Games 2024, Porto de Galinhas, Brazil, 16 July 2024*. ACM. doi:10.1145/3663532.3664466

[137] Rangeet Pan, Myeongsoo Kim, Rahul Krishna, Raju Pavuluri, and Saurabh Sinha. 2025. ASTER: Natural and Multi-Language Unit Test Generation with LLMs. In *47th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice, SEIP@ICSE 2025, Ottawa, ON, Canada, April 27 - May 3, 2025*. IEEE, 413–424. doi:10.1109/ICSE-SEIP66354.2025.00042

[138] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. 2024. Unifying Large Language Models and Knowledge Graphs: A Roadmap. *IEEE Transactions on Knowledge and Data Engineering* 36, 7 (2024), 3580–3599. doi:10.1109/TKDE.2024.3352100

[139] Anthony Peruma, Khalid Almalki, Christian D. Newman, Mohamed Wiem Mkaouer, Ali Ouni, and Fabio Palomba. 2020. TsDetect: An Open Source Test Smells Detection Tool. In *ESEC/FSE '20: 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual Event, USA, November 8-13, 2020*. ACM, 1650–1654. doi:10.1145/3368089.3417921

[140] Juan Altmayer Pizzorno and Emery D. Berger. 2023. SlipCover: Near Zero-Overhead Code Coverage for Python. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2023, Seattle, WA, USA, July 17-21, 2023*. ACM, 1195–1206. doi:10.1145/3597926.3598128

[141] Juan Altmayer Pizzorno and Emery D. Berger. 2025. CoverUp: Effective High Coverage Test Generation for Python. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 2897–2919. doi:10.1145/3729398

[142] Archiki Prasad, Elias Stengel-Eskin, Justin Chih-Yao Chen, Zaid Khan, and Mohit Bansal. 2025. Learning to Generate Unit Tests for Automated Debugging. *CoRR* abs/2502.01619 (2025). doi:10.48550/ARXIV.2502.01619

[143] Severin Primbs, Benedikt Fein, and Gordon Fraser. 2025. AsserT5: Test Assertion Generation Using a Fine-Tuned Code Language Model. In *IEEE/ACM International Conference on Automation of Software Test, AST@ICSE 2025, Ottawa, ON, Canada, April 28-29, 2025*. IEEE, 12–23. doi:10.1109/AST66626.2025.00008

[144] Binhang Qi, Yun Lin, Xinyi Weng, Yuhuan Huang, Chenyan Liu, Hailong Sun, and Jin Song Dong. 2025. Intention-Driven Generation of Project-Specific Test Cases. *CoRR* abs/2507.20619 (2025). doi:10.48550/ARXIV.2507.20619

[145] Shaojian Qiu, Weibiao Chen, Haiyang Liu, Shaosheng Wang, and Han Huang. 2026. Boosting Unit Test Generation via Structure-Aware Fine-Tuning of Pre-Trained Model. *Information and Software Technology* 190 (2026), 107948. doi:10.1016/J.INFSOF.2025.107948

[146] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language Models are Unsupervised Multitask Learners. *OpenAI blog* 1, 8 (2019), 9.

[147] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research* 21 (2020), 140:1–140:67. https://jmlr.org/papers/v21/20-074.html

[148] Dudekula Mohammad Rafi, Katam Reddy Kiran Moses, Kai Petersen, and Mika Mäntylä. 2012. Benefits and Limitations of Automated Software Testing: Systematic Literature Review and Practitioner Survey. In *7th International Workshop on Automation of Software Test, AST 2012, Zurich, Switzerland, June 2-3, 2012*. IEEE Computer Society, 36–42. doi:10.1109/IWAST.2012.6228988

[149] Shanto Rahman, Sachit Kuhar, Berk Çirisci, Pranav Garg, Shiqi Wang, Xiaofei Ma, Anoop Deoras, and Baishakhi Ray. 2025. UTFix: Change Aware Unit Test Repairing using LLM. *Proceedings of the ACM on Software Engineering* 9, OOPSLA1 (2025), 143–168. doi:10.1145/3720419

[150] Alex Rav-Acha, Yael Pritch, Dani Lischinski, and Shmuel Peleg. 2007. Dynamosaicing: Mosaicing of Dynamic Scenes. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 29, 10 (2007), 1789–1801. doi:10.1109/TPAMI.2007.1091

[151] Shaheer Rehan, Baidaa Al-Bander, and Amro Al-Said Ahmad. 2025. Harnessing Large Language Models for Automated Software Testing: A Leap Towards Scalable Test Case Generation. *Electronics (2079-9292)* 14, 7 (2025).

[152] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton-Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2023. Code Llama: Open Foundation Models for Code. *CoRR* abs/2308.12950 (2023). doi:10.48550/ARXIV.2308.12950

[153] Gabriel Ryan, Siddhartha Jain, Mingyue Shang, Shiqi Wang, Xiaofei Ma, Murali Krishna Ramanathan, and Baishakhi Ray. 2024. Code-Aware Prompting: A Study of Coverage-Guided Test Generation in Regression Setting using LLM. *Proc. ACM Softw. Eng.* 1, FSE (2024), 951–971. doi:10.1145/3643769

[154] Ripon K. Saha, Yingjun Lyu, Wing Lam, Hiroaki Yoshida, and Mukul R. Prasad. 2018. Bugs.jar: A Large-Scale, Diverse Dataset of Real-World Java Bugs. In *Proceedings of the 15th International Conference on Mining Software Repositories, MSR 2018, Gothenburg, Sweden, May 28-29, 2018*. ACM, 10–13. doi:10.1145/3196398.3196473

[155] Claude Sammut. 2010. Greedy Search. In *Encyclopedia of Machine Learning*. Springer, 482–483.

[156] Robson Santos, Ítalo Santos, Cleyton V. C. de Magalhães, and Ronnie de Souza Santos. 2024. Are We Testing or Being Tested? Exploring the Practical Applications of Large Language Models in Software Testing. In *IEEE Conference on Software Testing, Verification and Validation, ICST 2024, Toronto, ON, Canada, May 27-31, 2024*. IEEE, 353–360. doi:10.1109/ICST60714.2024.00039

[157] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2023. Adaptive Test Generation Using a Large Language Model. *CoRR* abs/2302.06527 (2023). doi:10.48550/ARXIV.2302.06527

[158] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2024. An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. *IEEE Transactions on Software Engineering* 50, 1 (2024), 85–105. doi:10.1109/TSE.2023.3334955

[159] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2024. An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. *IEEE Transactions on Software Engineering* 50, 1 (2024), 85–105. doi:10.1109/TSE.2023.3334955

[160] Ye Shang, Quanjun Zhang, Chunrong Fang, Siqi Gu, Jianyi Zhou, and Zhenyu Chen. 2025. A Large-Scale Empirical Study on Fine-Tuning Large Language Models for Unit Testing. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 1678–1700. doi:10.1145/3728951

[161] Ze Sheng, Zhicheng Chen, Shuning Gu, Heqing Huang, Guofei Gu, and Jeff Huang. 2026. LLMs in Software Security: A Survey of Vulnerability Detection Techniques and Insights. *Comput. Surveys* 58, 5 (2026), 134:1–134:35. doi:10.1145/3769082

[162] Betim Sherifi, Khaled Slhoub, and Fitzroy Nembhard. 2025. The Potential of LLMs in Automating Software Testing: From Generation to Reporting. *CoRR* abs/2501.00217 (2025). doi:10.48550/ARXIV.2501.00217

[163] Jiho Shin, Reem Aleithan, Hadi Hemmati, and Song Wang. 2024. Retrieval-Augmented Test Generation: How Far Are We? *CoRR* abs/2409.12682 (2024). doi:10.48550/ARXIV.2409.12682

[164] Jiho Shin, Sepehr Hashtroudi, Hadi Hemmati, and Song Wang. 2024. Domain Adaptation for Code Model-Based Unit Test Case Generation. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024, Vienna, Austria, September 16-20, 2024*. ACM, 1211–1222. doi:10.1145/3650212.3680354

[165] Jiho Shin, Clark Tang, Tahmineh Mohati, Maleknaz Nayebi, Song Wang, and Hadi Hemmati. 2025. Prompt Engineering or Fine-Tuning: An Empirical Assessment of LLMs for Code. In *22nd IEEE/ACM International Conference on Mining Software Repositories, MSR@ICSE 2025, Ottawa, ON, Canada, April 28-29, 2025*. IEEE, 490–502. doi:10.1109/MSR66628.2025.00082

[166] Yan Shoshitaishvili, Ruoyu Wang, Christopher Salls, Nick Stephens, Mario Polino, Andrew Dutcher, John Grosen, Siji Feng, Christophe Hauser, Christopher Krügel, and Giovanni Vigna. 2016. SOK: (State of) The Art of War: Offensive Techniques in Binary Analysis. In *IEEE Symposium on Security and Privacy, SP 2016, San Jose, CA, USA, May 22-26, 2016*. IEEE Computer Society, 138–157. doi:10.1109/SP.2016.17

[167] Mohammed Latif Siddiq, Joanna Cecilia da Silva Santos, Ridwanul Hasan Tanvir, Noshin Ulfat, Fahmid Al Rifat, and Vinicius Carvalho Lopes. 2024. Using Large Language Models to Generate JUnit Tests: An Empirical Study. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering, EASE 2024, Salerno, Italy, June 18-21, 2024*. ACM, 313–322. doi:10.1145/3661167.3661216

[168] Benjamin Steenhoek, Michele Tufano, Neel Sundaresan, and Alexey Svyatkovskiy. 2025. Reinforcement Learning from Automatic Feedback for High-Quality Unit Test Generation. In *IEEE/ACM International Workshop on Deep Learning for Testing and Testing for Deep Learning, DeepTest@ICSE 2025, Ottawa, ON, Canada, May 3, 2025*. IEEE, 37–44. doi:10.1109/DEEPTEST66595.2025.00011

[169] André Storhaug and Jingyue Li. 2024. Parameter-Efficient Fine-Tuning of Large Language Models for Unit Test Generation: An Empirical Study. *CoRR* abs/2411.02462 (2024). doi:10.48550/ARXIV.2411.02462

[170] SunwiseAUnit. 2026. SunwiseAUnit. https://www.sunwiseinfo.com/sunwiseaunit.

[171] Yutian Tang, Zhijie Liu, Zhichao Zhou, and Xiapu Luo. 2024. ChatGPT vs SBST: A Comparative Assessment of Unit Test Suite Generation. *IEEE Transactions on Software Engineering* 50, 6 (2024), 1340–1359. doi:10.1109/TSE.2024.3382365

[172] Tree-Sitter. 2026. Tree-Sitter. https://github.com/tree-sitter/tree-sitter.

[173] Michele Tufano, Shao Kun Deng, Neel Sundaresan, and Alexey Svyatkovskiy. 2022. METHODS2TEST: A Dataset of Focal Methods Mapped to Test Cases. In *19th IEEE/ACM International Conference on Mining Software Repositories, MSR 2022, Pittsburgh, PA, USA, May 23-24, 2022*. ACM, 299–303. doi:10.1145/3524842.3528009

[174] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, Shao Kun Deng, and Neel Sundaresan. 2020. Unit Test Case Generation with Transformers. *CoRR* abs/2009.05617 (2020). https://arxiv.org/abs/2009.05617

[175] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, and Neel Sundaresan. 2022. Generating Accurate Assert Statements for Unit Test Cases using Pretrained Transformers. In *IEEE/ACM International Conference on Automation of Software Test, AST@ICSE 2022, Pittsburgh, PA, USA, May 21-22, 2022*. ACM/IEEE, 54–64. doi:10.1145/3524481.3527220

[176] Vsevolod Tymofyeyev and Gordon Fraser. 2022. Search-Based Test Suite Generation for Rust. In *Search-Based Software Engineering - 14th International Symposium, SSBSE 2022, Singapore, November 17-18, 2022, Proceedings (Lecture Notes in Computer Science, Vol. 13711)*. Springer, 3–18. doi:10.1007/978-3-031-21251-2_1

[177] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*. 5998–6008. https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

[178] WALA. 2026. WALA. https://github.com/wala/WALA.

[179] Han Wang, Han Hu, Chunyang Chen, and Burak Turhan. 2024. Chat-like Asserts Prediction with the Support of Large Language Model. *CoRR* abs/2407.21429 (2024). doi:10.48550/ARXIV.2407.21429

[180] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software Testing With Large Language Models: Survey, Landscape, and Vision. *IEEE Transactions on Software Engineering* 50, 4 (2024), 911–936. doi:10.1109/TSE.2024.3368208

[181] Jason Wang, Basem Suleiman, and Muhammad Johan Alibasa. 2025. Automated Unit Test Case Generation: A Systematic Literature Review. *CoRR* abs/2504.20357 (2025). doi:10.48550/ARXIV.2504.20357

[182] Luping Wang, Sheng Chen, Linnan Jiang, Shu Pan, Runze Cai, Sen Yang, and Fei Yang. 2025. Parameter-Efficient Fine-Tuning in Large Language Models: A Survey of Methodologies. *Artificial Intelligence Review* 58, 8 (2025), 227. doi:10.1007/S10462-025-11236-4

[183] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. 2024. A Survey on Large Language Model Based Autonomous Agents. *Frontiers in Computer Science* 18, 6 (2024), 186345. doi:10.1007/S11704-024-40231-1

[184] Siwei Wang, Xue Mao, Ziguang Cao, Yujun Gao, Qucheng Shen, and Chao Peng. 2023. NxtUnit: Automated Unit Test Generation for Go. In *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering, EASE 2023, Oulu, Finland, June 14-16, 2023*. ACM, 176–179. doi:10.1145/3593434.3593443

[185] Taiyan Wang, Ruipeng Wang, Yu Chen, Lu Yu, Zulie Pan, Min Zhang, Huimin Ma, and Jinghua Zheng. 2024. Enhancing Black-box Compiler Option Fuzzing with LLM through Command Feedback. In *35th IEEE International Symposium on Software Reliability Engineering, ISSRE 2024, Tsukuba, Japan, October 28-31, 2024*. IEEE, 319–330. doi:10.1109/ISSRE62328.2024.00039

[186] Wenhan Wang, Chenyuan Yang, Zhijie Wang, Yuheng Huang, Zhaoyang Chu, Da Song, Lingming Zhang, An Ran Chen, and Lei Ma. 2025. TESTEVAL: Benchmarking Large Language Models for Test Case Generation. In *Findings of the Association for Computational Linguistics: NAACL 2025, Albuquerque, New Mexico, USA, April 29 - May 4, 2025 (Findings of ACL, Vol. NAACL 2025)*. Association for Computational Linguistics, 3547–3562. doi:10.18653/V1/2025.FINDINGS-NAACL.197

[187] Yaqing Wang, Subhabrata Mukherjee, Xiaodong Liu, Jing Gao, Ahmed Hassan Awadallah, and Jianfeng Gao. 2022. AdaMix: Mixture-of-Adapter for Parameter-efficient Tuning of Large Language Models. *CoRR* abs/2205.12410 (2022). doi:10.48550/ARXIV.2205.12410

[188] Yue Wang, Weishi Wang, Shafiq R. Joty, and Steven C. H. Hoi. 2021. CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*. Association for Computational Linguistics, 8696–8708. doi:10.18653/V1/2021.EMNLP-MAIN.685

[189] Yibo Wang, Congying Xia, Wenting Zhao, Jiangshu Du, Chunyu Miao, Zhongfen Deng, Philip S. Yu, and Chen Xing. 2025. ProjectTest: A Project-level LLM Unit Test Generation Benchmark and Impact of Error Fixing Mechanisms. *CoRR* abs/2502.06556 (2025). doi:10.48550/ARXIV.2502.06556

[190] Yinjie Wang, Ling Yang, Ye Tian, Ke Shen, and Mengdi Wang. 2025. Co-Evolving LLM Coder and Unit Tester via Reinforcement Learning. *CoRR* abs/2506.03136 (2025). doi:10.48550/ARXIV.2506.03136

[191] Yanlin Wang, Wanjun Zhong, Yanxian Huang, Ensheng Shi, Min Yang, Jiachi Chen, Hui Li, Yuchi Ma, Qianxiang Wang, and Zibin Zheng. 2025. Agents in Software Engineering: Survey, Landscape, and Vision. *Automated Software Engineering* 32, 2 (2025), 70. doi:10.1007/S10515-025-00544-2

[192] Zejun Wang, Kaibo Liu, Ge Li, and Zhi Jin. 2024. HITS: High-Coverage LLM-based Unit Test Generation via Method Slicing. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE 2024, Sacramento, CA, USA, October 27 - November 1, 2024*. ACM, 1258–1268. doi:10.1145/3691620.3695501

[193] Cody Watson, Michele Tufano, Kevin Moran, Gabriele Bavota, and Denys Poshyvanyk. 2020. On Learning Meaningful Assert Statements for Unit Test Cases. In *ICSE '20: 42nd International Conference on Software Engineering, Seoul, South Korea, 27 June - 19 July, 2020*. ACM, 1398–1409. doi:10.1145/3377811.3380429

[194] Yuxiang Wei, Zhe Wang, Jiawei Liu, Yifeng Ding, and Lingming Zhang. 2023. Magicoder: Source Code Is All You Need. *CoRR* abs/2312.02120 (2023). doi:10.48550/ARXIV.2312.02120

[195] Ratnadira Widyasari, Sheng Qin Sim, Camellia Lok, Haodi Qi, Jack Phan, Qijin Tay, Constance Tan, Fiona Wee, Jodie Ethelda Tan, Yuheng Yieh, Brian Goh, Ferdian Thung, Hong Jin Kang, Thong Hoang, David Lo, and Eng Lieh Ouh. 2020. BugsInPy: A Database of Existing Bugs in Python Programs to Enable Controlled Testing and Debugging Studies. In *ESEC/FSE '20: 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual Event, USA, November 8-13, 2020*. ACM, 1556–1560. doi:10.1145/3368089.3417943

[196] Claes Wohlin. 2014. Guidelines for Snowballing in Systematic Literature Studies and A Replication in Software Engineering. In *18th International Conference on Evaluation and Assessment in Software Engineering, EASE '14, London, England, United Kingdom, May 13-14, 2014*. ACM, 38:1–38:10. doi:10.1145/2601248.2601268

[197] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2024. Fuzz4All: Universal Fuzzing with Large Language Models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14-20, 2024*. ACM, 126:1–126:13. doi:10.1145/3597503.3639121

[198] Danni Xiao, Yimeng Guo, Yanhui Li, and Lin Chen. 2024. Optimizing Search-Based Unit Test Generation with Large Language Models: An Empirical Study. In *Proceedings of the 15th Asia-Pacific Symposium on Internetware, Internetware 2024, Macau, SAR, China, July 24-26, 2024*. ACM. doi:10.1145/3671016.3674813

[199] Weimin Xiong, Yiwen Guo, and Hao Chen. 2024. The Program Testing Ability of Large Language Models for Code. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: EMNLP 2024 - Industry Track, Miami, Florida, USA, November 12-16, 2024*. Association for Computational Linguistics, 23–34. doi:10.18653/V1/2024.EMNLP-INDUSTRY.3

[200] Frank F. Xu, Uri Alon, Graham Neubig, and Vincent Josua Hellendoorn. 2022. A Systematic Evaluation of Large Language Models of Code. In *MAPS@PLDI 2022: 6th ACM SIGPLAN International Symposium on Machine Programming, San Diego, CA, USA, 13 June 2022*. ACM, 1–10. doi:10.1145/3520312.3534862

[201] Ruiyang Xu, Jialun Cao, Yaojie Lu, Ming Wen, Hongyu Lin, Xianpei Han, Ben He, Shing-Chi Cheung, and Le Sun. 2025. CRUXEVAL-X: A Benchmark for Multilingual Code Reasoning, Understanding and Execution. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, Vienna, Austria, July 27 - August 1, 2025*. Association for Computational Linguistics, 23762–23779. https://aclanthology.org/2025.acl-long.1158/

[202] Zhiyi Xue, Liangguo Li, Senyue Tian, Xiaohong Chen, Pingping Li, Liangyu Chen, Tingting Jiang, and Min Zhang. 2024. LLM4Fin: Fully Automating LLM-Powered Test Case Generation for FinTech Software Acceptance Testing. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024, Vienna, Austria, September 16-20, 2024*. ACM, 1643–1655. doi:10.1145/3650212.3680388

[203] Boyang Yang, Zijian Cai, Fengling Liu, Bach Le, Lingming Zhang, Tegawendé F. Bissyandé, Yang Liu, and Haoye Tian. 2025. A Survey of LLM-based Automated Program Repair: Taxonomies, Design Paradigms, and Applications. *CoRR* abs/2506.23749 (2025). doi:10.48550/ARXIV.2506.23749

[204] Chen Yang, Junjie Chen, Bin Lin, Jianyi Zhou, and Ziqi Wang. 2024. Enhancing LLM-based Test Generation for Hard-to-Cover Branches via Program Analysis. *CoRR* abs/2404.04966 (2024). doi:10.48550/ARXIV.2404.04966

[205] Chen Yang, Lin Yang, Ziqi Wang, Dong Wang, Jianyi Zhou, and Junjie Chen. 2025. Clarifying Semantics of In-Context Examples for Unit Test Generation. *CoRR* abs/2510.01994 (2025). doi:10.48550/ARXIV.2510.01994

[206] Lin Yang, Chen Yang, Shutao Gao, Weijing Wang, Bo Wang, Qihao Zhu, Xiao Chu, Jianyi Zhou, Guangtai Liang, Qianxiang Wang, and Junjie Chen. 2024. On the Evaluation of Large Language Models in Unit Test Generation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE 2024, Sacramento, CA, USA, October 27 - November 1, 2024*. ACM, 1607–1619. doi:10.1145/3691620.3695529

[207] Ling Yang, Zhaochen Yu, Bin Cui, and Mengdi Wang. 2025. ReasonFlux: Hierarchical LLM Reasoning via Scaling Thought Templates. *CoRR* abs/2502.06772 (2025). doi:10.48550/ARXIV.2502.06772

[208] Ruofan Yang, Xianghua Xu, and Ran Wang. 2025. TestLoter: A logic-driven framework for automated unit test generation and error repair using large language models. *Journal of Computer Languages* 84 (2025), 101348. doi:10.1016/J.COLA.2025.101348

[209] Zhuoying Yang and Lei Wang. 2025. IntelliUnitGen: A Unit Test Case Generation Framework Based on the Integration of Static Analysis and Prompt Learning. *IEEE Access* 13 (2025), 177760–177784. doi:10.1109/ACCESS.2025.3615990

[210] Ahmadreza Saboor Yaraghi, Darren Holden, Nafiseh Kahani, and Lionel C. Briand. 2025. Automated Test Case Repair Using Language Models. *IEEE Transactions on Software Engineering* 51, 4 (2025), 1104–1133. doi:10.1109/TSE.2025.3541166

[211] Xin Yin, Chao Ni, Xinrui Li, Liushan Chen, Guojun Ma, and Xiaohu Yang. 2025. Enhancing LLM's Ability to Generate More Repository-Aware Unit Tests Through Precise Contextual Information Injection. *CoRR* abs/2501.07425 (2025). doi:10.48550/ARXIV.2501.07425

[212] Xin Yin, Chao Ni, Xiaodan Xu, and Xiaohu Yang. 2025. What You See is What You Get: Attention-Based Self-Guided Automatic Unit Test Generation. In *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 - May 6, 2025*. IEEE, 1039–1051. doi:10.1109/ICSE55347.2025.00105

[213] Zhiqiang Yuan, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, Xin Peng, and Yiling Lou. 2024. Evaluating and Improving ChatGPT for Unit Test Generation. *Proc. ACM Softw. Eng.* 1, FSE (2024), 1703–1726. doi:10.1145/3660783

[214] Cen Zhang, Yaowen Zheng, Mingqiang Bai, Yeting Li, Wei Ma, Xiaofei Xie, Yuekang Li, Limin Sun, and Yang Liu. 2024. How Effective Are They? Exploring Large Language Model Based Fuzz Driver Generation. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024, Vienna, Austria, September 16-20, 2024*. ACM, 1223–1235. doi:10.1145/3650212.3680355

[215] He Zhang, Muhammad Ali Babar, and Paolo Tell. 2011. Identifying Relevant Studies in Software Engineering. *Information and Software Technology* 53, 6 (2011), 625–637. doi:10.1016/J.INFSOF.2010.12.010

[216] Junwei Zhang, Xing Hu, Shan Gao, Xin Xia, David Lo, and Shanping Li. 2025. Less Is More: On the Importance of Data Quality for Unit Test Generation. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 1293–1316. doi:10.1145/3715778

[217] Jiyang Zhang, Yu Liu, Pengyu Nie, Junyi Jessy Li, and Milos Gligoric. 2024. Generating Exceptional Behavior Tests with Reasoning Augmented Large Language Models. *CoRR* abs/2405.14619 (2024). doi:10.48550/ARXIV.2405.14619

[218] Jie M. Zhang, Mark Harman, Lei Ma, and Yang Liu. 2022. Machine Learning Testing: Survey, Landscapes and Horizons. *IEEE Transactions on Software Engineering* 48, 2 (2022), 1–36. doi:10.1109/TSE.2019.2962027

[219] Kunpeng Zhang, Shuai Wang, Jitao Han, Xiaogang Zhu, Xian Li, Shaohua Wang, and Sheng Wen. 2025. Your Fix Is My Exploit: Enabling Comprehensive DL Library API Fuzzing with Large Language Models. In *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 - May 6, 2025*. IEEE, 3110–3122. doi:10.1109/ICSE55347.2025.00041

[220] Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. 2024. TinyLlama: An Open-Source Small Language Model. *CoRR* abs/2401.02385 (2024). doi:10.48550/ARXIV.2401.02385
[221] Quanjun Zhang, Chunrong Fang, Siqi Gu, Ye Shang, Zhenyu Chen, and Liang Xiao. 2025. Large Language Models for Unit Testing: A Systematic Literature Review. *CoRR* abs/2506.15227 (2025). doi:10.48550/ARXIV.2506.15227
[222] Quanjun Zhang, Chunrong Fang, Yang Xie, Yaxin Zhang, Yun Yang, Weisong Sun, Shengcheng Yu, and Zhenyu Chen. 2023. A Survey on Large Language Models for Software Engineering. *CoRR* abs/2312.15223 (2023). doi:10.48550/ARXIV.2312.15223
[223] Quanjun Zhang, Chunrong Fang, Yi Zheng, Ruixiang Qian, Shengcheng Yu, Yuan Zhao, Jianyi Zhou, Yun Yang, Tao Zheng, and Zhenyu Chen. 2025. Improving Retrieval-Augmented Deep Assertion Generation via Joint Training. *IEEE Transactions on Software Engineering* 51, 4 (2025), 1232–1247. doi:10.1109/TSE.2025.3545970
[224] Quanjun Zhang, Chunrong Fang, Yi Zheng, Yaxin Zhang, Yuan Zhao, Rubing Huang, Jianyi Zhou, Yun Yang, Tao Zheng, and Zhenyu Chen. 2025. Improving Deep Assertion Generation via Fine-Tuning Retrieval-Augmented Pre-Trained Language Models. *ACM Transactions on Software Engineering and Methodology* 34, 7 (2025), 209:1–209:23. doi:10.1145/3721128
[225] Quanjun Zhang, Ye Shang, Chunrong Fang, Siqi Gu, Jianyi Zhou, and Zhenyu Chen. 2024. TestBench: Evaluating Class-Level Test Case Generation Capability of Large Language Models. *CoRR* abs/2409.17561 (2024). doi:10.48550/ARXIV.2409.17561
[226] Quanjun Zhang, Weifeng Sun, Chunrong Fang, Bowen Yu, Hongyan Li, Meng Yan, Jianyi Zhou, and Zhenyu Chen. 2025. Exploring Automated Assertion Generation via Large Language Models. *ACM Transactions on Software Engineering and Methodology* 34, 3 (2025), 81:1–81:25. doi:10.1145/3699598
[227] Xiaoyu Zhang, Weipeng Jiang, Chao Shen, Qi Li, Qian Wang, Chenhao Lin, and Xiaohong Guan. 2025. Deep Learning Library Testing: Definition, Methods and Challenges. *Comput. Surveys* 57, 7 (2025), 187:1–187:37. doi:10.1145/3716497
[228] Yuwei Zhang, Qingyuan Lu, Kai Liu, Wensheng Dou, Jiaxin Zhu, Li Qian, Chunxi Zhang, Zheng Lin, and Jun Wei. 2025. CITYWALK: Enhancing LLM-Based C++ Unit Test Generation via Project-Dependency Awareness and Language-Specific Knowledge. *CoRR* abs/2501.16155 (2025). doi:10.48550/ARXIV.2501.16155
[229] Yehong Zhang, Jun Wu, and Hui Xu. 2025. RuMono: Fuzz Driver Synthesis for Rust Generic APIs. *ACM Transactions on Software Engineering and Methodology* 34, 6 (2025), 169:1–169:28. doi:10.1145/3709359
[230] Yuanhe Zhang, Zhiquan Yang, Shengyi Pan, and Zhongxin Liu. 2025. Unit Test Update through LLM-Driven Context Collection and Error-Type-Aware Refinement. *CoRR* abs/2509.24419 (2025). doi:10.48550/ARXIV.2509.24419
[231] Ziyin Zhang, Chaoyu Chen, Bingchang Liu, Cong Liao, Zi Gong, Hang Yu, Jianguo Li, and Rui Wang. 2024. Unifying the Perspectives of NLP and Software Engineering: A Survey on Language Models for Code. *Transactions on Machine Learning Research* 2024 (2024). https://openreview.net/forum?id=hkNnGqZnpa
[232] Zhe Zhang, Xingyu Liu, Yuanzhang Lin, Xiang Gao, Hailong Sun, and Yuan Yuan. 2024. LLM-Based Unit Test Generation via Property Retrieval. *CoRR* abs/2410.13542 (2024). doi:10.48550/ARXIV.2410.13542
[233] Zhe Zhang, Xingyu Liu, Yuanzhang Lin, Xiang Gao, Hailong Sun, and Yuan Yuan. 2025. Reference-Based Retrieval-Augmented Unit Test Generation. *ACM Transactions on Software Engineering and Methodology* (2025). doi:10.1145/3765758
[234] Heri Zhao, Jeffrey Hui, Joshua Howland, Nam Nguyen, Siqi Zuo, Andrea Hu, Christopher A. Choquette-Choo, Jingyue Shen, Joe Kelley, Kshitij Bansal, Luke Vilnis, Mateo Wirth, Paul Michel, Peter Choy, Pratik Joshi, Ravin Kumar, Sarmad Hashmi, Shubham Agrawal, Zhitao Gong, Jane Fine, Tris Warkentin, Ale Jakse Hartman, Bin Ni, Kathy Korevec, Kelly Schaefer, and Scott Huffman. 2024. CodeGemma: Open Code Models Based on Gemma. *CoRR* abs/2406.11409 (2024). doi:10.48550/ARXIV.2406.11409
[235] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. 2023. A Survey of Large Language Models. *CoRR* abs/2303.18223 (2023). doi:10.48550/ARXIV.2303.18223
[236] Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Lei Shen, Zihan Wang, Andi Wang, Yang Li, Teng Su, Zhilin Yang, and Jie Tang. 2023. CodeGeeX: A Pre-trained Model for Code Generation with Multilingual Benchmarking on HumanEval-X. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2023, Long Beach, CA, USA, August 6-10, 2023*. ACM, 5673–5684. doi:10.1145/3580305.3599790
[237] Zibin Zheng, Kaiwen Ning, Qingyuan Zhong, Jiachi Chen, Wenqing Chen, Lianghong Guo, Weicheng Wang, and Yanlin Wang. 2025. Towards An Understanding of Large Language Models in Software Engineering Tasks. *Empirical Software Engineering* 30, 2 (2025), 50. doi:10.1007/S10664-024-10602-0
[238] Zhiyuan Zhong, Sinan Wang, Hailong Wang, Shaojin Wen, Hao Guan, Yida Tao, and Yepang Liu. 2024. Multi-Agent Assisted Automatic Test Generation for Java JSON Libraries. *CoRR* abs/2410.09414 (2024). doi:10.48550/ARXIV.2410.09414
[239] Shuaiyu Zhou, Zhengran Zeng, Xiaoling Zhou, Rui Xie, Shikun Zhang, and Wei Ye. 2025. Seed&Steer: Guiding Large Language Models with Compilable Prefix and Branch Signals for Unit Test Generation. *CoRR* abs/2507.17271 (2025). doi:10.48550/ARXIV.2507.17271
[240] Xin Zhou, Sicong Cao, Xiaobing Sun, and David Lo. 2025. Large Language Model for Vulnerability Detection and Repair: Literature Review and the Road Ahead. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 145:1–145:31. doi:10.1145/3708522
[241] Xin Zhou, Bowen Xu, DongGyun Han, Zhou Yang, Junda He, and David Lo. 2023. CCBERT: Self-Supervised Code Change Representation Learning. In *IEEE International Conference on Software Maintenance and Evolution, ICSME 2023, Bogotá, Colombia, October 1-6, 2023*.

IEEE, 182–193. doi:10.1109/ICSME58846.2023.00028

[242] Zhuotong Zhou, Yongzhuo Yang, Susheng Wu, Yiheng Huang, Bihuan Chen, and Xin Peng. 2024. Magneto: A Step-Wise Approach to Exploit Vulnerabilities in Dependent Libraries via LLM-Empowered Directed Fuzzing. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE 2024, Sacramento, CA, USA, October 27 - November 1, 2024*. ACM, 1633–1644. doi:10.1145/3691620.3695531

A Details of the collected papers

Table 10. Details of the collected papers.

ID	Topic	Paper Title	Year	Reference	Venues
1	Class-level Unit Test Generation	TestBench: Evaluating Class-Level Test Case Generation Capability of Large Language Models	2024	[225]	Arxiv
2	Class-level Unit Test Generation	Testart: Improving LLM-based Unit Test via Co-Evolution of Automated Generation and Repair Iteration	2024	[54]	Arxiv
3	Class-level Unit Test Generation	A System for Automated Unit Test Generation using Large Language Models and Assessment of Generated Test Suites	2025	[118]	Arxiv
4	Function-Level Test Generation	Unit Test Case Generation with Transformers and Focal Context	2020	[174]	Arxiv
5	Function-Level Test Generation	A3Test: Assertion-Augmented Automated Test Case Generation	2023	[5]	IST
6	Function-Level Test Generation	CodaMosa: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models	2023	[98]	ICSE
7	Function-Level Test Generation	Parameter-Efficient Fine-Tuning of Large Language Models for Unit Test Generation: An Empirical Study	2024	[169]	Arxiv
8	Function-Level Test Generation	Domain Adaptation for Code Model-Based Unit Test Case Generation	2024	[164]	ISSTA
9	Function-Level Test Generation	What You See Is What You Get: Attention-based Self-guided Automatic Unit Test Generation	2024	[212]	ICSE
10	Function-Level Test Generation	Unit Test Generation using Large Language Models for Unity Game Development	2024	[136]	FSE
11	Function-Level Test Generation	CPP-UT-Bench: Can LLMs Write Complex Unit Tests in C++?	2024	[11]	Arxiv
12	Function-Level Test Generation	LLM4Fin: Fully Automating LLM-Powered Test Case Generation for FinTech Software Acceptance Testing	2024	[202]	ISSTA
13	Function-Level Test Generation	LLM-based Unit Test Generation via Property Retrieval	2024	[232]	Arxiv
14	Function-Level Test Generation	Automated Unit Test Improvement using Large Language Models at Meta	2024	[7]	FSE

Continued on next page

ID	Topic	Paper Title	Year	Reference	Venues
15	Function-Level Test Generation	Advancing Bug Detection in Fastjson2 with Large Language Models Driven Unit Test Generation	2024	[238]	Arxiv
16	Function-Level Test Generation	LLM-TG: Towards Automated Test Case Generation for Processors Using Large Language Models	2024	[33]	ICCD
17	Function-Level Test Generation	HITS: High-coverage LLM-based Unit Test Generation via Method Slicing	2024	[192]	ASE
18	Function-Level Test Generation	Enhancing LLM-based Test Generation for Hard-to-Cover Branches via Program Analysis	2024	[204]	Arxiv
19	Function-Level Test Generation	Large Language Models as Test Case Generators: Performance Evaluation and Enhancement	2024	[101]	Arxiv
20	Function-Level Test Generation	Code-Aware Prompting: A Study of Coverage-Guided Test Generation in Regression Setting using LLM	2024	[153]	TOSEM
21	Function-Level Test Generation	Effective Test Generation using Pre-Trained Large Language Models and Mutation Testing	2024	[30]	IST
22	Function-Level Test Generation	Improving the Readability of Automatically Generated Tests using Large Language Models	2024	[12]	ICST
23	Function-Level Test Generation	Automated Test Generation to Evaluate Tool-Augmented LLMs as Conversational AI Agents	2024	[8]	Arxiv
24	Function-Level Test Generation	A Multi-Agent Approach for REST API Testing with Semantic Graphs and LLM-Driven Inputs	2024	[89]	ICSE
25	Function-Level Test Generation Assertion Oracle Generation Test Evolution	A Large-scale Empirical Study on Fine-tuning Large Language Models for Unit Testing	2024	[160]	ISSTA
26	Function-Level Test Generation	Large-Scale, Independent and Comprehensive Study of the Power of LLMs for Test Case Generation	2024	[133]	Arxiv
27	Function-Level Test Generation	On the Evaluation of Large Language Models in Unit Test Generation	2024	[206]	ASE
28	Function-Level Test Generation	An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation	2024	[158]	TSE
29	Function-Level Test Generation	Optimizing Search-Based Unit Test Generation with Large Language Models: An Empirical Study	2024	[198]	Internetware
30	Function-Level Test Generation	The Program Testing Ability of Large Language Models for Code	2024	[199]	Arxiv

Continued on next page

ID	Topic	Paper Title	Year	Reference	Venues
31	Function-Level Test Generation	Using Large Language Models to Generate JUnit Tests: An Empirical Study	2024	[167]	EASE
32	Function-Level Test Generation	Evaluating and Improving ChatGPT for Unit Test Generation	2024	[213]	FSE
33	Function-Level Test Generation	ChatGPT vs SBST: A Comparative Assessment of Unit Test Suite Generation	2024	[171]	TSE
34	Function-Level Test Generation and Test Evolution	Are We Testing or Being Tested? Exploring the Practical Applications of Large Language Models in Software Testing	2024	[156]	ICST
35	Function-Level Test Generation	Evaluation of Large Language Models for Unit Test Generation	2024	[159]	ASE
36	Function-Level Test Generation	TDD-Bench Verified: Can LLMs Generate Tests for Issues Before They Get Resolved?	2024	[4]	Arxiv
37	Function-Level Test Generation	TESTEVAL: Benchmarking Large Language Models for Test Case Generation	2024	[186]	NAACL
38	Function-Level Test Generation	UniTSyn: A Large-Scale Dataset Capable of Enhancing the Prowess of Large Language Models for Program Testing	2024	[62]	ISSTA
39	Function-Level Test Generation	The Potential of LLMs in Automating Software Testing: From Generation to Reporting	2025	[162]	Arxiv
40	Function-Level Test Generation and Test Evolution	TestLoter: A Logic-Driven Framework for Automated Unit Test Generation and Error Repair Using Large Language Models	2025	[208]	J. Comput. Lang.
41	Function-Level Test Generation	IntelliUnitGen: A Unit Test Case Generation Framework Based on the Integration of Static Analysis and Prompt Learning	2025	[209]	IEEE Access
42	Function-Level Test Generation	STRUT: Structured Seed Case Guided Unit Test Generation for C Programs using LLMs	2025	[109]	ISSTA
43	Function-Level Test Generation	Less Is More: On the Importance of Data Quality for Unit Test Generation	2025	[216]	FSE
44	Function-Level Test Generation	TestGenEval: A Real World Unit Test Generation and Test Completion Benchmark	2025	[78]	ICLR
45	Function-Level Test Generation	Reinforcement Learning from Automatic Feedback for High-Quality Unit Test Generation	2025	[168]	Arxiv
46	Function-Level Test Generation	ASTER: Natural and Multi-language Unit Test Generation with LLMs	2025	[137]	ICSE
47	Function-Level Test Generation	Rug: Turbo LLM for Rust Unit Test Generation	2025	[22]	ICSE
48	Function-Level Test Generation	Test Intention Guided LLM-Based Unit Test Generation	2025	[124]	ICSE
49	Function-Level Test Generation	Reference-Based Retrieval-Augmented Unit Test Generation	2025	[233]	TOSEM

Continued on next page

ID	Topic	Paper Title	Year	Reference	Venues
50	Function-Level Test Generation	Enhancing LLM's Ability to Generate More Repository-Aware Unit Tests Through Precise Contextual Information Injection	2025	[211]	Arxiv
51	Function-Level Test Generation	CITYWALK: Enhancing LLM-Based C++ Unit Test Generation via Project-Dependency Awareness and Language-Specific Knowledge	2025	[228]	TOSEM
52	Function-Level Test Generation	Learning to Generate Unit Tests for Automated Debugging	2025	[142]	Arxiv
53	Project-Level Test Generation	ProjectTest: A Project-level LLM Unit Test Generation Benchmark and Impact of Error Fixing Mechanisms	2025	[189]	Arxiv
54	Function-Level Test Generation	LLM-based Unit Test Generation for Dynamically-Typed Programs	2025	[115]	Arxiv
55	Function-Level Test Generation	Boosting Rust Unit Test Coverage through Hybrid Program Analysis and Large Language Models	2025	[25]	Arxiv
56	Function-Level Test Generation	Co-Evolving LLM Coder and Unit Tester via Reinforcement Learning	2025	[190]	Arxiv
57	Function-Level Test Generation	Precisely Detecting Python Type Errors via LLM-based Unit Test Generation	2025	[21]	Arxiv
58	Function-Level Test Generation	Seed&Steer: Guiding Large Language Models with Compilable Prefix and Branch Signals for Unit Test Generation	2025	[239]	Arxiv
59	Function-Level Test Generation	Learning to Generate Unit Test via Adversarial Reinforcement Learning	2025	[97]	Arxiv
60	Function-Level Test Generation	Navigating the Labyrinth: Path-Sensitive Unit Test Generation with Large Language Models	2025	[106]	Arxiv
61	Function-Level Test Generation	Clarifying Semantics of In-Context Examples for Unit Test Generation	2025	[205]	ASE
62	Function-Level Test Generation	LSPRAG: LSP-Guided RAG for Language-Agnostic Real-Time Unit Test Generation	2025	[51]	ICSE
63	Function-Level Test Generation	KTester: Leveraging Domain and Testing Knowledge for More Effective LLM-based Test Generation	2025	[99]	Arxiv
64	Function-Level Test Generation	LLM-Powered Test Case Generation for Detecting Bugs in Plausible Programs	2025	[112]	Arxiv
65	Function-Level Test Generation	Vulnerability-Triggering Test Case Generation from Third-Party Libraries	2025	[47]	Arxiv
66	Function-Level Test Generation	The Prompt Alchemist: Automated LLM-Tailored Prompt Optimization for Test Case Generation	2025	[46]	Arxiv
67	Function-Level Test Generation	Issue2Test: Generating Reproducing Test Cases from Issue Reports	2025	[125]	Arxiv

Continued on next page

ID	Topic	Paper Title	Year	Reference	Venues
68	Project-Level Test Generation	Intention-Driven Generation of Project-Specific Test Cases	2025	[144]	Arxiv
69	Function-Level Test Generation	ATGen: Adversarial Reinforcement Learning for Test Case Generation	2025	[102]	Arxiv
70	Function-Level Test Generation	Automated Generation of Issue-Reproducing Tests by Combining LLMs and Search-Based Testing	2025	[90]	Arxiv
71	Function-Level Test Generation	Klear-CodeTest: Scalable Test Case Generation for Code Reinforcement Learning	2025	[45]	Arxiv
72	Function-Level Test Generation	TestWeaver: Execution-aware, Feedback-driven Regression Testing Generation with Large Language Models	2025	[95]	Arxiv
73	Function-Level Test Generation	Mutation-Guided LLM-based Test Generation at Meta	2025	[59]	FSE
74	Function-Level Test Generation	Retrieval-Augmented Test Generation: How Far Are We?	2025	[163]	Arxiv
75	Function-Level Test Generation	What Inputs Drive Effective LLM-based Unit Test Generation?	2025	[1]	IEEE Software
76	Function-Level Test Generation	CoverUp: Effective High Coverage Test Generation for Python	2025	[141]	FSE
77	Function-Level Test Generation	Unify and Triumph: Polyglot, Diverse, and Self-Consistent Generation of Unit Tests with LLMs	2025	[87]	Arxiv
78	Function-Level Test Generation	TestForge: Feedback-Driven, Agentic Test Suite Generation	2025	[77]	Arxiv
79	Function-Level Test Generation	LLM Test Generation via Iterative Hybrid Program Analysis	2025	[55]	Arxiv
80	Function-Level Test Generation	Otter: Generating Tests from Issues to Validate SWE Patches	2025	[3]	ICML
81	Function-Level Test Generation	Boosting Unit Test Generation via Structure-Aware Fine-Tuning of Pre-Trained Model	2025	[145]	IST
82	Function-Level Test Generation	Harnessing Large Language Models for Automated Software Testing: A Leap Towards Scalable Test Case Generation	2025	[151]	Electronics
83	Function-Level Test Generation	LLM-enhanced Evolutionary Test Generation for Untyped Languages	2025	[228]	ASE Journal
84	Function-Level Test Generation	CUBETESTERAI: Automated JUnit Test Generation using the LLaMA Model	2025	[52]	ICST
85	Function-Level Test Generation	AgentTester: An LLM-Based Tool for Unit Test Generation with Automatically Generated Prompts	2025	[18]	ICIC
86	Test Input Generation	Enriching Automatic Test Case Generation by Extracting Relevant Test Inputs From Bug Reports	2025	[134]	ESE

Continued on next page

ID	Topic	Paper Title	Year	Reference	Venues
87	Test Input Generation	Nuances are the Key: Unlocking ChatGPT to Find Failure-Inducing Tests with Differential Prompting	2023	[104]	ASE
88	Test Input Generation	Large Language Models are Few-shot Testers: Exploring LLM-based General Bug Reproduction	2023	[85]	ICSE
89	Test Input Generation	Towards Understanding the Effectiveness of Large Language Models on Directed Test Input Generation	2024	[81]	ASE
90	Test Input Generation	LLM-Powered Test Case Generation for Detecting Tricky Bugs	2024	[114]	Arxiv
91	Whole Oracle Generation	TOGA: A Neural Method for Test Oracle Generation	2022	[38]	ICSE
92	Whole Oracle Generation	Towards More Realistic Evaluation for Neural Test Oracle Generation	2023	[116]	ISSTA
93	Whole Oracle Generation	Neural-Based Test Oracle Generation: A Large-Scale Evaluation and Lessons Learned	2023	[66]	FSE
94	Whole Oracle Generation	ChatAssert: LLM-based Test Oracle Generation with External Tools Assistance	2024	[60]	TSE
95	Whole Oracle Generation	TOGLL: Correct and Strong Test Oracle Generation with LLMs	2024	[65]	ICSE
96	Whole Oracle Generation	Do LLMs Generate Test Oracles That Capture the Actual or the Expected Program Behaviour?	2024	[92]	Arxiv
97	Whole Oracle Generation	AugmenTest: Enhancing Tests with LLM-Driven Oracles	2025	[86]	ICST
98	Whole Oracle Generation	Doc2OracLL: Investigating the Impact of Documentation on LLM-based Test Oracle Generation	2025	[67]	FSE
99	Assertion Oracle Generation	Generating Accurate Assert Statements for Unit Test Cases using Pretrained Transformers	2022	[175]	AST
100	Assertion Oracle Generation	Retrieval-Based Prompt Selection for Code-Related Few-Shot Learning	2023	[126]	ICSE
101	Assertion Oracle Generation	Chat-like Asserts Prediction with the Support of Large Language Model	2024	[179]	Arxiv
102	Assertion Oracle Generation	Improving Deep Assertion Generation via Fine-Tuning Retrieval-Augmented Pre-Trained Language Models	2025	[224]	TOSEM
103	Assertion Oracle Generation	Exploring Automated Assertion Generation via Large Language Models	2025	[226]	TOSEM
104	Assertion Oracle Generation	Improving Retrieval-Augmented Deep Assertion Generation via Joint Training	2025	[223]	TSE
105	Assertion Oracle Generation	Assertion Messages with Large Language Models (LLMs) for Code	2025	[6]	Arxiv

Continued on next page

ID	Topic	Paper Title	Year	Reference	Venues
106	Assertion Oracle Generation	AsserT5: Test Assertion Generation Using a Fine-Tuned Code Language Model	2025	[143]	AST
107	Assertion Oracle Generation	AssertFix: Empowering Automated Assertion Fix via Large Language Models	2025	[121]	Arxiv
108	Exception Oracle Generation	Generating Exceptional Behavior Tests with Reasoning Augmented Large Language Models	2024	[217]	Arxiv
109	Test Evolution	Identify and Update Test Cases When Production Code Changes: A Transformer-Based Approach	2023	[72]	ASE
110	Test Evolution	Fix the Tests: Augmenting LLMs to Repair Test Cases with Static Collector and Neural Reranker	2024	[111]	ISSRE
111	Test Evolution	Leveraging Large Language Models for Enhancing the Understandability of Generated Unit Tests	2024	[32]	ICSE
112	Test Evolution	Automated Test Case Repair Using Language Models	2025	[210]	TSE
113	Test Evolution	UTFix: Change Aware Unit Test Repairing using LLM	2025	[149]	OOPSLA
114	Test Evolution	Automated Unit Test Refactoring	2025	[48]	FSE
115	Test Evolution	YATE: The Role of Test Repair in LLM-Based Unit Test Generation	2025	[93]	Arxiv
116	Test Evolution	Unit Test Update through LLM-Driven Context Collection and Error-Type-Aware Refinement	2025	[230]	ASE

B Details of replication packages for each study

Table 11. Details of replication packages for each study.

ID	Paper	Year	Replication Package
1	TestBench [225]	2024	https://github.com/iSEngLab/TestBench
2	Testart [54]	2024	https://github.com/sikygu/TestART
3	Lops et al. [118]	2025	https://anonymous.4open.science/r/classes2test
4	AthenaTest [174]	2020	None
5	A3Test [5]	2023	http://github.com/awsm-research/a3test
6	CodaMosa [98]	2023	https://github.com/microsoft/codamosa
7	Storhaug et al. [169]	2024	https://github.com/andstor/peft-unit-test-generation-replication-package
8	Shin et al. [164]	2024	https://github.com/shinjh0849/unit_tc_generation
9	AUGER [212]	2024	https://github.com/vinci-grape/AUGER
10	Paduraru [136]	2024	https://github.com/unibuc-cs/GameTestingWithLLMs
11	CPP-UT-Bench [11]	2024	https://huggingface.co/datasets/Nutanix/CPP-UNITTEST-BENCH

Continued on next page

ID	Paper	Year	Replication Package
12	LLM4Fin [202]	2024	https://github.com/13luoyu/intelligent-test
13	APT [232]	2024	None
14	TestGen-LLM [7]	2024	None
15	JSONTestGen [238]	2024	None
16	LLM-TG [33]	2024	https://github.com/LLM-TGIPrompt_Library
17	HITS [192]	2024	https://anonymous.4open.science/r/SlicePromptTest4J-6CF1/
18	TELPA [204]	2024	None
19	TestChain [101]	2024	None
20	SymPrompt [153]	2024	None
21	MuTAP [30]	2024	https://github.com/ExpertiseModel/MuTAP
22	Biagiola et al. [12]	2024	https://github.com/GhislottiGianluca/readability-llms
23	ALMITA [8]	2024	https://github.com/zendesk/almita-dataset
24	AutoRestTest [89]	2024	https://github.com/selab-gatech/AutoRestTest/
25	Shang et al. [160]	2024	https://github.com/iSEngLab/LLM4UT_Empirical
26	Ouédraogo et al. [133]	2024	https://anonymous.4open.science/r/LLM4TS-0F76/
27	Yang et al. [206]	2024	None
28	TestPilot [158]	2024	https://doi.org/10.6084/m9.figshare.23653371
29	Xiao et al. [198]	2024	None
30	Xiong et al. [199]	2024	https://github.com/WeiminXiong/TestingLLM
31	Siddiq et al. [167]	2024	https://doi.org/10.5281/zenodo.10530787
32	ChatTester [213]	2024	https://github.com/FudanSELab/ChatTester/tree/main
33	Tang et al. [171]	2024	https://sites.google.com/view/chatgpt-sbst
34	Santos et al. [156]	2024	https://figshare.com/s/f4209b4e7ebd1ec10ce0
35	Konuk et al. [159]	2024	None
36	TDD-Bench Verified [4]	2024	https://github.com/IBM/TDD-Bench-Verified
37	TestEval [186]	2024	https://github.com/LLM4SoftwareTesting/TestEval
38	UniTSyn [62]	2024	None
39	Sherifi et al. [162]	2025	None
40	TestLoter [208]	2025	None
41	IntelliUnitGen [209]	2025	https://github.com/yangzy0115/IntelliUnitGen
42	STRUT [109]	2025	https://anonymous.4open.science/r/STRUT-C
43	TestClean [216]	2025	https://doi.org/10.5281/zenodo.13767074
44	TestGenEval [78]	2025	https://figshare.com/s/51171ae97cd21d233d4f
45	RLSQM [168]	2025	https://doi.org/10.6084/m9.figshare.25983166
46	ASTER [137]	2025	https://github.com/aster-test-generation/aster
47	Rug [22]	2025	None
48	IntUT [124]	2025	None
49	PAGTest [233]	2025	https://github.com/PAGTest-project/PAGTest
50	RATester [211]	2025	None
51	CityWalk [228]	2025	https://zenodo.org/records/14022506
52	UTDebug [142]	2025	https://github.com/archiki/UTGenDebug
53	ProjectTest [189]	2025	https://github.com/YiboWANG214/ProjectTest
54	TypeTest [115]	2025	None
55	PALM [25]	2025	https://anonymous.4open.science/r/PALM-F612
56	CURE [190]	2025	https://github.com/Gen-Verse/CURE
57	rTED [21]	2025	None
58	Seed&Steer [239]	2025	https://github.com/monster29000/SeedandSteer
59	UTRL [97]	2025	None

Continued on next page

ID	Paper	Year	Replication Package
60	JUnitGenie [106]	2025	https://github.com/Dianshu-Liao/JUnitGenie
61	CLAST [205]	2025	https://github.com/chenyangyc/CLAST
62	LSPRAG [51]	2025	https://thu-wingtecher.github.io/LSPRAG
63	KTester [99]	2025	https://github.com/SYSUSELab/KTester
64	TrickCatcher [112]	2025	https://github.com/RinCloud/ TrickCatcher
65	VULEUT [47]	2025	https://anonymous.4open.science/r/VulEUT-38B3
66	MAPS [46]	2025	https://zenodo.org/records/14287744
67	Issue2Test [125]	2025	None
68	IntentionTest [144]	2025	https://sites.google.com/view/domain-specific-tester/home
69	ATGen [102]	2025	None
70	BLAST [90]	2025	https://doi.org/10.5281/zenodo.16949042
71	Klear-CodeTest [45]	2025	https://github.com/Kwai-Klear/CodeTest
72	TestWeaver [95]	2025	https://test-weaver.site
73	ACH [59]	2025	None
74	Shin et al. [163]	2025	https://github.com/shinjh0849/api_guided_testgen
75	Agarwal et al. [1]	2025	None
76	CoverUp [141]	2025	https://github.com/plasma-umass/coverup
77	Polytest [87]	2025	https://anonymous.4open.science/r/Polytest-C96C/
78	TestForge [77]	2025	https://anonymous.4open.science/r/OpenHands-7E28/
79	Panta [55]	2025	https://github.com/PANTA-TestAutomation/Panta
80	Otter [3]	2025	None
81	APPT [145]	2025	https://github.com/SCAU-CWB/SF-UTG
82	Rehan et al. [151]	2025	https://github.com/Shaheer-Rehan/Llama-2-for-Software-Testing
83	Yang et al. [228]	2025	None
84	CUBETESTERAI [52]	2025	None
85	AgentTester [18]	2025	None
86	BRMiner [134]	2025	https://anonymous.4open.science/r/BRMiner-C853/
87	Li et al. [104]	2023	https://differentialprompting.github.io/
88	LIBRO [85]	2023	https://anonymous.4open.science/r/llm-testgen-artifact-2753
89	Jiang et al. [81]	2024	https://github.com/CGCL-codes/PathEval
90	TrickCatcher [114]	2024	https://github.com/RinCloud/TrickCatcher
91	TOGA [38]	2022	https://github.com/microsoft/toga
92	TEval-plus [116]	2023	https://github.com/Tbabm/TEval-plus
93	TOGA [66]	2023	None
94	ChatAssert [60]	2024	https://github.com/ncsu-swat/chatassert
95	TOGLL [65]	2024	None
96	Konstantinou et al. [92]	2024	https://doi.org/10.5281/zenodo.13867480
97	AugmenTest [86]	2025	https://doi.org/10.5281/zenodo.13881826
98	Doc2OracLL [67]	2025	None
99	Tufano et al. [175]	2022	None
100	CEDAR [126]	2023	https://github.com/prompt-learning/cedar
101	CLAP [179]	2024	https://github.com/freddiewanah/CLAP
102	RetriGen [224]	2025	https://github.com/iSEngLab/RetriGen
103	Zhang et al. [226]	2025	https://github.com/iSEngLab/LLM4AG
104	AG-RAG [223]	2025	https://github.com/iSEngLab/AG-RAG
105	Aljohani et al [6]	2025	https://doi.org/10.5281/zenodo.15293133
106	AsserT5 [143]	2025	https://doi.org/10.5281/zenodo.14703162
107	AssertFix [121]	2025	https://anonymous.4open.science/AssertFix-DB3F

Continued on next page

ID	Paper	Year	Replication Package
108	exLóng [217]	2024	https://github.com/EngineeringSoftware/exLong
109	CEPROT [72]	2023	https://github.com/CEPROTest/CEPROT.git
110	SynTeR [111]	2024	https://github.com/nonsense-j/SynTeR
111	UTGen [32]	2024	https://doi.org/10.5281/zenodo.13329464
112	TaRGet [210]	2025	https://github.com/Ahmadreza-SY/TaRGet
113	UTFix [149]	2025	https://sites.google.com/view/utfix
114	UTRefactor [48]	2025	https://github.com/testmigrator/testsmellrefactoring
115	YATE [93]	2025	None
116	TESTUPDATER [230]	2025	https://github.com/ZJU-CTAG/TestUpdater

Received 1 October 2025; revised 20 January 2026; accepted 26 February 2026